

Refactoring: aperfeiçoando o design de código existente

Rodrigo Martins Pagliares
pagliares@bcc.unifal-mg.edu.br

última atualização: 27/11/2024

Apresentação

Rodrigo Martins Pagliares

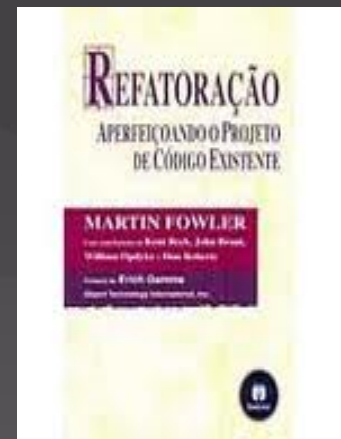
- Professor do Curso de Bacharelado em Ciência da Computação – UNIFAL–MG.
- Doutor em Ciências – ITA – 2017
- Mestre em Ciência da Computação – UFSC - 2002
- Bacharel em Ciência da Computação – UFOP – 1999.
- E-mail: pagliares@bcc.unifal-mg.edu.br

Livro Texto

Livro-Texto

Refatoração – Aperfeiçoando o Projeto de Código Existente.

Martin Fowler, Kent Beck, John Brant, William Opdyke,
Don Roberts



Código fonte

 **refactoring-hands-on** Public

 Pin

 Unwatch 1 ▾

 Fork 0 ▾

 Star 0 ▾

 main ▾

 1 Branch

 0 Tags

Q Go to file

t

+

<> Code ▾

 pagliares Update README.md

5d67133 · 2 years ago 88 Commits

Source_Code	Refactor the code replacing contants with Java Enu...	2 years ago
UML_Diagrams	Create UML Class Diagram for example 12	2 years ago
.gitignore	Exclude target directory from version control	2 years ago
README.md	Update README.md	2 years ago

README

Refactoring hands-on

Repository with source code of examples and exercises used in the refactoring hands-on taught by Rodrigo Martins Pagliares at UNIFAL-MG. There in total 15 Maven projects in this repository (folder Source_Code) used as examples during the hands-on and 2 Maven Projects with solution for the exercies. The examples are based on Martin Fowler's seminal book "Refactoring - Improving the design of existing c

About

No description, website, or topics provided.

 Readme

 Activity

 0 stars

 1 watching

 0 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors 2

<https://github.com/pagliares/refactoring-hands-on>

Observação

- O tradutor do livro optou por traduzir o código fonte dos exemplos
 - Por exemplo, `getPriceCode()` se tornou `lerCódigoPreço()`, incluindo o acento.
 - `setPriceCode(...)` se tornou `gravarCódigoPreço(...)`
- Os diagramas também foram traduzidos desta forma.
- Para ficar consistente, optei por manter os códigos nos slides de acordo com a tradução
 - Todavia, não recomendo traduzir nomes padronizados como **get/set** e muito menos incluir acentos em código fonte !

Agenda

- Capítulo 1: Refatoração, um primeiro exemplo.
- Capítulo 2: Princípios da refatoração.
- Capítulo 3: Os 22 “Bad Smells” no Código.
- Capítulo 4: Construindo testes.
- Capítulo 5: Em direção a um catálogo de refatorações.
- Capítulo 6: Compondo métodos.
- Capítulo 7: Movendo funcionalidades entre objetos.
- Capítulo 8: Organizando dados.
- Capítulo 9: Simplificando expressões condicionais.

Agenda

Capítulo 10: Tornando mais simples as chamadas de métodos.

Capítulo 11: Lidando com generalização.

Capítulo 12: Grandes refatorações.

Capítulo 13: Refatoração, reuso e realidade.

Capítulo 14: Ferramentas para refatoração.

Capítulo 15: Juntando tudo.

Extra: Refatorações no NetBeans, Eclipse e IntelliJ

Refatoração, um primeiro exemplo

Mini-Agenda

- **Versão 1:** Requisitos e solução inicial.
 - mudança de requisitos e oportunidade de refatoração.
 - refatoração e testes automatizados
- **Versão 2:** Extrair Método
- **Versão 3:** Mover Método
- **Versão 4:** Substituir Variável Temporária por Consulta
- **Versão 5:** Novamente Extrair Método e Mover Método

Mini-Agenda

- **Versão 6:** Novamente Substituir Variável Temporária por Consulta
- **Versão 7:** Atendendo à mudança de Requisitos com o método *contaHtml*
- Mudança de Requisitos e Oportunidade de Refatoração
- **Versão 8:** Novamente, Mover Método
- **Versão 9:** Herança e o Design Pattern State

Mini-Agenda

- Exercício extra #1: Novos requisitos
- Exercício extra #2 : Herança
- Exercício extra #3: um objeto não pode mudar sua classe
- Considerações finais

Não deixe de imaginar o exemplo deste capítulo
no contexto de um sistema muito maior !

Versão 1 : Requisitos e solução inicial

- Programa para calcular e imprimir o valor devido por um cliente em uma vídeo locadora no momento de devolução dos filmes.
- O programa recebe como entrada quais filmes o cliente alugou e por quanto tempo. Ele então calcula o valor devido
 - Com base no tipo de filme e tempo de locação

Versão 1 : Requisitos e solução inicial

- Há três tipos de filmes: **normais**, infantis e **lançamentos**.
 - Atualmente os preços de locação são:
 - R\$ 2,00 para filmes normais com direito a ficar com o filme por dois dias
 - R\$ 3,00 para lançamentos com direito a ficar com o filme por 1 dia
 - R\$ 1,50 para filmes infantis com direito a ficar com o filme por 3 dias

Versão 1 : Requisitos e solução inicial

- As multas são calculadas da seguinte forma:
 - Filmes normais
 - R\$ 1,50 de multa, por dia de atraso caso seja entregue **após 2 dias**.
 - Lançamentos
 - R\$ 3,00 de multa, por dia de atraso caso seja entregue **após 1 dia**.
 - Infantis
 - R\$ 1,50 de multa, por dia de atraso caso seja entregue **após 3 dias**.

Versão 1 : Requisitos e solução inicial

- Além de calcular os custos, o programa também calcula os pontos do locador frequente para possíveis descontos no futuro.
 - Cada filme alugado conta 1 ponto
 - 1 ponto adicional é fornecido caso o filme alugado seja um lançamento e o número de dias locado seja maior que 1

Versão 1 : Modelo do Domínio

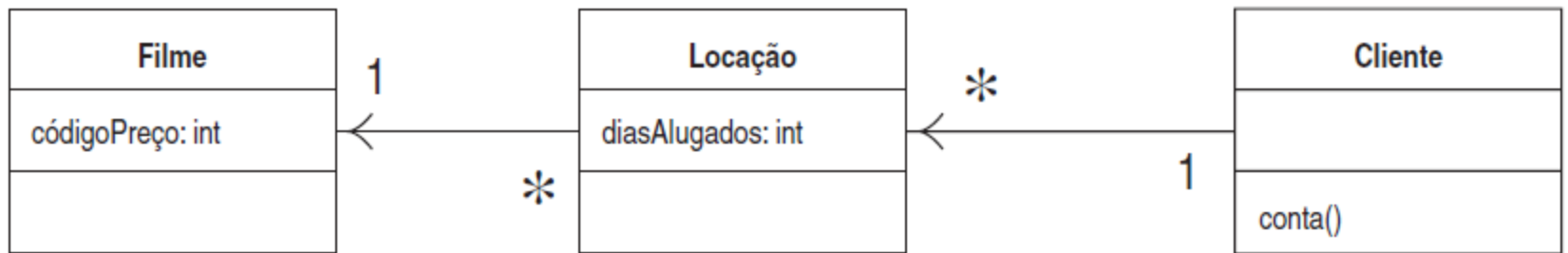
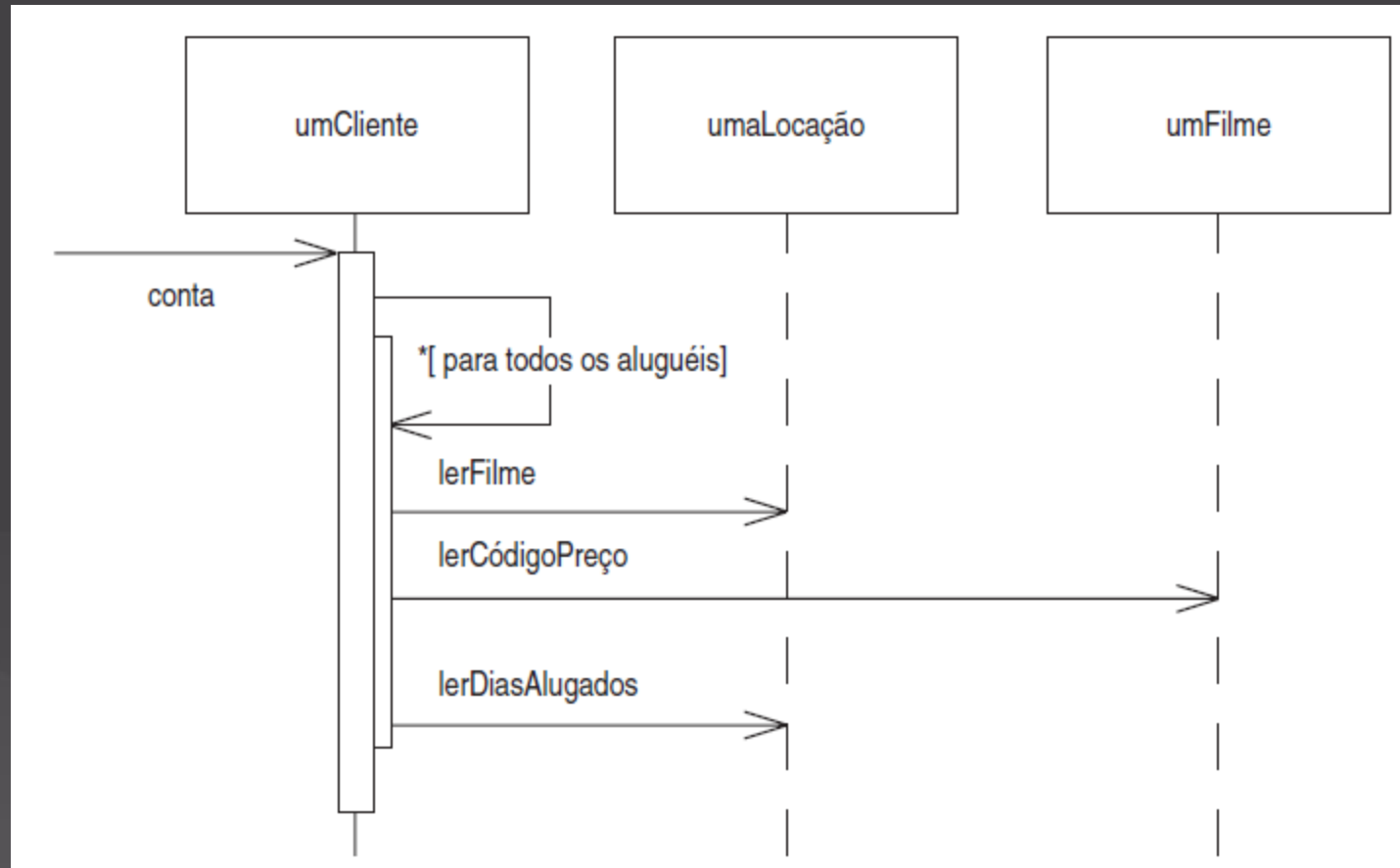


Figura 1.1 Diagrama de classes das classes iniciais. Apenas as características mais importantes estão mostradas. A notação é UML [Fowler, UML].

Versão 1: Diagrama de Sequência



Versão 1 : possível saída

Registro de Locacao para Rodrigo Martins Pagliares

Encontros e Desencontros	6.5
--------------------------	-----

Meia-Noite em Paris	9.0
---------------------	-----

Paris, Texas	3.5
--------------	-----

Bravura Indômita	3.0
------------------	-----

Shrek	3.0
-------	-----

Total devido, R\$ 25.0

Pontos adquiridos na locacao : 6

Versão 1: Exercício

- Você tem 90min para implementar uma solução para os requisitos descritos.
 - Os nomes de classes, métodos e variáveis devem estar de acordo com os diagramas de classe e sequencia exibidos.
 - Tente criar um teste automatizado com o **JUnit** para verificar a saída gerada com a esperada
 - assertEquals()
 - Objetivo é perceber a dificuldade de se automatizar um teste sobre um código mal projetado.

Versão 1: classe Filme (1/2)

```
public class Filme {  
    public static final int  NORMAL = 0; // R$ 2,00  
    public static final int  LANCAMENTO_NOVO = 1; // R$ 3,00  
    public static final int  INFANTIL = 2; // R$ 1,50  
  
    private String _titulo;  
    private int _codigoPreco; // codigoPreco contem uma das constantes acima  
  
    public Filme(String titulo, int codigoPreco) {  
        this._titulo = titulo;  
        this._codigoPreco = codigoPreco;  
    }  
}
```

Versão 1: classe Filme (2/2)

```
public int lerCodigoPreco() {  
    return _codigoPreco;  
}
```

```
public void gravarCodigoPreco(int codigoPreco) {  
    this._codigoPreco = codigoPreco;  
}
```

```
public String lerTitulo (){  
    return _titulo;  
}  
}
```

Versão 1 : Classe Locacao

```
public class Locacao {  
    private Filme _filme;  
    private int _diasAlugados;  
  
    public Locacao(Filme filme, int diasAlugados) {  
        this._filme = filme;  
        this._diasAlugados = diasAlugados;  
    }  
  
    public int lerDiasAlugados() {  
        return _diasAlugados;  
    }  
  
    public Filme lerFilme() {  
        return _filme;  
    }  
}
```

Versão 1 : Classe Cliente

```
public class Cliente {  
    private String _nome;  
    private Vector _locacoes = new Vector();  
  
    public Cliente(String nome) {  
        this._nome = nome;  
    }  
    public void adicionarLocacao(Locacao locacao) {  
        _locacoes.addElement(locacao);  
    }  
    public String lerNome() {  
        return _nome;  
    }  
    public String conta() { // PRÓXIMOS SLIDES !!
```

Versão 1: Método conta() – Parte 1/5

```
public String conta() {  
    double quantiaTotal = 0;  
    int pontosLocadorFrequente = 0;  
    Enumeration locacoes = _locacoes.elements();  
    String resultado = "Registro de Locacao para " + lerNome() + "\n";  
    while (locacoes.hasMoreElements()) {  
        double estaQuantia = 0;  
        Locacao cada = (Locacao) locacoes.nextElement();
```

Versão 1: Método conta() – Parte 2/5

// Valor da locacao para cada filme dentro do Vector

```
switch (cada.lerFilme().lerCodigoPreco()) {
```

```
    case Filme.NORMAL:
```

```
        estaQuantia += 2; // R$ 2,00 por filme normal alugado
```

```
        // Multa de 1.50 por dia de atraso caso seja entregue apos 2 dias.
```

```
        if (cada.lerDiasAlugados() > 2) {
```

```
            estaQuantia += (cada.lerDiasAlugados() - 2) * 1.5;
```

```
        }
```

```
        break;
```

```
        // R$ 3,00 a diaria para lancamentos por filme alugado
```

```
    case Filme.LANCAMENTO_NOVO:
```

```
        estaQuantia += cada.lerDiasAlugados() * 3;
```

```
        break;
```

Versão 1: Método conta() – Parte 3/5

case Filme.INFANTIL:

estaQuantia += 1.5; // R\$ 1,50 por 3 dias por filme infantil alugado

if (cada.lerDiasAlugados() > 3) {

 // Multa de 1,50 por dia de atraso caso seja entregue apos 3 dias.

 estaQuantia += (cada.lerDiasAlugados() - 3) * 1.5;

}

break;

}

Versão 1: Método conta() – Parte 4/5

```
pontosLocadorFrequente++;
```

```
    // Ponto adicional para o locatario para cada filme lancamento
```

```
    if ((cada.lerFilme().lerCodigoPreco() == Filme.LANCAMENTO_NOVO)
        && cada.lerDiasAlugados() > 1) {
        pontosLocadorFrequente++;
    }
```

```
    //armazena o valor do filme para cada filme locado
```

```
    resultado += "\t" + cada.lerFilme().lerTitulo() + "\t"
        + String.valueOf(estaQuantia) + "\n";
    quantiaTotal += estaQuantia; // total da locacao
```

Versão 1: Método conta() – Parte 5/5

//adiciona o rodape

```
    resultado += "Total devido, R$ " + String.valueOf(quantiaTotal) + "\n";  
    resultado += "Pontos adquiridos na locacao : " +  
                String.valueOf(pontosLocadorFrequente);  
    return resultado;
```

```
}
```

```
}
```

Versão 1: Senso crítico

1. O livro foi escrito em 1999. De lá para cá, a linguagem Java foi aperfeiçoada. Quais destes aperfeiçoamentos você usaria hoje para tornar o código mais legível e eficiente?
2. O modelo de domínio descrito no exemplo prevê 1 filme associado a cada locação. Argumente os prós e contras desta decisão de modelagem.

Versão 1: Senso crítico (cont.)

3. Algumas impressões a respeito do projeto desse programa :

- Mal projetado e uso ruim da OO.
- Para um programa simples, nenhum problema.
 - Não há nada errado com um programa simples com design ruim.
- Mas caso esse seja um fragmento representativo de um sistema mais complexo a história muda.
 - Método conta na classe Cliente muito longo.
 - Muito do que ele faz deveria ser feito pelas outras classes.

Mudança de requisitos e oportunidade de refatoração

- Requisitos mudam e sistemas são alterados por pessoas !
 - Um sistema mal projetado é difícil de ser alterado.
 - Pois não é fácil descobrir onde as alterações são necessárias.
- Se for difícil descobrir o que alterar
 - há uma grande chance de que o programador cometa um erro e introduza falhas.

Mudança de requisitos e oportunidade de refatoração

- Produzir uma versão *HTML* da **conta**
- As **classificações** dos filmes serão alteradas em breve
 - Junto com as **regras para estabelecimento de preços** e definição de pontos para locatários frequentes.

Mudança de requisitos e oportunidade de refatoração

Abordagem para solução:

- Reuso impossível de qualquer parte do método *conta()*
 - **Alternativa:** Escrever um novo método que duplique muito do comportamento do antigo método.
 - Crtl-C, Crtl-V

Mudança de requisitos e oportunidade de refatoração

O problema com **copiar** e colar código aparece quando você tem que alterar este código mais tarde.

Mudança de requisitos e oportunidade de refatoração

Se você estiver escrevendo um programa que não espera alterar, então copiar e colar não trará problemas.

Se o programa tiver vida longa e for **provável que sofra alterações**, então recortar e colar é uma ameaça.

Mudança de requisitos e oportunidade de refatoração

- O método `conta()` deve ser alterado para lidar com as mudanças nas regras de classificação e cobrança.
 - Devido à duplicação, as mudanças devem ser consistentes.
 - Atividade **propensa a erros** pelo programador.

Mudança de requisitos e oportunidade de refatoração

O medo de alterar o programa prevalece.

“se não está
estragado, não conserte.”

Mudança de requisitos e oportunidade de refatoração

O programa pode não estar estragado, mas tem problemas.

Está dificultando sua vida, porque você acha difícil fazer as mudanças que seus usuários querem.

É aí que a **refatoração** entra !!!

*Quando você tem que acrescentar uma característica a um programa e o código não está estruturado de uma forma adequada, **primeiro refatore** o programa para facilitar o acréscimo e só depois faça a adição.*

Mudança de requisitos e oportunidade de refatoração

Assim, não iremos duplicar o método `conta()` para uma **versão HTML**.

Primeiramente vamos **refatorar** o método existente para depois atender ao novo requisito !

O objetivo é tornar mais fácil escrever o método `conta()` em **HTML**, com muito menos **duplicação** de código.

Refatoração e testes

- Criar um conjunto de testes para aquele trecho de código.
 - Os testes são essenciais pois humanos erram.
- Os testes devem testar a si próprios.
 - Teste ok ou fail.
 - Não gastamos tempo checando à mão os resultados dos testes contra as referências.

Refatoração e testes

- A prática de **Testes** é essencial para o sucesso da **refatoração** !
- A refatoração está presente no **ciclo TDD**:
 - Criar um teste automatizado.
 - Criar uma implementação que passe no teste.
 - Refatorar.
- Não é objetivo aqui ensinar **TDD** ou outras técnicas de testes.
 - Porém, entender refatoração é essencial para entendermos, TDD, por exemplo.

Refatoração e Testes: JUnit (1/3)

@Test

```
public void locacaoFilmes() {  
    String saidaEsperadaTemp = "Registro de Locacao para Rodrigo Martins Pagliares\n"  
        + "\tEncontros e Desencontros      6.5\n"  
        + "\tMeia-Noite em Paris              9.0\n"  
        + "\tParis, Texas                        3.5\n"  
        + "\tBravura Indômita                    3.0\n"  
        + "\tShrek                               3.0\n"  
        + "Total devido, R$ 25.0\n"  
        + "Pontos adquiridos na locacao : 6\n";
```

```
Cliente rodrigo = new Cliente("Rodrigo Martins Pagliares");
```

```
....
```

.... CONTINUA NO PRÓXIMO SLIDE !

Refatoração e Testes: JUnit (2/3)

// Primeiro filme sendo locado(normal)

// Resultado esperado = R\$ 2,0 locacao + R\$ 4,50 de multa. Total parcial = R\$ 6,50

Filme encontros = new Filme("Encontros e Desencontros", 0);

Locacao loc1 = new Locacao(encontros, 5);

// Segundo filme sendo locado(Lancamento)

// Resultado = R\$ 3,0 locacao + R\$ 6,00 de multa. Total parcial = R\$ 9,00

Filme meia = new Filme("Meia-Noite em Paris", 1);

Locacao loc2 = new Locacao(meia, 3);

// Terceiro filme sendo locado(normal)

// Resultado = R\$ 2,0 locacao + R\$ 1,50 de multa. Total parcial = R\$ 3,50

Filme parisTexas = new Filme("Paris, Texas", 0);

Locacao loc3 = new Locacao(parisTexas, 3);

Refatoração e Testes: JUnit (3/3)

// Quarto filme sendo locado(lancamento)

// Resultado = R\$ 3,0 locacao , sem multa. Total parcial = R\$ 3,00)

Filme indomita = new Filme("Bravura Indômita", 1);

Locacao loc4 = new Locacao(indomita, 1);

// Quinto filme sendo locado(infantil)

// Resultado esperado = R\$ 1,50 locacao + R\$ 1,50 de multa. Total parcial = R\$ 3,00

Filme shrek = new Filme("Shrek", 2);

Locacao loc5 = new Locacao(shrek, 4);

rodrigo.adicionarLocacao(loc1);

rodrigo.adicionarLocacao(loc2);

rodrigo.adicionarLocacao(loc3);

rodrigo.adicionarLocacao(loc4);

rodrigo.adicionarLocacao(loc5);

assertEquals(saidaEsperadaTemp.trim(), rodrigo.conta().trim());

}

Eclipse File Edit Source Refactor Navigate Search Project Run Window Help

workspaceUNIFAL - Refactoring/src/test/java/cap1/versao1/TestesCap1.java - Eclipse IDE

Package Explorer

- cap1.versao9.parte3.substituir_condicional_por_polimorfismo
- cap1.versao9.parte4.move_method_locador_frequente
- src/test/java
 - cap1.versao1
 - TestesCap1.java**
 - cap1.versao2.extract_method
 - cap1.versao3.move_method.parte1.manter_metodo
 - cap1.versao3.move_method.parte2.remover_metodo
 - cap1.versao4.replace_temp_with_query
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - cap1.versao6.replace_temp_with_query_calculo_total_cont
 - cap1.versao6.replace_temp_with_query_calculo_total_pont
 - cap1.versao7.conta.html
 - cap1.versao8.polimorfismo.parte1.lerpreco
 - cap1.versao8.polimorfismo.parte2.lerpontos.locador.frequente
 - cap1.versao9.parte1.substituir_enum_state_strategy
 - cap1.versao9.parte2.move_method
 - cap1.versao9.parte3.substituir_condicional_por_polimorfismo
 - cap1.versao9.parte4.move_method_locador_frequente
 - refactoring.Refactoring
- JRE System Library [JavaSE-1.8]
- Maven Dependencies
- src
- target
- pom.xml

TestesCap1.java

```

56 // Quarto filme sendo locado(lancamento) (Resultado = R$ 3,0 locacao , sem multa - Total parcial = R$ 3,00)
57 Filme indomita = new Filme("Bravura Indômita", 1);
58 Locacao loc4 = new Locacao(indomita, 1);
59
60 // Quinto filme sendo locado(infantil) (Resultado = R$ 1,50 locacao + R$ 1,50 de multa - Total parcial = R$ 3,00)
61 Filme shrek = new Filme("Shrek", 2);
62 Locacao loc5 = new Locacao(shrek, 4);
63
64 rodrigo.adicionarLocacao(loc1);
65 rodrigo.adicionarLocacao(loc2);
66 rodrigo.adicionarLocacao(loc3);
67 rodrigo.adicionarLocacao(loc4);
68 rodrigo.adicionarLocacao(loc5);
69
70
71 assertEquals(saidaEsperadaTemp.trim(), rodrigo.conta().trim());
72
73 }
74
75 }
76

```

Outline

- cap1.versao1
 - TestesCap1**
 - TestesCap1()
 - setUpClass() : void
 - tearDownClass() : void
 - locacaoFilmes() : void

Problems Console Progress Git Repositories Rebase Interactive JUnit Git Reflog Debug Tasks

Finished after 0.04 seconds

Runs: 1/1 Errors: 0 Failures: 0

cap1.versao1.TestesCap1 [Runner: JUnit 4] (0.000 s)

locacaoFilmes (0.000 s)

Failure Trace

Writable Smart Insert 2 : 1

Versão 2: Extrair Método

Você tem um fragmento de código que pode ser agrupado.

Transforme o fragmento em um método cujo nome explique o propósito do mesmo.

Versão 2: Extrair Método

```
void imprimirDivida(double quantia) {  
    imprimirCabecalho();  
    // imprime os detalhes  
    System.out.println("nome: " + nome);  
    System.out.println("quantia: " + quantia);  
}
```



```
void imprimirDivida(double quantia) {  
    imprimirCabecalho();  
    imprimirDetalhes(quantia);  
}  
  
private void imprimirDetalhes(double quantia) {  
    System.out.println("nome: " + nome);  
    System.out.println("quantia: " + quantia);  
}
```

Versão 2: Candidato à extração

```
while (locacoes.hasMoreElements()) {  
    double estaQuantia = 0;  
    Locacao cada = (Locacao) locacoes.nextElement();  
    switch (cada.lerFilme().lerCodigoPreco()) {  
        case Filme.NORMAL:  
            estaQuantia += 2;  
            if (cada.lerDiasAlugados() > 2)  
                estaQuantia += (cada.lerDiasAlugados() - 2) * 1.5;  
            break;  
        case Filme.LANCAMENTO_NOVO:  
            estaQuantia += cada.lerDiasAlugados() * 3;  
            break;  
        case Filme.INFANTIL:  
            estaQuantia += 1.5;  
            if (cada.lerDiasAlugados() > 3)  
                estaQuantia += (cada.lerDiasAlugados() - 3) * 1.5;  
            break;  
    }  
    ....  
    ....  
}
```

Versão 2: Exercício

- Você tem 20min para usar o *refactoring extrair método* no trecho de código indicado no slide anterior.
 - Lembre-se de usar o teste automatizado fornecido para verificar que nada foi quebrado com sua alteração.
 - O método criado deverá ser chamado `quantiaDe()`

Versão 2: Extrair Método - Passos

- Criar um método com o nome associado à intenção do Código.
- Copiar o código extraído.
- Procurar variáveis locais e parâmetros.
 - Transforme-os em parâmetros.
 - Transforme-os em valores de retorno.
 - Declare-os dentro do método.
- Compile.
- Substitua o fragmento de código com uma chamada para o novo método.
- Compile e teste.

Versão 2: Extração do cálculo da locação

```
private double quantiaDe(Locacao cada) { // Método novo criado. Repare no argumento !
    double estaQuantia = 0; // Nova variável criada
    switch (cada.lerFilme().lerCodigoPreco()) {
        case Filme.NORMAL:
            estaQuantia += 2;
            if (cada.lerDiasAlugados() > 2)
                estaQuantia += (cada.lerDiasAlugados() - 2) * 1.5;
            break;
        case Filme.LANCAMENTO_NOVO:
            estaQuantia += cada.lerDiasAlugados() * 3;
            break;
        case Filme.INFANTIL:
            estaQuantia += 1.5;
            if (cada.lerDiasAlugados() > 3)
                estaQuantia += (cada.lerDiasAlugados() - 3) * 1.5;
            break;
    }
    return estaQuantia;
}
```

Versão 2: O Método Conta Após Extração

```
public String conta() {  
    double quantiaTotal = 0;  
    int pontosLocadorFrequente = 0;  
    Enumeration locacoes = _locacoes.elements();  
    String resultado = "Registro de Locacao para " + lerNome() + "\n";  
    while (locacoes.hasMoreElements()) {  
        double estaQuantia = 0;  
        Locacao cada = (Locacao) locacoes.nextElement();  
        estaQuantia = quantiaDe(cada);  
        pontosLocadorFrequente++;  
        // Ponto adicional para o locatario para cada filme lancamento  
        if ((cada.lerFilme().lerCodigoPreco() == Filme.LANCAMENTO_NOVO)  
            && cada.lerDiasAlugados() > 1) {  
            pontosLocadorFrequente++;  
        }  
        ....  
        ...  
    }  
}
```

Compile e Teste !!

*A **refatoração** altera os programas em pequenos passos. Se você cometer um erro, é fácil encontrar a falha.*

Eclipse File Edit Source Refactor Navigate Search Project Run Window Help workspaceUNIFAL - Refactoring/src/main/java/cap1/versao2/extract_method/Cliente.java - Eclipse IDE

Package Explorer

- Object_Orientation
- Refactoring
 - src/main/java
 - cap1.versao1
 - cap1.versao2.extract_method
 - Cliente.java
 - Filme.java
 - Locacao.java
 - cap1.versao3.move_method.parte1.manter_metodo
 - cap1.versao3.move_method.parte2.remover_metodo
 - cap1.versao4.replace_temp_with_query
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - cap1.versao6.replace_temp_with_query_calculo_total_cont
 - cap1.versao6.replace_temp_with_query_calculo_total_pont
 - cap1.versao7.conta.html
 - cap1.versao8.polimorfismo.parte1.lerpreco
 - cap1.versao8.polimorfismo.parte2.lerpontos.locador.frequ
 - cap1.versao9.parte1.substituir_enum_state_strategy
 - cap1.versao9.parte2.move_method
 - cap1.versao9.parte3.substituir_condicional_por_polimorfi
 - cap1.versao9.parte4.move_method_locador_frequente
 - src/test/java
 - cap1.versao1
 - TestesCap1.java
 - cap1.versao2.extract_method

Outline

- cap1.versao2.extract_method
 - Cliente
 - _nome : String
 - _locacoes : Vector
 - Cliente(String)
 - adicionarLocacao(Locacao) : void
 - lerNome() : String
 - conta() : String
 - quantiaDe(Locacao) : double

Cliente.java

```
62 private double quantiaDe(Locacao umaLocacao) {
63     double resultado = 0;
64     // Valor da locacao para cada filme dentro do Vector
65     switch (umaLocacao.lerFilme().lerCodigoPreco()) { // codigoPreco associado ao tipo do filme
66         case Filme.NORMAL:
67             resultado += 2; // R$ 2,00 por filme normal alugado
68             if (umaLocacao.lerDiasAlugados() > 2) { // Multa de 1.50 por dia de atraso caso seja entregue apos 2 dia
69                 resultado += (umaLocacao.lerDiasAlugados() - 2) * 1.5;
70             }
71             break;
72         case Filme.LANCAMENTO_NOVO:
73             resultado += umaLocacao.lerDiasAlugados() * 3; // R$ 3,00 a diaria para lancamentos por filme alugado
74
75             break;
76         case Filme.INFANTIL:
77             resultado += 1.5; // R$ 1,50 por 3 dias por filme infantil alugado
78             if (umaLocacao.lerDiasAlugados() > 3) {
79                 resultado += (umaLocacao.lerDiasAlugados() - 3) * 1.5; // Multa de 1,50 por dia de atraso caso seja ei
80             }
81             break;
82     }
```

Problems Console Progress Git Repositories Rebase Interactive JUnit Git Reflog Debug Tasks

Finished after 0.044 seconds

Runs: 1/1 Errors: 0 Failures: 0

cap1.versao2.extract_method.TestesCap1 [Runner: JUnit 4] (0.000 s)

Failure Trace

Versão 2 : Renomeando Variáveis

```
private double quantiaDe(Locacao umaLocacao) { // de cada para umaLocacao
    double resultado = 0; // de estaQuantia para resultado
    switch (umaLocacao.lerFilme().lerCodigoPreco()) {
        case Filme.NORMAL:
            resultado += 2;
            if (umaLocacao.lerDiasAlugados() > 2)
                resultado += (umaLocacao.lerDiasAlugados() - 2) * 1.5;
            break;
        case Filme.LANCAMENTO_NOVO:
            resultado += umaLocacao.lerDiasAlugados() * 3;
            break;
        case Filme.INFANTIL:
            resultado += 1.5;
            if (umaLocacao.lerDiasAlugados() > 3)
                resultado += (umaLocacao.lerDiasAlugados() - 3) * 1.5;
            break;
    }
    return resultado;
}
```

Versão 2 : Renomeando Variáveis

- Vale a pena renomear variáveis?
 - **Claro!** Um bom código deve transmitir com clareza o que ele está fazendo.
- Com boas ferramentas de busca e substituição, este trabalho normalmente não é complicado.

Um código que comunica seu propósito é muito importante !!

Compile e Teste !!

Eclipse File Edit Source Refactor Navigate Search Project Run Window Help

workspaceUNIFAL - Refactoring/src/main/java/cap1/versao2/extract_method_rename_variable/Cliente.java - Eclipse IDE

Package Explorer

- Object_Orientation
- Refactoring
 - src/main/java
 - cap1.versao1
 - cap1.versao2.extract_method_rename_variable
 - Cliente.java
 - Filme.java
 - Locacao.java
 - cap1.versao3.move_method.parte1.manter_metodo
 - cap1.versao3.move_method.parte2.remover_metodo
 - cap1.versao4.replace_temp_with_query
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - cap1.versao6.replace_temp_with_query_calculo_total_cont
 - cap1.versao6.replace_temp_with_query_calculo_total_pont
 - cap1.versao7.conta.html
 - cap1.versao8.polimorfismo.parte1.lerpreco
 - cap1.versao8.polimorfismo.parte2.lerpontos.locador.frequ
 - cap1.versao9.parte1.substituir_enum_state_strategy
 - cap1.versao9.parte2.move_method
 - cap1.versao9.parte3.substituir_condicional_por_polimorfi
 - cap1.versao9.parte4.move_method_locador_frequente
 - src/test/java
 - cap1.versao1
 - TestesCap1.java
 - cap1.versao2.extract_method_rename_variable

Outline

- cap1.versao2.extract_method_rename_variable
 - Cliente
 - _nome : String
 - _locacoes : Vector
 - Cliente(String)
 - adicionarLocacao(Locacao) : void
 - lerNome() : String
 - conta() : String
 - quantiaDe(Locacao) : double

Cliente.java

```
62 private double quantiaDe(Locacao umaLocacao) {
63     double resultado = 0;
64     // Valor da locacao para cada filme dentro do Vector
65     switch (umaLocacao.lerFilme().lerCodigoPreco()) { // codigoPreco associado ao tipo do filme
66         case Filme.NORMAL:
67             resultado += 2; // R$ 2,00 por filme normal alugado
68             if (umaLocacao.lerDiasAlugados() > 2) { // Multa de 1.50 por dia de atraso caso seja entregue apos 2 dia
69                 resultado += (umaLocacao.lerDiasAlugados() - 2) * 1.5;
70             }
71             break;
72         case Filme.LANCAMENTO_NOVO:
73             resultado += umaLocacao.lerDiasAlugados() * 3; // R$ 3,00 a diaria para lancamentos por filme alugado
74
75             break;
76         case Filme.INFANTIL:
77             resultado += 1.5; // R$ 1,50 por 3 dias por filme infantil alugado
78             if (umaLocacao.lerDiasAlugados() > 3) {
79                 resultado += (umaLocacao.lerDiasAlugados() - 3) * 1.5; // Multa de 1,50 por dia de atraso caso seja e
80             }
81             break;
82     }
```

Problems Console Progress Git Repositories Rebase Interactive JUnit Git Reflog Debug Tasks

Finished after 0.039 seconds

Runs: 1/1 Errors: 0 Failures: 0

cap1.versao2.extract_method_rename_variable.TestesCap1 [Runner: JUnit 4] (0.001 s)

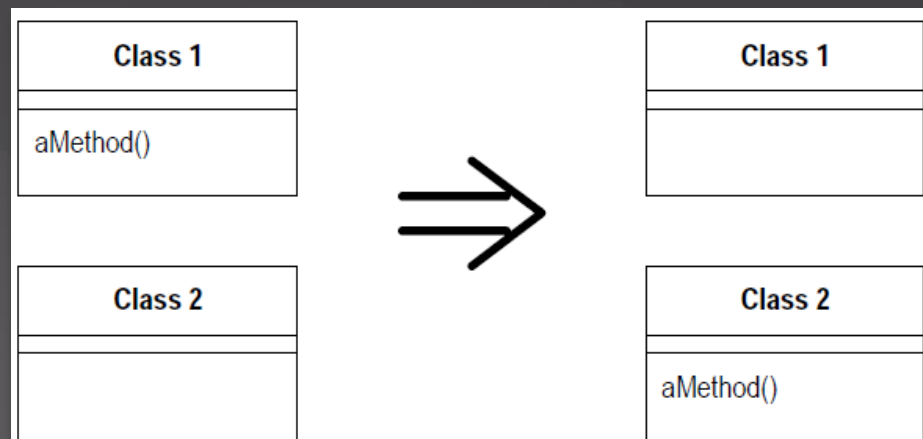
Failure Trace

Writable Smart Insert 2 : 1

Versão 3: Mover Método

Um método está, ou estará, usando ou sendo usado por mais recursos de outra classe do que a classe na qual ele está definido.

Crie um novo método com um corpo similar na classe que ele mais usa. Transforme o método antigo em uma simples delegação ou remova-o completamente.



Versão 3 : Candidato à Mudança

```
private double quantiaDe(Locacao umaLocacao) {  
    double resultado = 0;  
    switch (umaLocacao.lerFilme().lerCodigoPreco()) {  
        case Filme.NORMAL:  
            resultado += 2;  
            if (umaLocacao.lerDiasAlugados() > 2)  
                resultado += (umaLocacao.lerDiasAlugados() - 2) * 1.5;  
            break;  
        case Filme.LANCAMENTO_NOVO:  
            resultado += umaLocacao.lerDiasAlugados() * 3;  
            break;  
        case Filme.INFANTIL:  
            resultado += 1.5;  
            if (umaLocacao.lerDiasAlugados() > 3)  
                resultado += (umaLocacao.lerDiasAlugados() - 3) * 1.5;  
            break;  
    }  
    return resultado;  
}
```

Observe que este método de `Cliente` somente usa dados de `umaLocação` e não de `Cliente`

Versão 3: Exercício

- Você tem 20min para usar o **Mover Método** no trecho de código indicado no slide anterior.
 - Após movido, o método deverá ser renomeado no destino para **lerPreco**.
 - Lembre-se de usar o teste automatizado fornecido para verificar que nada foi quebrado com sua alteração.

Versão 3: Mover Método - Passos

- Declare o método na classe destino.
- Copie e ajuste o código no novo local.
- Configure uma referência do objeto origem para o objeto destino.
- Transforme o método na classe de origem em um método de delegação.
 - `quantiaDe(Locacao cada) { return cada.lerPreco(); }`
 - Verifique a sobreescrita de métodos.
- Compile e teste.
- Encontre todos os usuários do método.
 - Ajuste-os para chamar o método no destino.
- Remova o método na classe de origem.
- Compile e teste.

Versão 3: Movendo quantiaDe() para Locacao renomeando para lerPreco()

```
public class Locacao { ...  
    public double lerPreco() {  
        double resultado = 0;  
        switch (lerFilme().lerCodigoPreco()) {  
            case Filme.NORMAL:  
                resultado += 2;  
                if (lerDiasAlugados() > 2)  
                    resultado += (lerDiasAlugados() - 2) * 1.5;  
                break;  
            case Filme.LANCAMENTO_NOVO:  
                resultado += lerDiasAlugados() * 3;  
                break;  
            case Filme.INFANTIL:  
                resultado += 1.5;  
                if (lerDiasAlugados() > 3)  
                    resultado += (lerDiasAlugados() - 3) * 1.5;  
                break;  
        }  
        return resultado;  
    }  
}
```

Observe que movemos o método para a classe Locação, eliminamos o parâmetro e aproveitamos para renomeá-lo

Versão 3: Movendo quantiaDe() para Locacao renomeando para lerPreco()

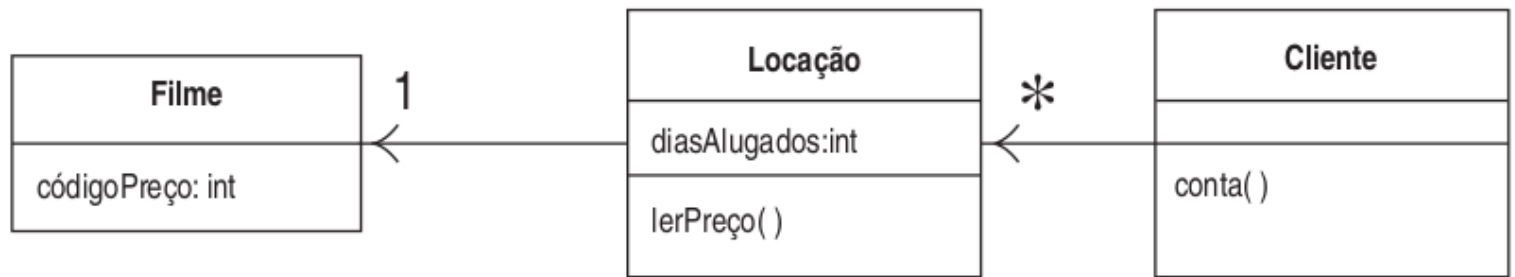


Figura 1.3 Estado das classes após a movimentação do método de cobrança

Novo método (lerPreco) na classe Locação

Versão 3: Alteração no código Cliente

```
public class Cliente { ...  
    public String conta() { ...  
        Locacao cada = (Locacao) locacoes.nextElement();  
        estaQuantia = cada.lerPreco();  
  
        pontosLocadorFrequente++;  
        // Ponto adicional para o locatario para cada filme lancamento  
        if ((cada.lerFilme().lerCodigoPreco() == Filme.LANCAMENTO_NOVO)  
            && cada.lerDiasAlugados() > 1) {  
            pontosLocadorFrequente++;  
        }  
  
        //armazena o valor do filme para cada filme locado  
        resultado += "\t" + cada.lerFilme().lerTitulo() + "\t"  
            + String.valueOf(estaQuantia) + "\n";  
        quantiaTotal += estaQuantia; // total da locacao  
  
    }  
}
```

Como alternativa,
poderíamos manter
o método
`quantiaDe()` na
classe
`Cliente...`

...a partir dele delegar
para o método
`lerPreco()` na
classe `Locacao`

Compile e Teste !!

Eclipse File Edit Source Refactor Navigate Search Project Run Window Help

workspaceUNIFAL - Refactoring/src/main/java/cap1/versao3/move_method/parte2/remover_metodo/Cliente.java - Eclipse IDE

Package Explorer

- TesteCap1.java
- cap1.versao4.replace_temp_with_query
- cap1.versao5.extract_method_locatario_frequente.parte1
- cap1.versao5.move_method_locatario_frequente.parte2
- cap1.versao6.replace_temp_with_query_calculo_total_cont
- cap1.versao6.replace_temp_with_query_calculo_total_pont
- cap1.versao7.conta.html
- cap1.versao8.polimorfismo.parte1.lerpreco
- cap1.versao8.polimorfismo.parte2.lerpontos.locador.frequ
- cap1.versao9.parte1.substituir_enum_state_strategy
- cap1.versao9.parte2.move_method
- cap1.versao9.parte3.substituir_condicional_por_polimorfi
- cap1.versao9.parte4.move_method_locador_frequente
- src/test/java
 - cap1.versao1
 - cap1.versao2.extract_method_rename_variable
 - cap1.versao3.move_method.parte1.manter_metodo
 - TestesCap1.java
 - cap1.versao3.move_method.parte2.remover_metodo
 - TestesCap1.java
 - cap1.versao4.replace_temp_with_query
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - cap1.versao6.replace_temp_with_query_calculo_total_cont
 - cap1.versao6.replace_temp_with_query_calculo_total_pont
 - cap1.versao7.conta.html

Cliente.java

```
32 double quantiaTotal = 0;
33 int pontosLocadorFrequente = 0;
34 Enumeration locais = _locaes.elements();
35 String resultado = "Registro de Locacao para " + lerNome() + "\n";
36 while (locaes.hasMoreElements()) {
37     double estaQuantia = 0;
38     Locacao cada = (Locacao) locais.nextElement();
39
40     estaQuantia = cada.lerPreco();
41
42     pontosLocadorFrequente++;
43     // Ponto adicional para o locatario para cada filme lancamento
44     if ((cada.lerFilme().lerCodigoPreco() == Filme.LANCAMENTO_NOVO)
45         && cada.lerDiasAlugados() > 1) {
46         pontosLocadorFrequente++;
47     }
48
49     //armazena o valor do filme para cada filme locado
50     resultado += "\t" + cada.lerFilme().lerTitulo() + "\t"
51         + String.valueOf(estaQuantia) + "\n";
52     quantiaTotal += estaQuantia; // total da locacao
53 }
```

Outline

- cap1.versao3.move_method.parte2.remover_metodo
- Cliente
 - _nome : String
 - _locaes : Vector
 - Cliente(String)
 - adicionarLocacao(Locacao) : void
 - lerNome() : String
 - conta() : String

Problems Console Progress Git Repositories Rebase Interactive JUnit Git Reflog Debug Tasks

Finished after 0.053 seconds

Runs: 1/1 Errors: 0 Failures: 0

cap1.versao3.move_method.parte2.remover_metodo.TestesCap1 [Runner: JUnit 4] (0.000 s) Failure Trace

Writable Smart Insert 40 : 27

Versão 4: Substituir Variável Temporária por Consulta

Você está usando uma variável temporária para armazenar o resultado de uma expressão.

Extraia a expressão para um método. Substitua todas as referências à variável temporária pela expressão. Este novo método pode então ser usado em outros métodos.

Versão 4: Substituir Variável Temporária por Consulta - Passos

- Encontre uma variável temporária com uma instrução simples.
- Extraia o lado direito da atribuição para um método.
- Substitua todas as referências à variável temporária com o novo método.
- Remova a declaração e atribuição da variável temporária.
- **Compile e teste !**

Versão 4: Substituir Variável Temporária por Consulta

```
public class Cliente { ...
    public String conta() { ...
        Locacao cada = (Locacao) locacoes.nextElement();
        estaQuantia = cada.lerPreco();

        pontosLocadorFrequente++;
        // Ponto adicional para o locatario para cada filme lancamento
        if ((cada.lerFilme().lerCodigoPreco() == Filme.LANCAMENTO_NOVO)
            && cada.lerDiasAlugados() > 1) {
            pontosLocadorFrequente++;
        }

        //armazena o valor do filme para cada filme locado
        resultado += "\t" + cada.lerFilme().lerTitulo() + "\t"
            + String.valueOf(estaQuantia) + "\n";
        quantiaTotal += estaQuantia; // total da locacao
    }
}
```

Observe que a variável `estaQuantia` recebe o resultado da chamada `cada.lerPreco` e depois não é alterada.

Versão 4: Exercício

- Você tem 5min para usar o **Substituir Variável Temporária por Consulta** no trecho de código indicado no slide anterior.
 - Lembre-se de usar o teste automatizado fornecido para verificar que nada foi quebrado com sua alteração.

Versão 4: Substituir Variável Temporária por Consulta

```
public class Cliente { ...  
    public String conta() { ...  
        Locacao cada = (Locacao) locacoes.nextElement();  
  
        pontosLocadorFrequente++;  
        // Ponto adicional para o locatario para cada filme lancamento  
        if ((cada.lerFilme().lerCodigoPreco() == Filme.LANCAMENTO_NOVO)  
            && cada.lerDiasAlugados() > 1) {  
            pontosLocadorFrequente++;  
        }  
  
        //armazena o valor do filme para cada filme locado  
        resultado += "\t" + cada.lerFilme().lerTitulo() + "\t"  
            + String.valueOf(cada.lerPreco( )) + "\n";  
        quantiaTotal += cada.lerPreco( ); // total da locacao  
    }  
}
```

variável
temporária
estaQuantia
removida

Discutiremos
sobre
performance no
próximo
capítulo.

Compile e Teste !!

Eclipse File Edit Source Refactor Navigate Search Project Run Window Help

workspaceUNIFAL - Refactoring/src/main/java/cap1/versao4/replace_temp_with_query/Cliente.java - Eclipse IDE

Package Explorer

- cap1.versao9.parte3.substituir_condicional_por_polimorfismo
- cap1.versao9.parte4.move_method_locador_frequente
- src/test/java
 - cap1.versao1
 - cap1.versao2.extract_method_rename_variable
 - cap1.versao3.move_method.parte1.manter_metodo
 - cap1.versao3.move_method.parte2.remover_metodo
 - cap1.versao4.replace_temp_with_query
 - TestesCap1.java
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - cap1.versao6.replace_temp_with_query_calculo_total_conta
 - cap1.versao6.replace_temp_with_query_calculo_total_pontos
 - cap1.versao7.conta.html
 - cap1.versao8.polimorfismo.parte1.lerpreco
 - cap1.versao8.polimorfismo.parte2.lerpontos.locador.frequente
 - cap1.versao9.parte1.substituir_enum_state_strategy
 - cap1.versao9.parte2.move_method
 - cap1.versao9.parte3.substituir_condicional_por_polimorfismo
 - cap1.versao9.parte4.move_method_locador_frequente
 - refactoring.Refactoring
- JRE System Library [JavaSE-1.8]
- Maven Dependencies
- src
- target
- pom.xml

Outline

- cap1.versao4.replace_temp_with_query
- Cliente
 - _nome : String
 - _locacoes : Vector
 - Cliente(String)
 - adicionarLocacao(Locacao) : void
 - lerNome() : String
 - conta() : String

Locacao.java Filme.java Cliente.java

```
41 if ((cada.lerFilme().lerCodigoPreco() == Filme.LANCAMENTO_NOVO)
42     && cada.lerDiasAlugados() > 1) {
43     pontosLocadorFrequente++;
44 }
45
46 //armazena o valor do filme para cada filme locado
47 resultado += "\t" + cada.lerFilme().lerTitulo() + "\t"
48     + String.valueOf(cada.lerPreco()) + "\n";
49 quantiaTotal += cada.lerPreco(); // total da locacao
50
51 }
52 //adiciona o rodape
53 resultado += "Total devido, R$ " + String.valueOf(quantiaTotal) + "\n";
54 resultado += "Pontos adquiridos na locacao : " + String.valueOf(pontosLocadorFrequente);
55 return resultado;
56
57 }
58
59
60
61 }
```

Problems Console Progress Git Repositories Rebase Interactive JUnit Git Reflog Debug Tasks

Finished after 0.046 seconds

Runs: 1/1 Errors: 0 Failures: 0

cap1.versao4.replace_temp_with_query.TestesCap1 [Runner: JUnit 4] (0.000 s)

- locacaoFilmes (0.000 s)

Failure Trace

Versão 5 - Extrair e Mover Método: antes

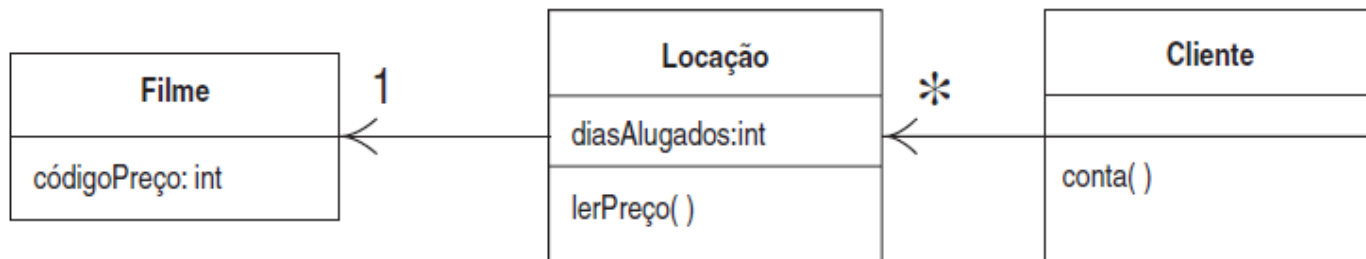


Figura 1.4 Diagrama de classe de antes da extração e movimentação do cálculo dos pontos dos locadores freqüentes.

Versão 5 - Extrair e Mover Método: antes

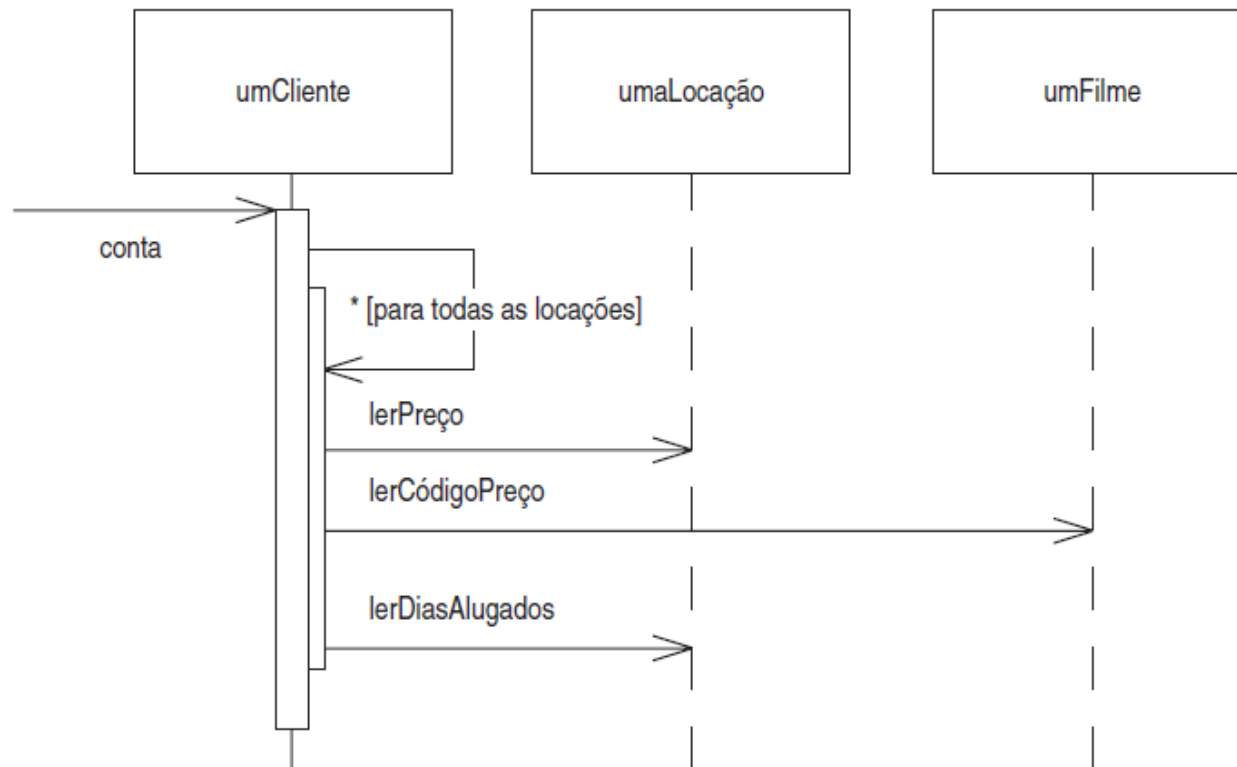


Figura 1.5 Diagramas de seqüência antes da extração e movimentação do cálculo dos pontos dos locadores freqüentes.

Versão 5 - Extrair e Mover Método: antes

```
public class Cliente { ...  
    public String conta() { ...  
        Locacao cada = (Locacao) locacoes.nextElement();  
  
        pontosLocadorFrequente++;  
        // Ponto adicional para o locatário para cada filme lançamento  
        if ((cada.lerFilme().lerCodigoPreco() == Filme.LANCAMENTO_NOVO)  
            && cada.lerDiasAlugados() > 1) {  
            pontosLocadorFrequente++;  
        }  
  
        //armazena o valor do filme para cada filme locado  
        resultado += "\t" + cada.lerFilme().lerTitulo() + "\t"  
            + String.valueOf(cada.lerPreco( )) + "\n";  
        quantiaTotal += cada.lerPreco( ); // total da locacao  
  
    }  
}
```

Demonstraremos
via diagramas
UML nos
próximos slides

Versão 5: Exercício

- Você tem 10min para usar os *refactorings* **Extair Método** e **Mover Método** no trecho de código indicado no slide anterior.
 - O método extraído e movido deverá ser chamado de `lerPontosLocadorFrequente()`
 - Lembre-se de usar o teste automatizado fornecido para verificar que nada foi quebrado com sua alteração.

Eclipse File Edit Source Refactor Navigate Search Project Run Window Help

workspaceUNIFAL - Refactoring/src/main/java/cap1/versao5/move_method_locatario_frequente/parte2/Locacao.java - Eclipse IDE

Package Explorer

- cap1.versao9.parte3.substituir_condicional_por_polimorfismo
- cap1.versao9.parte4.move_method_locador_frequente
- src/test/java
 - cap1.versao1
 - cap1.versao2.extract_method_rename_variable
 - cap1.versao3.move_method.parte1.manter_metodo
 - cap1.versao3.move_method.parte2.remover_metodo
 - cap1.versao4.replace_temp_with_query
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - TestesCap1.java
 - cap1.versao6.replace_temp_with_query_calculo_total_cont
 - cap1.versao6.replace_temp_with_query_calculo_total_pont
 - cap1.versao7.conta.html
 - cap1.versao8.polimorfismo.parte1.lerpreco
 - cap1.versao8.polimorfismo.parte2.lerpontos.locador.frequente
 - cap1.versao9.parte1.substituir_enum_state_strategy
 - cap1.versao9.parte2.move_method
 - cap1.versao9.parte3.substituir_condicional_por_polimorfismo
 - cap1.versao9.parte4.move_method_locador_frequente
- refactoring.Refactoring
- JRE System Library [JavaSE-1.8]
- Maven Dependencies
- src
- target
- pom.xml

Locacao.java

```
44     case Filme.INFANTIL:
45         resultado += 1.5; // R$ 1,50 por 3 dias por filme infantil alugado
46         if (lerDiasAlugados() > 3) {
47             resultado += (lerDiasAlugados() - 3) * 1.5; // Multa de 1,50 por dia de atraso caso seja entregue apos 3 dias
48         }
49         break;
50     }
51     return resultado;
52 }
53
54 public int lerPontosLocadorFrequente() {
55     // Ponto adicional para o locatario para cada filme lancamento
56     if ((lerFilme().lerCodigoPreco() == Filme.LANCAMENTO_NOVO)
57         && lerDiasAlugados() > 1) {
58         return 2;
59     }
60     return 1;
61 }
62 }
63 }
64 }
```

Outline

- cap1.versao5.move_method_locatario_frequente.parte2
- Locacao
 - _filme : Filme
 - _diasAlugados : int
 - Locacao(Filme, int)
 - lerDiasAlugados() : int
 - lerFilme() : Filme
 - lerPreco() : double
 - lerPontosLocadorFrequente() : int

Problems Console Progress Git Repositories Rebase Interactive JUnit Git Reflog Debug Tasks

Finished after 0.051 seconds

Runs: 1/1 Errors: 0 Failures: 0

cap1.versao5.move_method_locatario_frequente.parte2.TestesCap1 [Runner: JUnit 4] (0.000 s)

locacaoFilmes (0.000 s)

Versão 5 - Extrair e Mover Método: depois

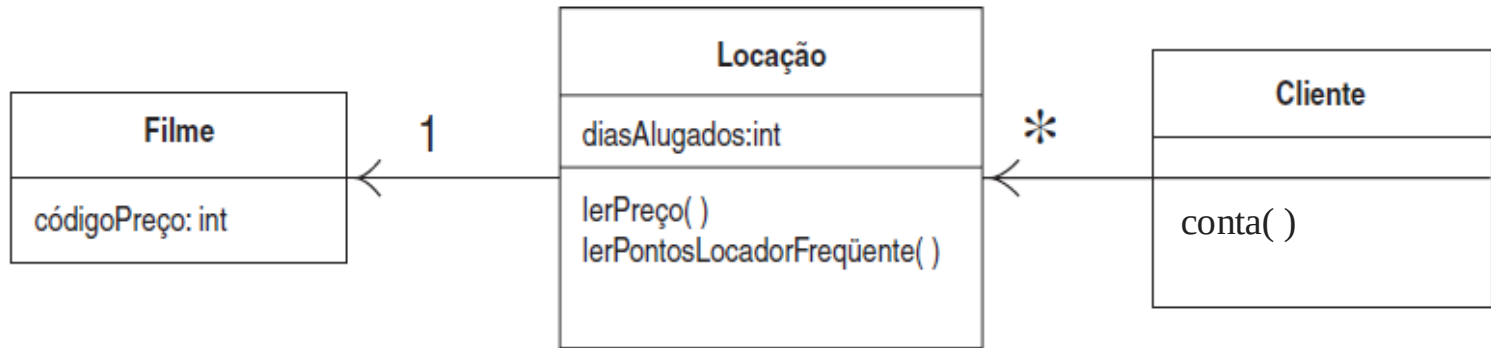


Figura 1.6 Diagrama de classe após a extração e movimentação do cálculo dos pontos dos locadores frequentes.

Versão 5 - Extrair e Mover Método: depois

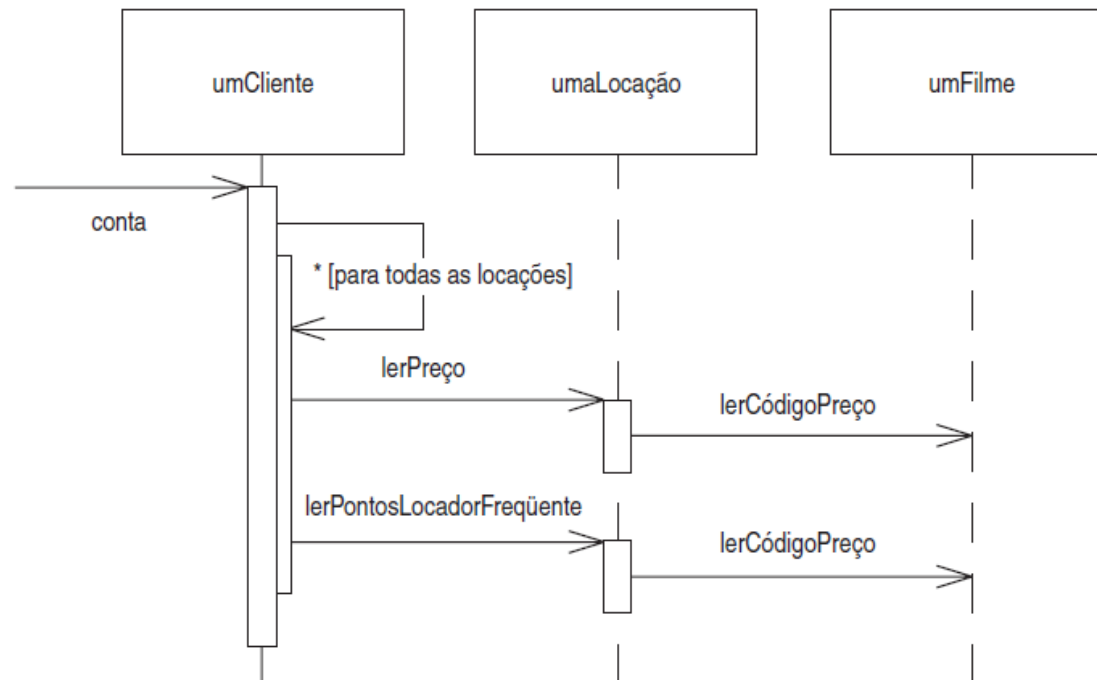


Figura 1.7 Diagramas de sequência após a extração e movimentação do cálculo dos pontos dos locadores frequentes.

Versão 6: Novamente Substituir Variável Temporária por Consulta

- Variáveis temporárias podem ser um problema.
 - São úteis apenas dentro de suas próprias rotinas e desta forma encorajam rotinas longas e complexas.
- Tanto a versão **ASCII** quanto a **HTML** requerem o total da locação e dos pontos de locadores freqüentes
 - Podemos usar *Substituir Variável Temporária por Consulta*
 - Substituindo `quantiaTotal` e `pontosLocadorFreqüente`

Versão 6: Substituir Variável Temporária por Consulta - antes

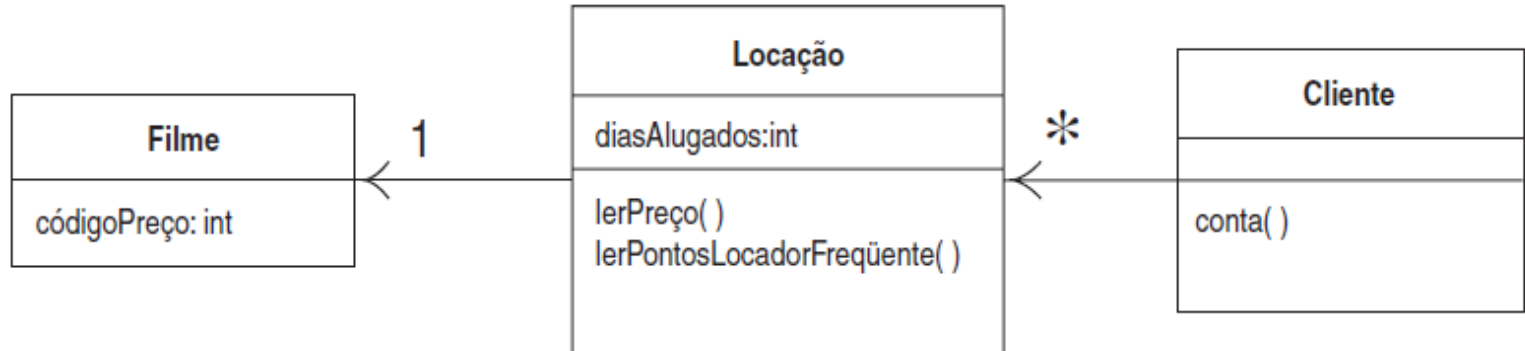


Figura 1.8 Diagrama de classes antes da extração dos totais.

Versão 6: Substituir Variável Temporária por Consulta - antes

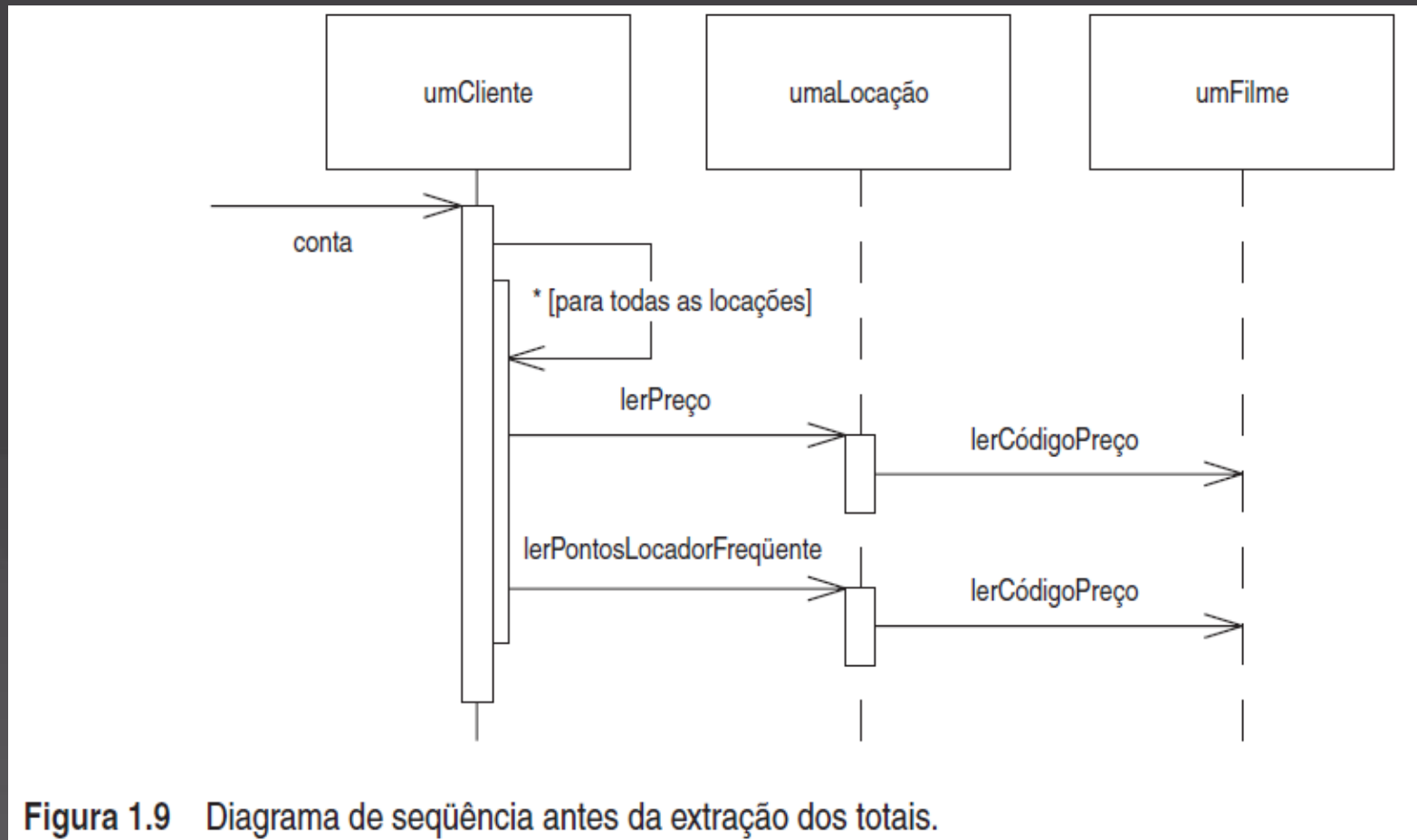


Figura 1.9 Diagrama de seqüência antes da extração dos totais.

Versão 6: Exercício

- Você tem 20min para aplicar a refatoração *Substituir Variável Temporária por Consulta* de forma a eliminar as variáveis `quantiaTotal` e `pontosLocadorFrequente` do método `conta( )`
 - Os métodos criados deverão ser chamados de `LerPrecoTotal` e `lerTotalPontosLocadorFrequente`
 - Lembre-se de usar o teste automatizado fornecido para verificar que nada foi quebrado com sua alteração.
 - Veja o próximo slide para as alterações a serem feitas.

```

public String conta() {
    double quantiaTotal = 0;
    int pontosLocadorFrequente = 0;
    Enumeration locacoes = _locacoes.elements();
    String resultado = "Registro de Locacao para " + lerNome() + "\n";
    while (locacoes.hasMoreElements()) {
        double estaQuantia = 0;
        Locacao cada = (Locacao) locacoes.nextElement();
        pontosLocadorFrequente += cada.lerPontosLocadorFrequente();

        //mostra o valor do filme para cada filme locado
        resultado += "\t" + cada.lerFilme().lerTitulo() + "\t"
            + String.valueOf(cada.lerPreco()) + "\n";
        quantiaTotal += cada.lerPreco(); // total da locacao
    }
    //adiciona o rodape
    resultado += "Total devido, R$ " + String.valueOf(quantiaTotal) + "\n";
    resultado += "Pontos adquiridos na locacao : " +
        String.valueOf(pontosLocadorFrequente);
    return resultado;
}

```

Após escrever os métodos, este código deverá ser atualizado para utilizá-los

Versão 6: Os novos métodos

Class Cliente { ...

public double lerPrecoTotal() {

double resultado = 0.0;

Enumeration locacoes = _locacoes.elements();

while (locacoes.hasMoreElements()) {

double estaQuantia = 0;

Locacao cada = (Locacao) locacoes.nextElement();

resultado += cada.lerPreco(); // total da locacao

}

return resultado;

}

Versão 6: Os novos métodos

Class Cliente { ...

```
public int lerTotalPontosLocadorFrequente() {  
    int resultado = 0;  
    Enumeration locacoes = _locacoes.elements();  
    while (locacoes.hasMoreElements()) {  
        Locacao cada = (Locacao) locacoes.nextElement();  
        resultado += cada.lerPontosLocadorFrequente();  
    }  
    return resultado;  
}
```

Versão 6: Substituir Variável Temporária por Consulta

```
public String conta() {  
    Enumeration locacoes = _locacoes.elements();  
    String resultado = "Registro de Locacao para " + lerNome() + "\n";  
    while (locacoes.hasMoreElements()) {  
        Locacao cada = (Locacao) locacoes.nextElement();  
        //mostra os valores para cada locação  
        resultado += "\t" + cada.lerFilme().lerTitulo() + "\t"  
            + String.valueOf(cada.lerPreco()) + "\n";  
    }  
    //adiciona o rodapé  
    resultado += "Total devido, R$ " + String.valueOf(lerPrecoTotal()) + "\n";  
    resultado += "Pontos adquiridos na locacao : " +  
        String.valueOf(lerTotalPontosLocadorFrequente());  
    return resultado;  
}
```

Versão 6: Substituir Variável Temporária por Consulta - depois

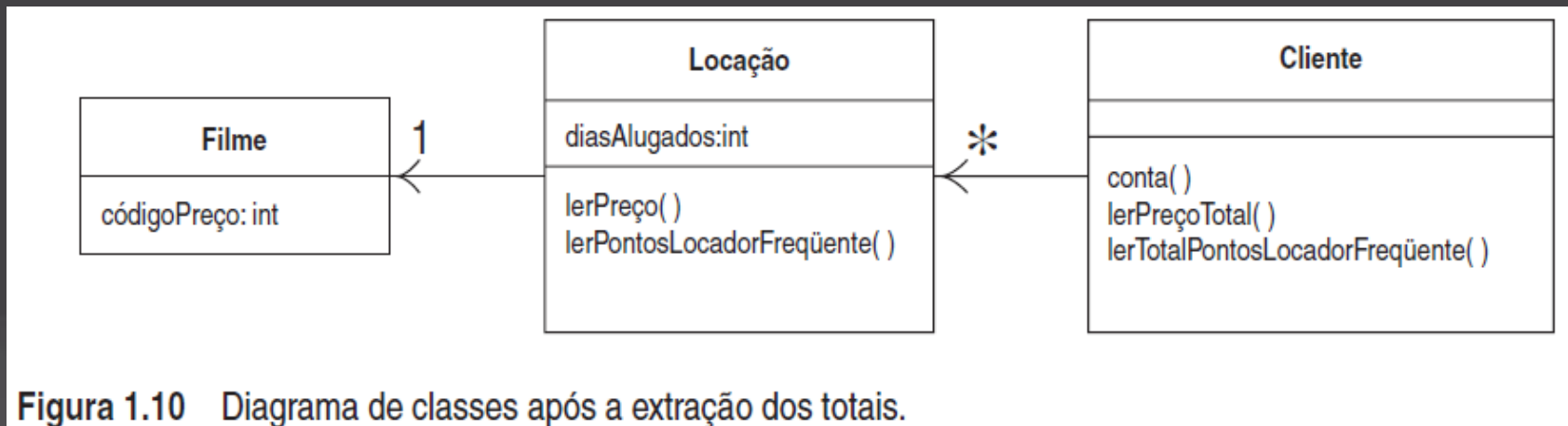


Figura 1.10 Diagrama de classes após a extração dos totais.

Versão 6: Substituir Variável Temporária por Consulta - depois

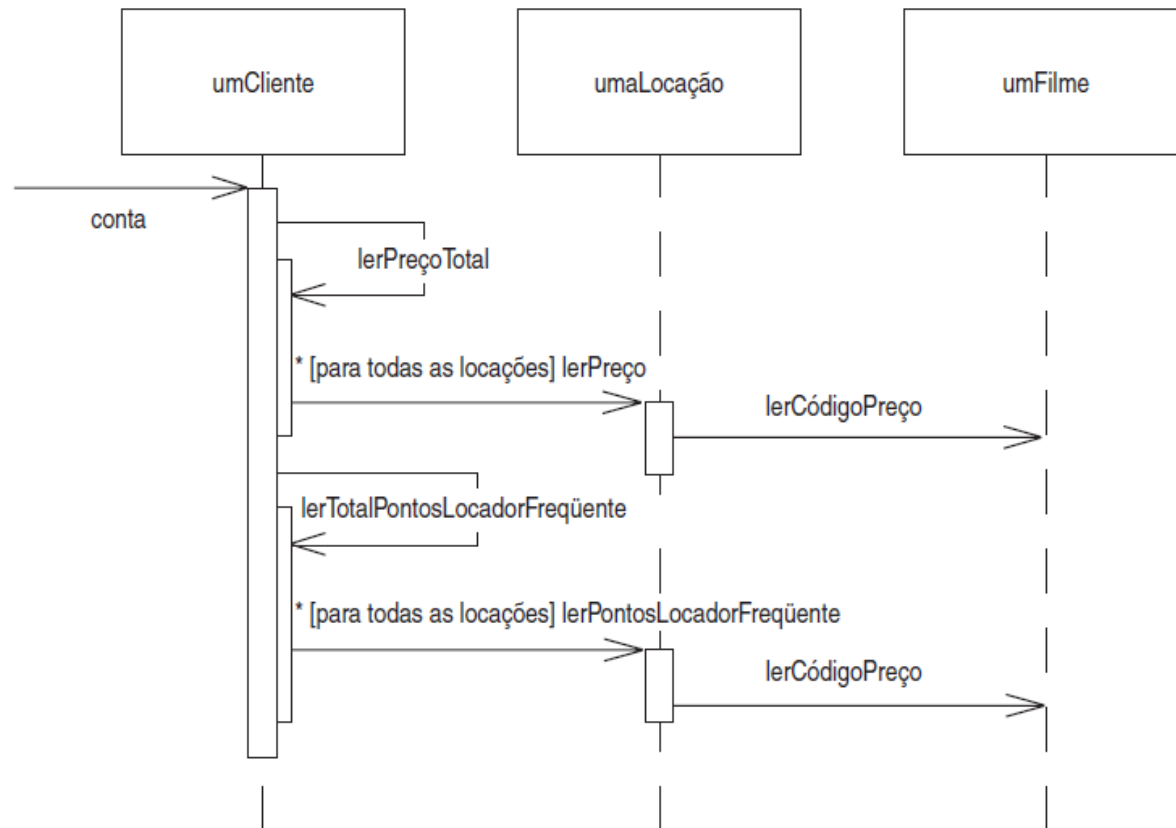


Figura 1.11 Diagrama de seqüência após a extração dos totais.

Eclipse File Edit Navigate Search Project Run Window Help workspaceUNIFAL - Eclipse IDE

Package Explorer

- cap1.versao9.parte1.substituir_enum_state_strategy
- cap1.versao9.parte2.move_method
- cap1.versao9.parte3.substituir_condicional_por_polimorfismo
- cap1.versao9.parte4.move_method_locador_frequente
- src/test/java
 - cap1.versao1
 - cap1.versao2.extract_method_rename_variable
 - cap1.versao3.move_method.parte1.manter_metodo
 - cap1.versao3.move_method.parte2.remover_metodo
 - cap1.versao4.replace_temp_with_query
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - cap1.versao6.replace_temp_with_query_calculo_total_conta.parte1
 - cap1.versao6.replace_temp_with_query_calculo_total_pontos.parte2
 - TestesCap1.java
 - cap1.versao7.conta.html
 - cap1.versao8.polimorfismo.parte1.lerpreco
 - cap1.versao8.polimorfismo.parte2.lerpontos.locador.frequente
 - cap1.versao9.parte1.substituir_enum_state_strategy
 - cap1.versao9.parte2.move_method
 - cap1.versao9.parte3.substituir_condicional_por_polimorfismo
 - cap1.versao9.parte4.move_method_locador_frequente
 - refactoring.Refactoring
- JRE System Library [JavaSE-1.8]
- Maven Dependencies
- src

Outline

An outline is not available.

Problems Console Progress Git Repositories Rebase Interactive JUnit Git Reflog Debug Tasks

Finished after 0.052 seconds

Runs: 1/1 Errors: 0 Failures: 0

cap1.versao6.replace_temp_with_query_calculo_total_pontos.parte2.TestesCap1 [Ru] Failure Trace

locacaoFilmes (0.000 s)

Versão 6: Reflexão

- A maioria das refatorações reduz a quantidade de código.
 - A anterior aumenta.
 - Ineficiência de **Java** para gerar um laço acumulador.
- Desempenho
 - O código anterior executava o **laço while** uma vez
 - A nova versão executa **3 vezes**
 - Compensa usar um **Profiler** para verificar se realmente o desempenho geral está sendo afetado.

Estamos agora no ponto
onde tiramos nosso chapéu de refatoração e
colocamos nosso chapéu de adição de
funções.



Versão 7: Saída Esperada para o método contaHtml ()

```
<h1>Registro de Locacao para <em>Rodrigo Martins Pagliares</em></h1>
<ul>
  <li>Encontros e Desencontros      6.5 </li>>
  <li>Meia-Noite em Paris           9.0 </li>
  <li>Paris, Texas                  3.5 </li>
  <li>Bravura Indômita              3.0 </li>
  <li>Shrek                         3.0 </li>
</ul>
<p>Total devido, <em>R$ 25.0</em></p>
<p>Pontos adquiridos na locação: <em></em></p>
```

Versão 7: Exercício

- Você tem 50min para criar um teste automatizado para verificar se a saída do seu programa é igual a saída apresentada no slide anterior

```
@Test
```

```
public void testeContaHTML( ) { ... }
```

- Após criado o teste, você deverá fornecer a implementação para que o teste passe.
 - Deve ser criado o método `contaHtml()` na classe cliente

Boa prática: Antes de incluir novas funcionalidades, escrever um teste unitário !

Versão 7: O Método contaHtml ()

```
public String contaHTML() {

    Enumeration locacoes = _locacoes.elements();
    String resultado = "<h1>Registro de Locação para <em>" + lerNome() + "</em></h1>\n"
        + "<ul>\n";

    while (locacoes.hasMoreElements()) {
        Locacao cada = (Locacao) locacoes.nextElement();

        //armazena o valor do filme para cada filme locado
        resultado += "\t" + "<li>" + cada.lerFilme().lerTitulo() + "\t"
            + String.valueOf(cada.lerPreco()) + "</li>\n";
    }

    resultado+="  
</ul>\n";

    //adiciona o rodape
    resultado += "<p>Total devido, <em>R$ " + String.valueOf(lerPrecoTotal()) + "</em></p>\n";
    resultado += "<p>Pontos adquiridos na locação: <em>" + String.valueOf(obj: lerTotalPontosLocadorFrequente()) + "</em></p>";

    return resultado;
}
```

Versão 7: Validando a Saída Esperada para o método contaHtml () via CodePen

The screenshot shows a CodePen editor interface. The top bar includes the CodePen logo, the text "codepen.io", and navigation icons. Below the top bar, the editor is titled "Untitled" and "Captain Anonymous". The editor has tabs for "HTML", "CSS", and "JS". The "HTML" tab is active, showing the following code:

```
1 <h1>Registro de Locacao para <em>Rodrigo Martins Pagliares</em></h1>
2 <ul>
3 <li>Encontros e Desencontros 6.5 </li>
4 <li>Meia-Noite em Paris 9.0 </li>
5 <li>Paris, Texas 3.5 </li>
6 <li>Bravura Indômita 3.0 </li>
7 <li>Shrek 3.0 </li>
8 </ul>
9 <p>Total devido, <em>R$ 25.0</em></p>
10 <p>Pontos adquiridos na locação: <em>6</em></p>
```

The rendered output on the right shows the HTML code converted to a visual representation. The title is "Registro de Locacao para *Rodrigo Martins Pagliares*". Below the title is a bulleted list of movies and their ratings:

- Encontros e Desencontros 6.5
- Meia-Noite em Paris 9.0
- Paris, Texas 3.5
- Bravura Indômita 3.0
- Shrek 3.0

Below the list, the text "Total devido, R\$ 25.0" is displayed. At the bottom, the text "Pontos adquiridos na locação: 6" is displayed.

Não copiamos e colamos as regras de cálculo,
assim se as regras de cálculo mudarem só
temos um local no código aonde ir.

Eclipse File Edit Source Refactor Navigate Search Project Run Window Help

workspaceUNIFAL - Refactoring/src/main/java/cap1/versao7/conta/html/Cliente.java - Eclipse IDE

Package Explorer

- cap1.versao9.parte2.move_method
- cap1.versao9.parte3.substituir_condicional_por_polimorfismo
- cap1.versao9.parte4.move_method_locador_frequente
- src/test/java
 - cap1.versao1
 - cap1.versao2.extract_method_rename_variable
 - cap1.versao3.move_method.parte1.manter_metodo
 - cap1.versao3.move_method.parte2.remover_metodo
 - cap1.versao4.replace_temp_with_query
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - cap1.versao6.replace_temp_with_query_calculo_total_conta.parte1
 - cap1.versao6.replace_temp_with_query_calculo_total_pontos.parte2
 - cap1.versao7.conta.html
 - TestesCap1.java
- cap1.versao8.polimorfismo.parte1.lerpreco
- cap1.versao8.polimorfismo.parte2.lerpontos.locador.frequente
- cap1.versao9.parte1.substituir_enum_state_strategy
- cap1.versao9.parte2.move_method
- cap1.versao9.parte3.substituir_condicional_por_polimorfismo
- cap1.versao9.parte4.move_method_locador_frequente
- refactoring.Refactoring
- JRE System Library [JavaSE-1.8]
- Maven Dependencies
- src
- target

Outline

- cap1.versao7.conta.html
- Cliente
 - _nome : String
 - _locacoes : Vector
 - Cliente(String)
 - adicionarLocacao(Locacao) : void
 - lerNome() : String
 - conta() : String
 - contaHtml() : String
 - lerTotalPontosLocadorFrequente() : int
 - lerPrecoTotal() : double

```
27 public String lerNome() {
28     return _nome;
29 }
30
31 public String conta() {
32     Enumeration locacoes = _locacoes.elements();
33     String resultado = "Registro de Locacao para " + lerNome() + "\n";
34     while (locacoes.hasMoreElements()) {
35         Locacao cada = (Locacao) locacoes.nextElement();
36         //mostra os valores para cada locação
37         resultado += "\t" + cada.lerFilme().lerTitulo() + "\t"
38             + String.valueOf(cada.lerPreco()) + "\n";
39     }
40     //adiciona o rodape
41     resultado += "Total devido, R$ " + String.valueOf(lerPrecoTotal()) + "\n";
42     resultado += "Pontos adquiridos na locacao : " + String.valueOf(lerTotalPontosLocadorFrequente());
43     return resultado;
44 }
45
46
47 public String contaHtml() {
```

Problems Console Progress Git Repositories Rebase Interactive JUnit Git Reflog Debug Tasks

Finished after 0.058 seconds

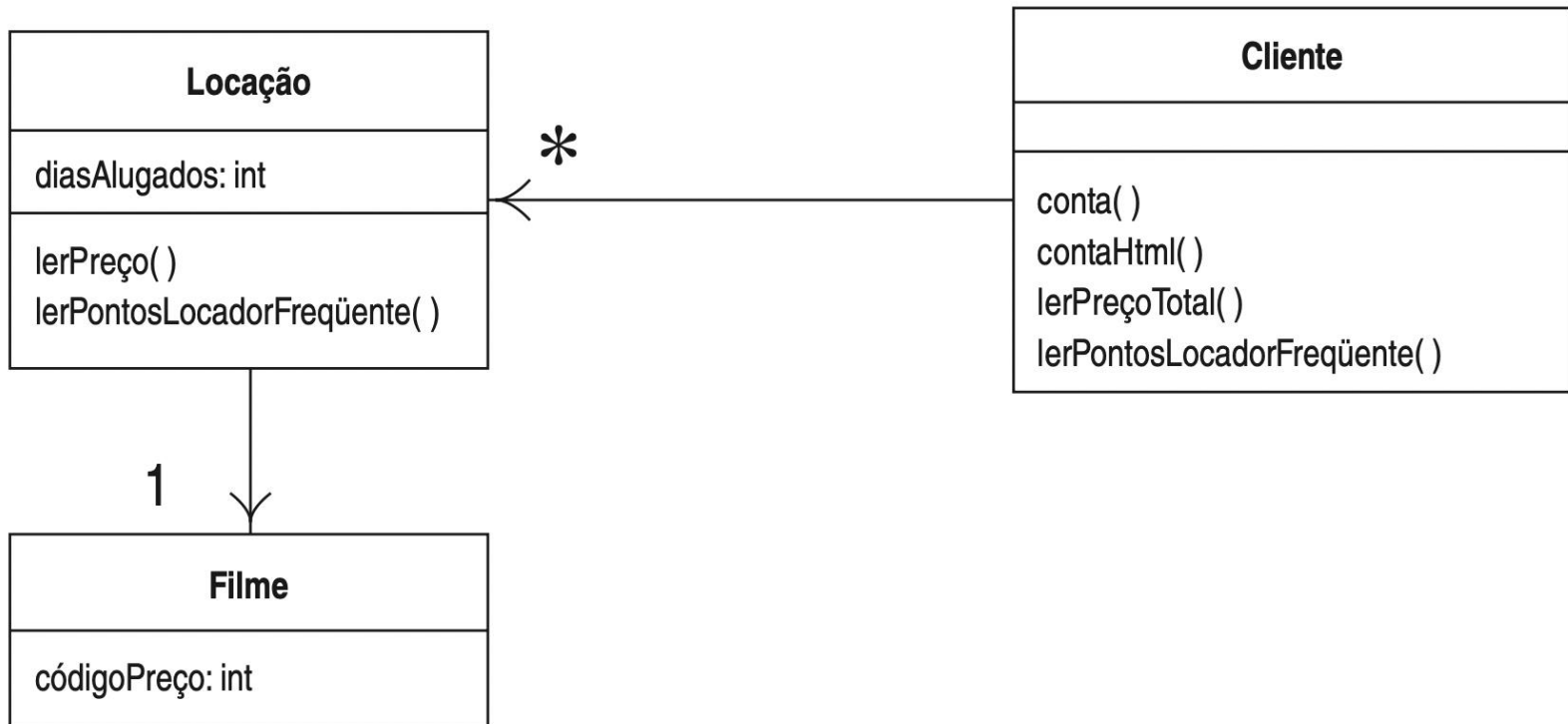
Runs: 2/2 Errors: 0 Failures: 0

cap1.versao7.conta.html.TestesCap1 [Runner: JUnit 4] (0.000 s)

- testeContaHTML (0.000 s)
- locacaoFilmes (0.000 s)

Failure Trace

cap1.versao7.conta.html.TestesCap1.java - Refactoring/src/test/java



Vamos refatorar mais este exemplo, eliminando mais duplicidades, extraindo métodos para cabeçalho, rodapé e linhas de detalhe no momento que formos estudar a refatoração

Criar um Método Padrão.

(Design pattern Template Method)

Mudança de Requisitos

- Alterar a classificação dos filmes.
 - Ainda não está claro que mudanças devem ser feitas.
 - Aparentemente novas classificações serão introduzidas, e as existentes poderão ser alteradas.
- As designações dos custos e pontos de locadores freqüentes dessas classificações ainda estão por ser determinadas.

De volta ao chapéu de refatoração.



Versão 8: novamente, Mover Método

- É uma **má idéia** fazer um **switch** baseado em um atributo de outro objeto.
 - **Switch deve ser usado sobre os próprios dados**, não nos de outros.
- Em nosso exemplo, temos a lógica condicional na classe `Locacao` baseada no código do preço do filme.

```
class Locacao { ...
public double lerPreco() {
    double resultado = 0;
    // Valor da locacao para cada filme dentro do Vector
    switch (lerFilme().lerCodigoPreco()) {
        case Filme.NORMAL:
            resultado += 2;
            if (lerDiasAlugados() > 2) {
                resultado += (lerDiasAlugados() - 2) * 1.5;
            }
            break;
        case Filme.LANCAMENTO_NOVO:
            resultado += lerDiasAlugados() * 3;
            break;
        case Filme.INFANTIL:
            resultado += 1.5;
            if (lerDiasAlugados() > 3) {
                resultado += (lerDiasAlugados() - 3) * 1.5;
            }
            break;
    }
    return resultado;
}
```

Switch no lugar
errado !

Versão 8: Exercício

- Você tem **10min** para aplicar a refatoração Mover Método no método `lerPreco()` exibido no slide anterior
 - Para qual classe ele deve ir?
 - Compensa ou não manter o método `lerPreco()` na classe original delegando a chamada para o método `lerPreco()` na nova classe?

```
class Filme { ...  
public double lerPreco(int diasAlugados) {  
  
    double resultado = 0;  
    switch (lerCodigoPreco()) {  
        case Filme.NORMAL:  
            resultado += 2; /  
            if (diasAlugados > 2) { /  
                resultado += (diasAlugados - 2) * 1.5;  
            }  
            break;  
        case Filme.LANCAMENTO_NOVO:  
            resultado += diasAlugados * 3;  
            break;  
        case Filme.INFANTIL:  
            resultado += 1.5;  
            if (diasAlugados > 3) {  
                resultado += (diasAlugados - 3) * 1.5;  
            }  
            break;  
    }  
    return resultado;  
}
```

Versão 8: novamente, Mover Método

```
Class Locacao { ...  
public double lerPreco() {  
    return _filme.lerPreco(_diasAlugados);  
}  
}
```

Delegação !

Vamos fazer o mesmo com o cálculo de pontos de locadores freqüentes.

Dessa forma isto mantém ambos os métodos que variam com o tipo de filme, junto à classe que possui o tipo.

Versão 8: Exercício

- Você tem 10min para aplicar a refatoração Mover Método no método lerPontosLocadorFrequente presente originalmente na classe Locacao
 - Para qual classe ele deve ir?
 - Compensa ou não manter o método lerPontosLocadorFrequente na classe original delegando a chamada para o método lerPreco na nova classe?

Versão 8: Novamente, Mover Método

```
class Locacao { ...  
    public int lerPontosLocadorFrequente() {  
        if ((lerFilme().lerCodigoPreco() == Filme.LANCAMENTO_NOVO)  
            && lerDiasAlugados() > 1) {  
            return 2;  
        }  
        return 1;  
    }  
}
```

Antes da Refatoração

```
class Locacao { ...  
    public int lerPontosLocadorFrequente() {  
        return _filme.lerPontosLocadorFrequente(_diasAlugados);  
    }  
}
```

Após a Refatoração:
Continua no próximo
slide !

Versão 8: Novamente, Mover Método

```
class Filme { ...  
    public int lerPontosLocadorFrequente(int numDiasAlugados) {  
        if ((lerCodigoPreco() == Filme.LANCAMENTO_NOVO)  
            && numDiasAlugados > 1) {  
            return 2;  
        }  
        return 1;  
    }  
}
```

Após a Refatoração

Eclipse File Edit Source Refactor Navigate Search Project Run Window Help

workspaceUNIFAL - Refactoring/src/main/java/cap1/versao8/polimorfismo/parte1/lerpreco/Locacao.java - Eclipse IDE

Package Explorer

- Object_Orientation
 - Refactoring
 - src/main/java
 - cap1.versao1
 - cap1.versao2.extract_method_rename_variable
 - cap1.versao3.move_method.parte1.manter_metodo
 - cap1.versao3.move_method.parte2.remover_metodo
 - cap1.versao4.replace_temp_with_query
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - cap1.versao6.replace_temp_with_query_calculo_total_conta.parte1
 - cap1.versao6.replace_temp_with_query_calculo_total_pontos.parte2
 - cap1.versao7.conta.html
 - cap1.versao8.polimorfismo.parte1.lerpreco
 - Cliente.java
 - Filme.java
 - Locacao.java
 - TesteCap1HTML.java
 - cap1.versao8.polimorfismo.parte2.lerpontos.locador.frequente
 - cap1.versao9.parte1.substituir_enum_state_strategy
 - cap1.versao9.parte2.move_method
 - cap1.versao9.parte3.substituir_condicional_por_polimorfismo
 - cap1.versao9.parte4.move_method_locador_frequente
 - src/test/java
 - cap1.versao1
 - cap1.versao2.extract_method_rename_variable

Outline

- cap1.versao8.polimorfismo.parte1.lerpreco
 - Locacao
 - _filme : Filme
 - _diasAlugados : int
 - Locacao(Filme, int)
 - lerDiasAlugados() : int
 - lerFilme() : Filme
 - lerPontosLocadorFrequente() : int
 - lerPreco() : double

Locacao.java

```
25 public Filme lerFilme() {
26     return _filme;
27 }
28
29
30
31 public int lerPontosLocadorFrequente() {
32     // Ponto adicional para o locatario para cada filme lancamento
33     if ((lerFilme().lerCodigoPreco() == Filme.LANCAMENTO_NOVO)
34         && lerDiasAlugados() > 1) {
35         return 2;
36     }
37     return 1;
38 }
39
40
41 public double lerPreco() {
42     return _filme.lerPreco(_diasAlugados);
43 }
44 }
45
```

JUnit

Finished after 0.063 seconds

Runs: 2/2 Errors: 0 Failures: 0

cap1.versao8.polimorfismo.parte1.lerpreco.TestesCap1 [Runner: JUnit 4] (0.000 s)

- testeContaHTML (0.000 s)
- locacaoFilmes (0.000 s)

Writable Smart Insert 44 : 2

Versão 9: Herança e o design pattern State

- Temos diversos tipos de filmes que têm diferentes maneiras de responder à mesma solicitação.
- Que tal este design?

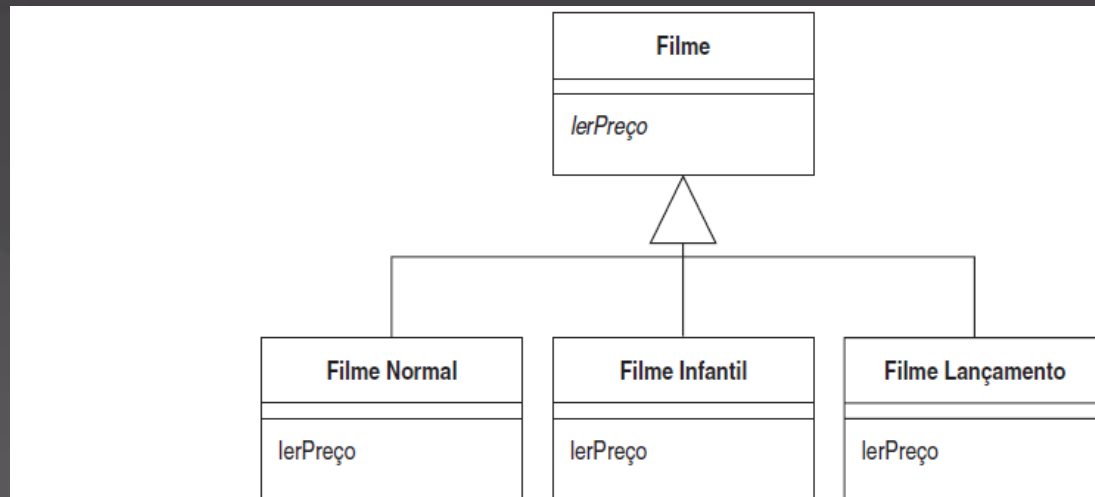


Figura 1.14 Usando herança na classe Filme.

Versão 9: Exercício

- Reflita sobre a afirmação :

Um filme pode mudar sua classificação durante sua existência. Um **objeto** não pode alterar sua **classe** durante sua **existência**.

Ela invalida o design da solução anterior?

Versão 9: Senso Crítico

- Um Filme durante sua existência pode possuir classificações distintas
 - O autor do livro usou este exemplo para justificar o **design pattern State**
- O autor chega a afirmar que a versão do slide anterior **não funciona**.
 - **Não concordo**. Para este exemplo, não necessitamos mudar a classificação do objeto em tempo de execução.
 - Um objeto poderia ser instanciado a partir do banco de dados.

Versão 9: Herança e o Design Pattern State

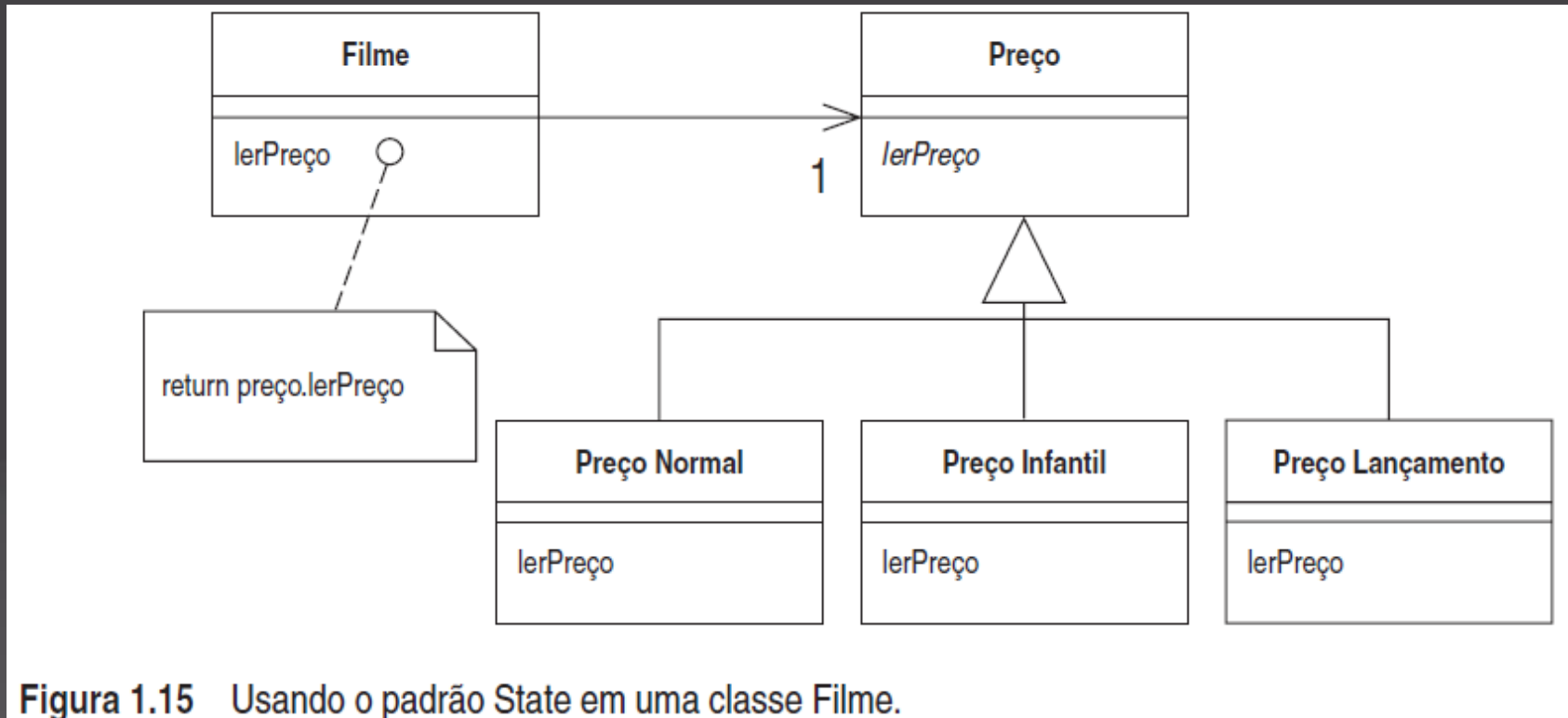


Figura 1.15 Usando o padrão State em uma classe Filme.

Apesar da versão anterior funcionar, vamos refatorar para este novo design, encapsulando o código que varia.

State ou Strategy?

Versão 9: Herança e o Design Pattern State

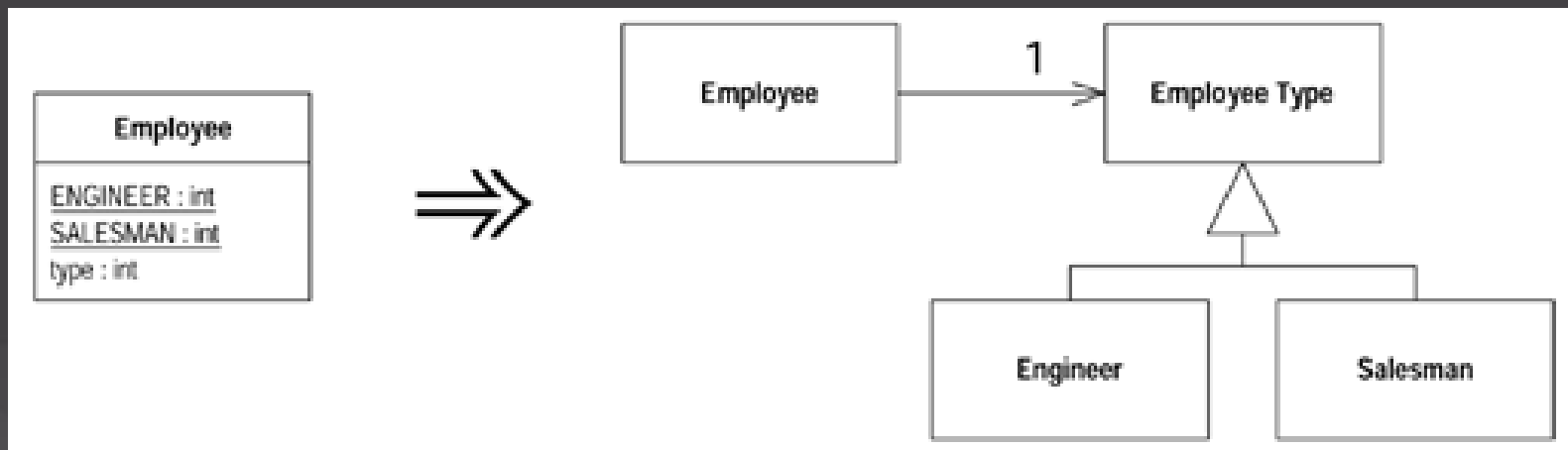
- Apresentaremos a evolução do design para o design pattern State com 3 refatorações menores:
 1. Substituir Enumeração pelo Padrão State/Strategy.
 2. Mover Método
 3. Substituir Comando Condicional por Polimorfismo

Versão 9, Parte 1: Substituir Enumeração pelo Padrão Strategy/State

Você possui um código com base em tipo que afeta o comportamento de uma classe, mas você não pode usar subclasse.

Substitua o código com base em tipo por um objeto State.

Versão 9, Parte 1: Substituir Enumeração pelo Padrão Strategy/State



Versão 9, Parte 1: Substituir Enumeração pelo Padrão Strategy/State - Passos

- Crie uma nova classe “**State**” para o código com base no tipo.
- Adicione subclasses ao objeto **State**, um para cada tipo.
- Crie uma **consulta abstrata** na superclasse para retornar o código do tipo. Sobrescreva nas subclasses para retornar o código correto.
- **Compile.**

Versão 9: Substituir Enumeração pelo Padrão Strategy/State - Passos

- Crie um **campo** no código original para o objeto **State**.
- Altere o código com consulta com base no tipo para delegar para o objeto State.
- Mude os métodos de configuração do código de tipo para atribuir uma instância da subclasse.
- **Compile e teste.**

Versão 9, Parte 1: Substituir Enumeração pelo Padrão State/Strategy

- Primeiramente, vamos encapsular o campo `preço` de forma que só possa ser enxergado via `set/get`

```
public Filme(String titulo, int codigoPreco) {  
    this._titulo = titulo;  
    this._codigoPreco = codigoPreco;  
}
```

- usar o método `gravar` em vez disso

```
public Filme(String titulo, int codigoPreco) {  
    this._titulo = titulo;  
    gravarCodigoPreco(codigoPreco); // set  
}
```

Versão 9, Parte 1: Substituir Enumeração pelo Padrão State/Strategy

- Primeiramente, vamos encapsular o campo `preço` de forma que só possa ser enxergado via `set/get`

Observe que nesta **primeira parte** estamos apenas trabalhando no método `lerCodigoPreco()` e NÃO no método `lerPreco(int numDiasAlugados)`.

Baby steps !

- usar o método `gravar` em vez disso

```
public Filme(String titulo, int codigoPreco) {  
    this._titulo = titulo;  
    gravarCodigoPreco(codigoPreco); // set  
}
```

Versão 9, Parte 1: Substituir Enumeração pelo Padrão State/Strategy

```
abstract class Preço {  
    abstract int lerCodigoPreco();  
}
```

```
class PreçoInfantil extends Preço {  
    int lerCodigoPreco() {  
        return Filme.INFANTIL;  
    }  
}
```

```
class PreçoLançamento extends Preço {  
    int lerCodigoPreco() {  
        return Filme.LANCAMENTO;  
    }  
}
```

```
class PreçoNormal extends Preço {  
    int lerCodigoPreco() {  
        return Filme.NORMAL;  
    }  
}
```

Versão 9, Parte 1: Classe Filme (antes)

```
class Filme { ...  
    private int _codigoPreço;  
  
    public int lerCodigoPreco () {  
        return _codigoPreço;  
    }  
  
    public void gravarCodigoPreço (int codigoPreco) {  
        _codigoPreço = codigoPreco;  
    }  
}
```

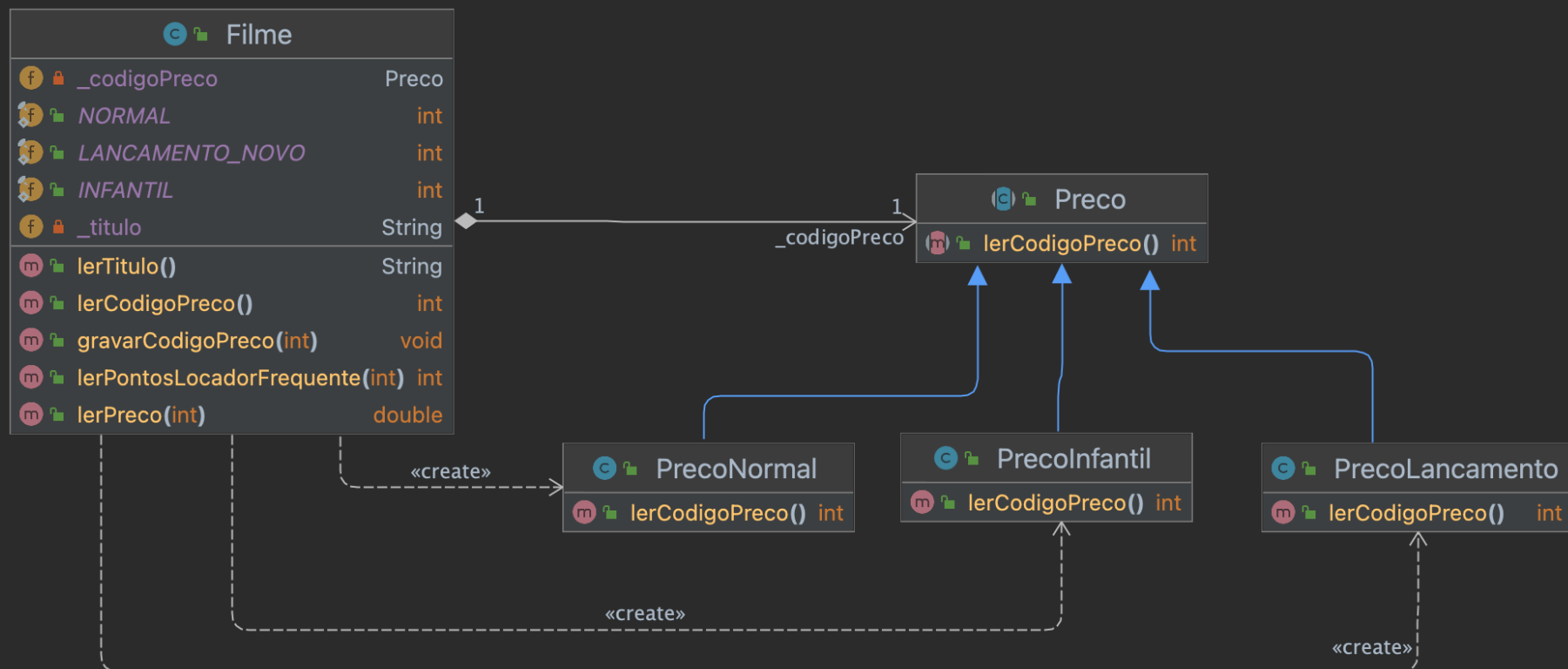
Versão 9, Parte 1 :Classe Filme (depois)

class Filme...

```
private Preço _preco;  
public int lerCodigoPreco() {  
    return _preco.lerCodigoPreco();  
}
```

```
public void gravarCodigoPreço(int codigoPreco) {  
    switch (codigoPreco) {  
        case NORMAL:  
            _preco = new PreçoNormal();  
            break;  
        case INFANTIL:  
            _preco = new PreçoInfantil();  
            break;  
        case LANÇAMENTO:  
            _preco = new PreçoLançamento();  
    }
```

O parâmetro continua sendo um int, mas agora ele é usado para determinar qual objeto Preço instanciar.



Compile e Teste !!

Eclipse File Edit Source Refactor Navigate Search Project Run Window Help

workspaceUNIFAL - Refactoring/src/main/java/cap1/versao9/parte1/substituir_enum_state_strategy/Filme.java - Eclipse IDE

Package Explorer

- cap1.versao9.parte3.substituir_condicional_por_polimorfismo
- cap1.versao9.parte4.move_method_locador_frequente
- src/test/java
 - cap1.versao1
 - cap1.versao2.extract_method_rename_variable
 - cap1.versao3.move_method.parte1.manter_metodo
 - cap1.versao3.move_method.parte2.remover_metodo
 - cap1.versao4.replace_temp_with_query
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - cap1.versao6.replace_temp_with_query_calculo_total_conta.parte1
 - cap1.versao6.replace_temp_with_query_calculo_total_pontos.parte2
 - cap1.versao7.conta.html
 - cap1.versao8.parte1.lerpreco
 - cap1.versao8.parte2.lerpontos.locador.frequente
 - cap1.versao9.parte1.substituir_enum_state_strategy
 - TestesCap1.java
 - cap1.versao9.parte2.move_method
 - cap1.versao9.parte3.substituir_condicional_por_polimorfismo
 - cap1.versao9.parte4.move_method_locador_frequente
 - refactoring.Refactoring
- JRE System Library [JavaSE-1.8]
- Maven Dependencies
- src
- target
- pom.xml

Outline

- cap1.versao9.parte1.substituir_enum_state_strategy
 - Filme
 - NORMAL : int
 - LANCAMENTO_NOVO : int
 - INFANTIL : int
 - _titulo : String
 - _preco : Preço
 - Filme(String, int)
 - lerCodigoPreco() : int
 - gravarCodigoPreco(int) : void
 - lerTitulo() : String
 - lerPreco(int) : double
 - lerPontosLocadorFrequente(int) : int

Filme.java

```
25
26 // Este metodo agora nao precisa apenas atribuir o codigo do preco a variavel de instancia _codigoPreco
27 // por uma variavel de referencia do tipo Preço. Por isso, precisamos receber o codigo como parametro e
28 // Preço deve ser instanciado e atribuido à nova variavel de instancia _preco. Este codigo ficava no meto
29 // para a hierarquia de precos.
30 public void gravarCodigoPreco(int codigoPreco) {
31     switch(codigoPreco){
32         case NORMAL:
33             _preco = new PreçoNormal();
34             break;
35         case INFANTIL:
36             _preco = new PreçoInfantil();
37             break;
38         case LANCAMENTO_NOVO:
39             _preco = new PreçoLancamento();
40             break;
41         default:
42             throw new IllegalArgumentException("Código de preço incorreto");
43     }
44 }
45 }
```

Problems Console Progress Git Repositories Rebase Interactive JUnit Git Reflog Debug Tasks

Finished after 0.057 seconds

Runs: 2/2 Errors: 0 Failures: 0

cap1.versao9.parte1.substituir_enum_state_strategy.TestesCap1 [Runner: JUnit 4] (Failure Trace)

- testeContaHTML (0.000 s)
- locacaoFilmes (0.000 s)

Versão 9, Parte 2: Substituir Enumeração pelo Padrão Strategy/State

```
class Filme...
    double lerPreço(int diasAlugados) {
        return _preco.lerPreço(diasAlugados);
    }

class Preço...
    double lerPreço(int diasAlugados) {
        double resultado = 0;
        switch (lerCodigoPreço()) {
            case Filme.NORMAL:
                resultado += 2;
                if (diasAlugados > 2)
                    resultado += (diasAlugados - 2) * 1.5;
                break;
            case Filme.LANÇAMENTO:
                resultado += diasAlugados * 3;
                break;
            case Filme.INFANTIL: ....
        }
```

Usamos Mover
Método em
lerPreço()
da classe
Cliente para a
classe Preço

Compile e Teste !!

Eclipse File Edit Source Refactor Navigate Search Project Run Window Help

workspaceUNIFAL - Refactoring/src/main/java/cap1/versao9/parte2/move_method/Preco.java - Eclipse IDE

Package Explorer

- Cliente.java
- Filme.java
- Locacao.java
- Preco.java
- PrecoInfantil.java
- PrecoLancamento.java
- PrecoNormal.java
- TesteCap1HTML.java
- cap1.versao9.parte3.substituir_condicional_por_polimorfismo
- cap1.versao9.parte4.move_method_locador_frequente
- src/test/java
 - cap1.versao1
 - cap1.versao2.extract_method_rename_variable
 - cap1.versao3.move_method.parte1.manter_metodo
 - cap1.versao3.move_method.parte2.remover_metodo
 - cap1.versao4.replace_temp_with_query
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - cap1.versao6.replace_temp_with_query_calculo_total_conta.parte1
 - cap1.versao6.replace_temp_with_query_calculo_total_pontos.parte2
 - cap1.versao7.conta.html
 - cap1.versao8.parte1.lerpreco
 - cap1.versao8.parte2.lerpontos.locador.frequente
 - TestesCap1.java
 - cap1.versao9.parte1.substituir_enum_state_strategy
 - TestesCap1.java
 - cap1.versao9.parte2.move_method
 - TestesCap1.java
 - cap1.versao9.parte3.substituir_condicional_por_polimorfismo
 - cap1.versao9.parte4.move_method_locador_frequente
 - refactoring.Refactoring

Preco.java

```
11 public abstract class Preco {
12     abstract int lerCodigoPreco();
13
14     public double lerPreco(int diasAlugados) {
15
16         double resultado = 0;
17         // Valor da locacao para cada filme dentro do Vector
18         switch (lerCodigoPreco()) { // codigoPreco associado ao tipo do filme
19             case Filme.NORMAL:
20                 resultado += 2; // R$ 2,00 por filme normal alugado
21                 if (diasAlugados > 2) { // Multa de 1.50 por dia de atraso caso seja entregue apos 2 dias
22                     resultado += (diasAlugados - 2) * 1.5;
23                 }
24                 break;
25             case Filme.LANCAMENTO_NOVO:
26                 resultado += diasAlugados * 3; // R$ 3,00 a diaria para lancamentos por filme alugado
27
28                 break;
29             case Filme.INFANTIL:
30                 resultado += 1.5; // R$ 1,50 por 3 dias por filme infantil alugado
31                 if (diasAlugados > 3) {
32                     resultado += (diasAlugados - 3) * 1.5; // Multa de 1,50 por dia de atraso caso seja entregue a
33                 }
34                 break;
35         }
```

Outline

- cap1.versao9.parte2.move_method
- Preco
 - lerCodigoPreco() : int
 - lerPreco(int) : double

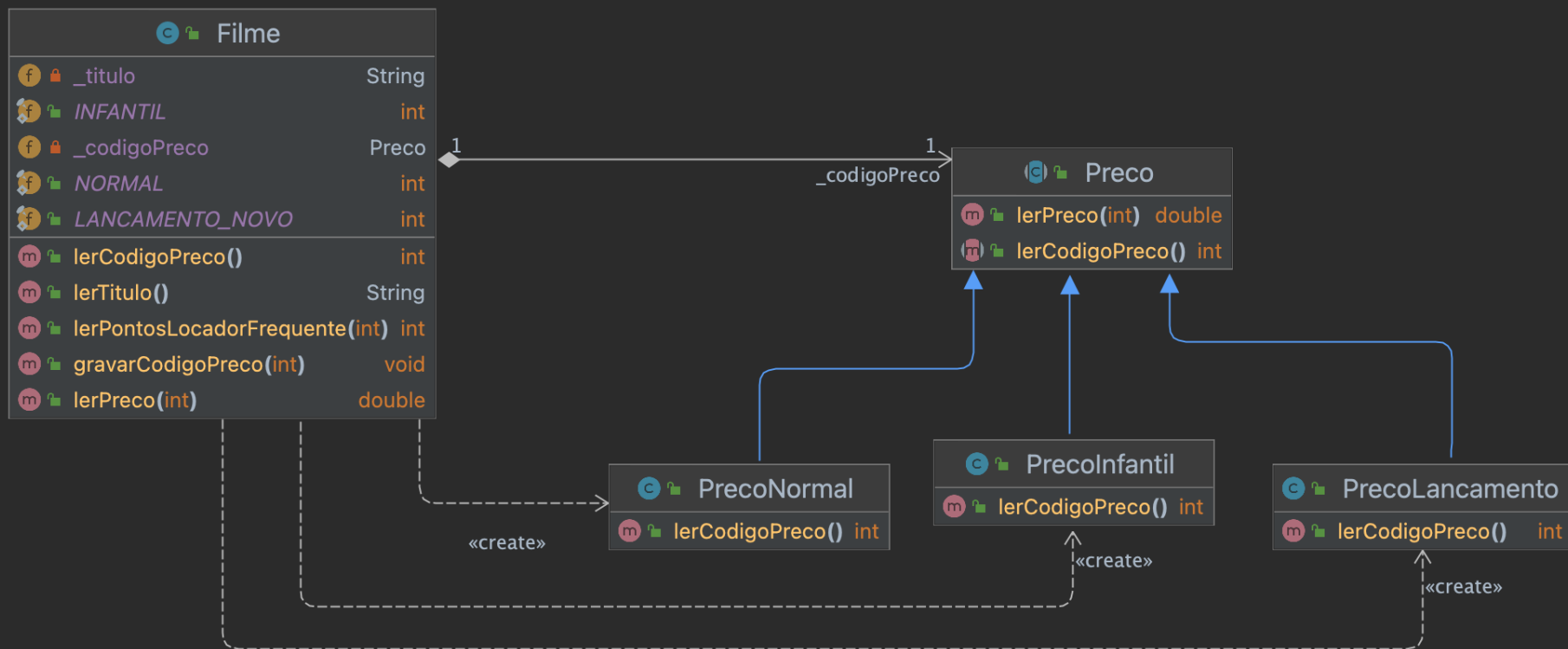
Problems Console Progress Git Repositories Rebase Interactive JUnit Git Reflog Debug Tasks

Finished after 0.046 seconds

Runs: 2/2 Errors: 0 Failures: 0

cap1.versao9.parte2.move_method.TestesCap1 [Runner: JUnit 4] (0.001 s)

- testeContaHTML (0.000 s)
- locacaoFilmes (0.001 s)



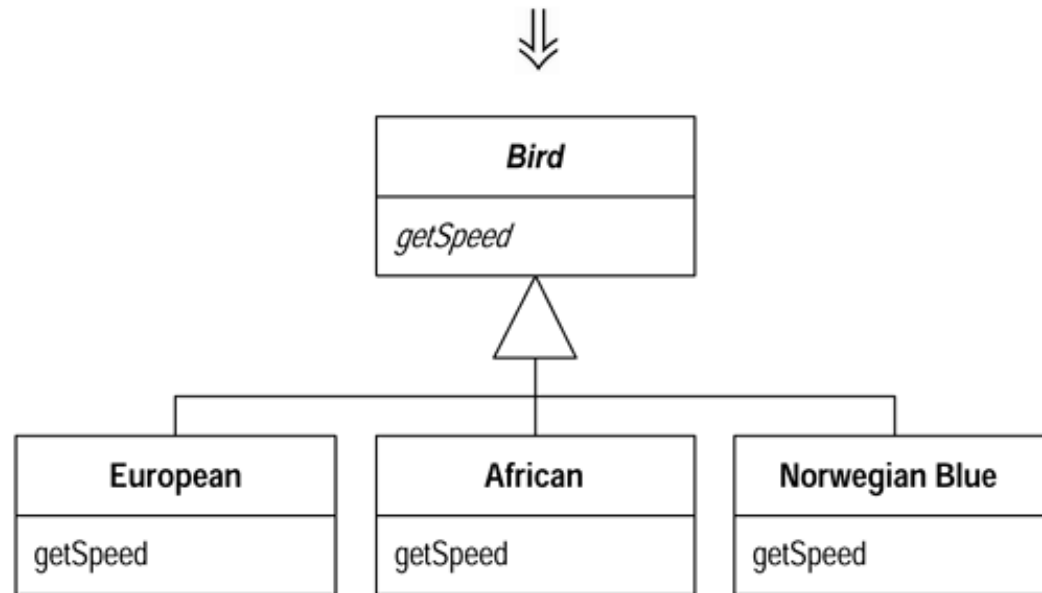
Versão 9, Parte 3: Substituir Comando Condicional por Polimorfismo

Você possui uma lógica condicional que escolhe entre diferentes comportamentos dependendo do tipo de um objeto.

Mude cada ramificação do condicional para um método sobrescrito em uma subclasse. Torne o método original, abstrato.

Versão 9, Parte 3: Substituir Comando Condicional por Polimorfismo

```
double getSpeed() {  
    switch (_type) {  
        case EUROPEAN:  
            return getBaseSpeed();  
        case AFRICAN:  
            return getBaseSpeed() - getLoadFactor() * _numberOfCoconuts;  
        case NORWEGIAN_BLUE:  
            return (_isNailed) ? 0 : getBaseSpeed(_voltage);  
    }  
    throw new RuntimeException ("Should be unreachable");  
}
```



Versão 9, Parte 3: Substituir Comando Condicional por Polimorfismo - Passos

- Mova o switch para a superclasse da estrutura de herança.
- Copie uma ramificação da instrução “case” para a subclasse.
- **Compile e teste.**
- Repita para todas as outras ramificações.
- Substitua a **instrução case** com um método abstrato.

Versão 9, Parte 3: Substituir Comando Condicional por Polimorfismo

class **Preco...** // Antes de usar Substituir Comando Condicional por Polimorfismo

```
double lerPreco(int diasAlugados) {  
    double resultado = 0;  
    switch (lerCodigoPreco()) {  
        case Filme.NORMAL:  
            resultado += 2;  
            If (diasAlugados > 2)  
                resultado += (diasAlugados - 2) * 1.5;  
            break;  
        case Filme.LANÇAMENTO:  
            resultado += diasAlugados * 3;  
            break;  
        case Filme.INFANTIL:  
            resultado += 1.5;  
            if (diasAlugados > 3)  
                resultado += (diasAlugados - 3) * 1.5;  
            break;  
    }  
    return resultado;  
}
```

Versão 9, Parte 3: Substituir Comando Condicional por Polimorfismo

- Vamos tomar uma das alternativas do comando *case por vez e criando no seu lugar um método sobrescrito.*
 - *Começando com PrecoNormal:*

```
class PrecoNormal ...  
    double lerPreço (int diasAlugados) {  
        double resultado = 2;  
        if (diasAlugados > 2)  
            resultado += (diasAlugados - 2) * 1.5;  
        return resultado;  
    }  
}
```

- Isso anula o *case* correspondente na superclasse. Vamos fazer o mesmo para as outras ramificações.

Versão 9, Parte 3: Substituir Comando Condicional por Polimorfismo

```
class PrecoInfantil ...
```

```
    double lerPreco (int diasAlugados) {  
        double resultado = 1.5;  
        If (diasAlugados > 3)  
            resultado += (diasAlugados - 3) * 1.5;  
        return resultado;  
    }
```

```
class PreçoLançamento ...
```

```
    double lerPreco (int diasAlugados) {  
        return diasAlugados * 3;  
    }
```

Compile e Teste !!

Eclipse File Edit Source Refactor Navigate Search Project Run Window Help

workspaceUNIFAL - Refactoring/src/main/java/cap1/versao9/parte3/substituir_condicional_por_polimorfismo/PrecoInfantil.java - Eclipse IDE

Package Explorer

- PrecoInfantil.java
- PrecoLancamento.java
- PrecoNormal.java
- TesteCap1HTML.java
- cap1.versao9.parte4.move_method_locador_frequente
- src/test/java
 - cap1.versao1
 - cap1.versao2.extract_method_rename_variable
 - cap1.versao3.move_method.parte1.manter_metodo
 - cap1.versao3.move_method.parte2.remover_metodo
 - cap1.versao4.replace_temp_with_query
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - cap1.versao6.replace_temp_with_query_calculo_total_conta.parte1
 - cap1.versao6.replace_temp_with_query_calculo_total_pontos.parte2
 - cap1.versao7.conta.html
 - cap1.versao8.parte1.lerpreco
 - cap1.versao8.parte2.lerpontos.locador.frequente
 - cap1.versao9.parte1.substituir_enum_state_strategy
 - cap1.versao9.parte2.move_method
 - cap1.versao9.parte3.substituir_condicional_por_polimorfismo
 - TestesCap1.java
 - cap1.versao9.parte4.move_method_locador_frequente
 - refactoring.Refactoring
- JRE System Library [JavaSE-1.8]
- Maven Dependencies
- src
- target
- pom.xml

Preco.java TestesCap1.java PrecoInfantil.java

```
2+ * To change this template, choose Tools | Templates
5 package cap1.versao9.parte3.substituir_condicional_por_polimorfismo;
6
7 /**
8  *
9  * @author pagliares
10 */
11 public class PrecoInfantil extends Preco {
12
13     @Override
14     int lerCodigoPreco() {
15         return Filme.INFANTIL;
16     }
17
18     @Override
19     public double lerPreco(int diasAlugados) {
20         double resultado = 1.5; // R$ 1,50 por 3 dias por filme infantil alugado
21         if (diasAlugados > 3) {
22             resultado += (diasAlugados - 3) * 1.5; // Multa de 1,50 por dia de atraso caso seja entregue apos 3
23         }
24         return resultado;
25     }
26 }
```

Outline

- cap1.versao9.parte3.substituir_condicional_por_polimorfismo
- PrecoInfantil
 - lerCodigoPreco() : int
 - lerPreco(int) : double

Problems Console Progress Git Repositories Rebase Interactive JUnit Git Reflog Debug Tasks

Finished after 0.048 seconds

Runs: 2/2 Errors: 0 Failures: 0

cap1.versao9.parte3.substituir_condicional_por_polimorfismo.TestesCap1 [Runner: ...] Failure Trace

- testeContaHTML (0.000 s)
- locacaoFilmes (0.000 s)

Versão 9, Parte 3: Substituir Comando Condicional por Polimorfismo

- Quando tiver terminado todos os passos, declaramos o método `lerPreco()` da classe `Preco` como abstrato.

```
class Preco ..  
    abstract double lerPreço(int diasAlugados);
```

Compile e Teste !!

Eclipse File Edit Source Refactor Navigate Search Project Run Window Help

workspaceUNIFAL - Refactoring/src/main/java/cap1/versao9/parte3/substituir_condicional_por_polimorfismo/Preco.java - Eclipse IDE

Package Explorer

- PrecoInfantil.java
- PrecoLancamento.java
- PrecoNormal.java
- TesteCap1HTML.java
- cap1.versao9.parte4.move_method_locador_frequente
- src/test/java
 - cap1.versao1
 - cap1.versao2.extract_method_rename_variable
 - cap1.versao3.move_method.parte1.manter_metodo
 - cap1.versao3.move_method.parte2.remover_metodo
 - cap1.versao4.replace_temp_with_query
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - cap1.versao6.replace_temp_with_query_calculo_total_conta.parte1
 - cap1.versao6.replace_temp_with_query_calculo_total_pontos.parte2
 - cap1.versao7.conta.html
 - cap1.versao8.parte1.lerpreco
 - cap1.versao8.parte2.lerpontos.locador.frequente
 - cap1.versao9.parte1.substituir_enum_state_strategy
 - cap1.versao9.parte2.move_method
 - cap1.versao9.parte3.substituir_condicional_por_polimorfismo
 - TestesCap1.java
 - cap1.versao9.parte4.move_method_locador_frequente
 - refactoring.Refactoring
- JRE System Library [JavaSE-1.8]
- Maven Dependencies
- src
- target
- pom.xml

Outline

- cap1.versao9.parte3.substituir_condicional_por_polimorfismo
 - Preco
 - lerCodigoPreco() : int
 - lerPreco(int) : double

TestesCap1.java

```
2+ * To change this template, choose Tools | Templates
5 package cap1.versao9.parte3.substituir_condicional_por_polimorfismo;
6
7 /**
8  *
9  * @author pagliares
10 */
11 public abstract class Preco {
12     abstract int lerCodigoPreco();
13
14     public abstract double lerPreco(int diasAlugados);
15
16
17
18 }
19
```

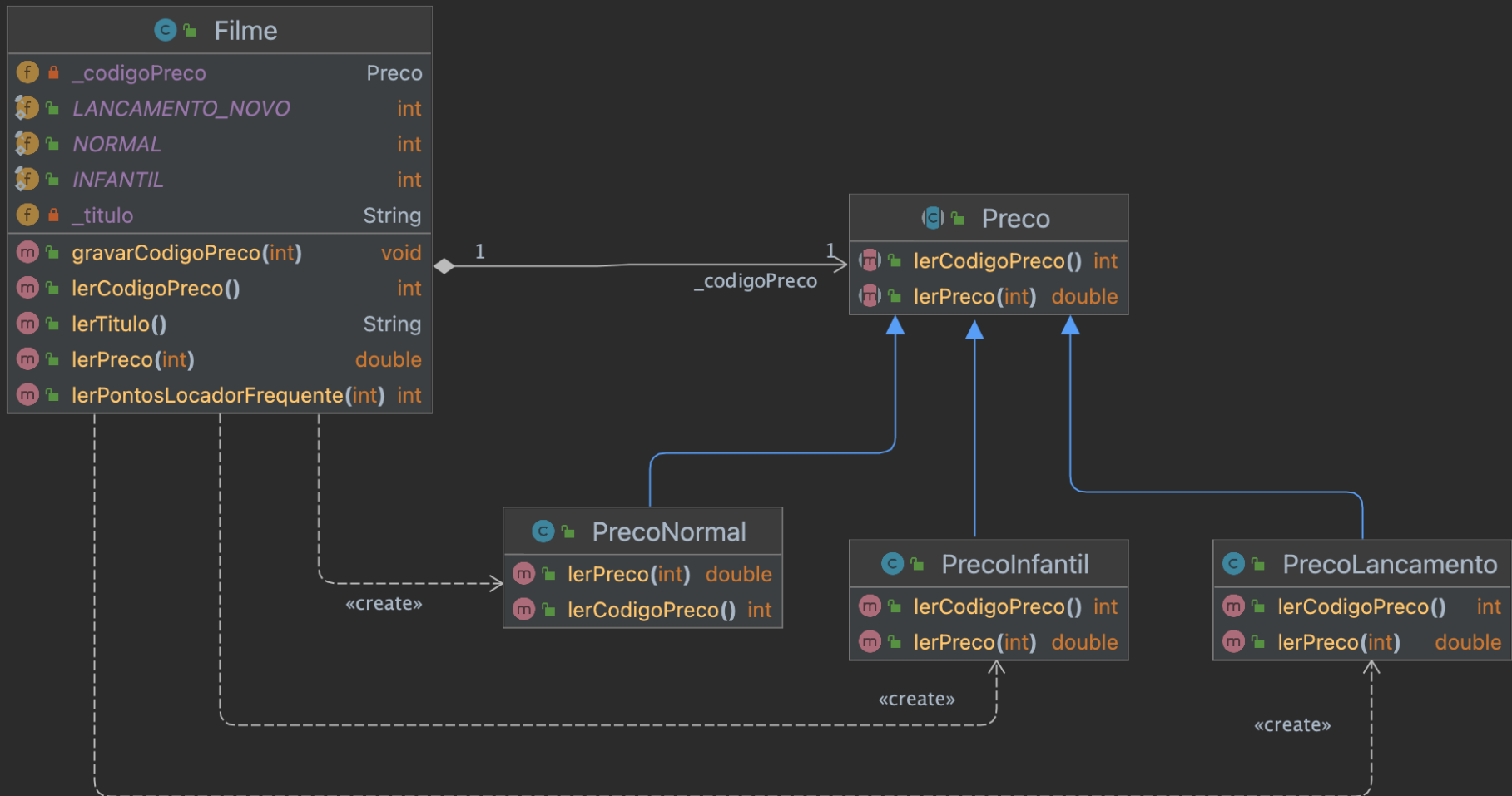
Problems Console Progress Git Repositories Rebase Interactive JUnit Git Reflog Debug Tasks

Finished after 0.041 seconds

Runs: 2/2 Errors: 0 Failures: 0

cap1.versao9.parte3.substituir_condicional_por_polimorfismo.TestesCap1 [Runner: . Failure Trace]

- testeContaHTML (0.000 s)
- locacaoFilmes (0.000 s)



Versão 9, Parte 4: Substituir Comando Condicional por Polimorfismo

- Agora podemos fazer o mesmo com `lerPontosLocadorFrequente( )`, movendo-o para a classe `Preco`

```
class Filme... // Antes de mover
    int lerPontosLocadorFrequente(int diasAlugados) {
        if ((lerCodigoPreco() == Filme.LANCAMENTO) && diasAlugados > 1)
            return 2;
        else
            return 1;
    }
```

Versão 9, Parte 4: Substituir Comando Condicional por Polimorfismo

Class **Filme** ...

```
int lerPontosLocadorFrequente (int diasAlugados) {  
    return _preco.lerPontosLocadorFrequente(diasAlugados);  
}
```

← Deleção

Class **Preço** ...

```
int lerPontosLocadorFrequente (int diasAlugados) {  
    if (( lerCodigoPreço () == Filme.LANCAMENTO) && (diasAlugados > 1)  
        return 2;  
    else  
        return 1;  
}
```

Versão 9, Parte 4: Substituir Comando Condicional por Polimorfismo

- Como o cálculo só varia para lançamentos, não torno o método da superclasse abstrato.
 - Em vez disso, crio um método de sobrescrita somente para os lançamentos e deixo um método definido (como *default*) na superclasse:

```
class PreçoLançamento {  
    int lerPontosLocadorFrequente (int diasAlugados) {  
        return (diasAlugados > 1)? 2:1;  
    }  
  
class Preço ...  
    int lerPontosLocadorFrequente (int diasAlugados) {  
        return 1;  
    }  
}
```

Eclipse File Edit Source Refactor Navigate Search Project Run Window Help

workspaceUNIFAL - Refactoring/src/main/java/cap1/versao9/parte4/move_method_locador_frequente/PrecoLancamento.java - Eclipse IDE

Package Explorer

- Locacao.java
- Preco.java
- PrecoInfantil.java
- PrecoLancamento.java
- PrecoNormal.java
- TesteCap1HTML.java
- src/test/java
 - cap1.versao1
 - cap1.versao2.extract_method_rename_variable
 - cap1.versao3.move_method.parte1.manter_metodo
 - cap1.versao3.move_method.parte2.remover_metodo
 - cap1.versao4.replace_temp_with_query
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - cap1.versao6.replace_temp_with_query_calculo_total_conta.parte1
 - cap1.versao6.replace_temp_with_query_calculo_total_pontos.parte2
 - cap1.versao7.conta.html
 - cap1.versao8.parte1.lerpreco
 - cap1.versao8.parte2.lerpontos.locador.frequente
 - cap1.versao9.parte1.substituir_enum_state_strategy
 - cap1.versao9.parte2.move_method
 - cap1.versao9.parte3.substituir_condicional_por_polimorfismo
 - cap1.versao9.parte4.move_method_locador_frequente
 - TestesCap1.java
 - refactoring.Refactoring
 - JRE System Library [JavaSE-1.8]
 - Maven Dependencies
 - src
 - target

Outline

- cap1.versao9.parte4.move_method_locador_frequente
 - PrecoLancamento
 - lerCodigoPreco() : int
 - lerPreco(int) : double
 - lerPontosLocadorFrequente(int) : int

PrecoLancamento.java

```
6
7 /**
8  *
9  * @author pagliares
10 */
11 public class PrecoLancamento extends Preco {
12
13     @Override
14     int lerCodigoPreco() {
15         return Filme.LANCAMENTO_NOVO;
16     }
17
18     @Override
19     public double lerPreco(int diasAlugados) {
20         return diasAlugados * 3; // R$ 3,00 a diaria para lancamentos por filme alugado
21     }
22
23     @Override
24     public int lerPontosLocadorFrequente(int numDiasAlugados) {
25         return (numDiasAlugados > 1)? 2:1;
26     }
27 }
28
```

Problems Console Progress Git Repositories Rebase Interactive JUnit Git Reflog Debug Tasks

Finished after 0.04 seconds

Runs: 2/2 Errors: 0 Failures: 0

cap1.versao9.parte4.move_method_locador_frequente.TestesCap1 [Runner: JUnit 4] Failure Trace

- testeContaHTML (0.001 s)
- locacaoFilmes (0.001 s)

Versão 9: Senso Crítico

- Aplicar o padrão **State** foi um esforço e tanto. Valeu a pena?
 - O ganho é que, se eu alterar qualquer comportamento do preço, acrescentar novos preços ou acrescentar comportamento adicional dependente do preço, essa mudança será muito mais fácil de ser feita.
 - O resto da aplicação não fica sabendo a respeito do uso do padrão **State**.

Versão 9: Senso Crítico

- Para a pequena quantidade de comportamento, dependente do preço, que tenho no momento, isso não é grande coisa.
 - Porém, em um sistema mais complexo, com uma dúzia ou mais de métodos dependentes do preço, isso faria uma grande diferença.

Versão 9: Senso Crítico

- Vai ser muito mais fácil alterar a estrutura de classificação de filmes e mudar as regras de cobrança e o sistema de pontuação de locadores freqüentes.

Comparação: diagrama de classes da versão inicial

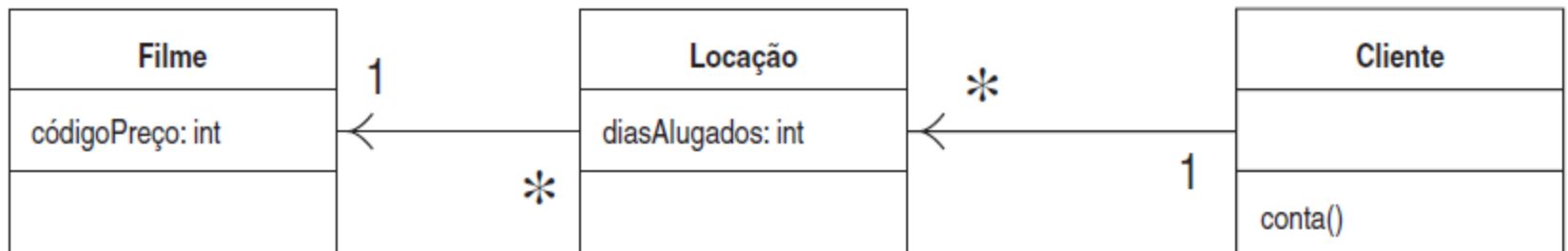
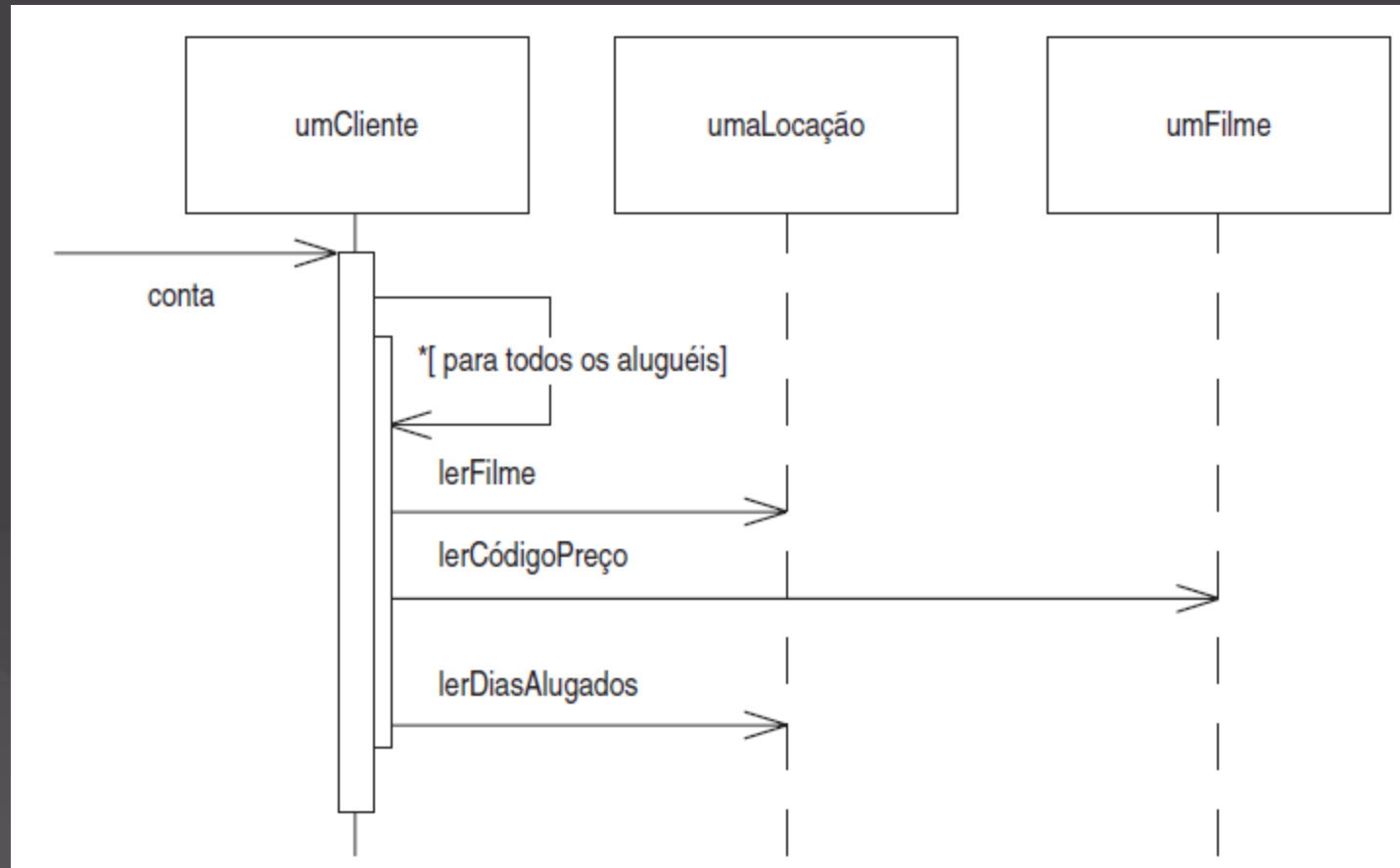


Figura 1.1 Diagrama de classes das classes iniciais. Apenas as características mais importantes estão mostradas. A notação é UML [Fowler, UML].

Comparação: Diagrama de sequência da versão inicial



Comparação: Diagrama de classes da versão final

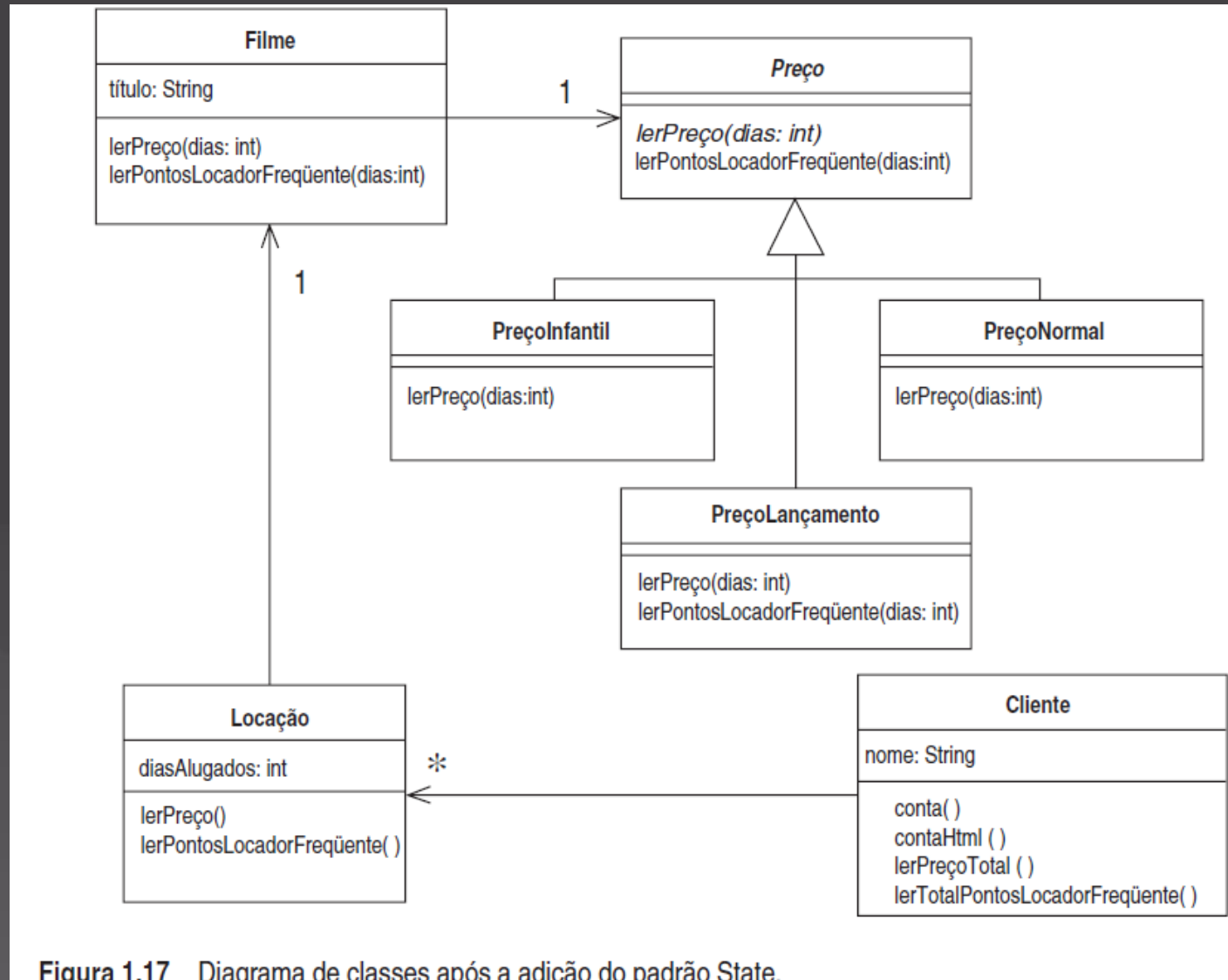


Figura 1.17 Diagrama de classes após a adição do padrão State.

Comparação: Diagrama de Sequência da versão final

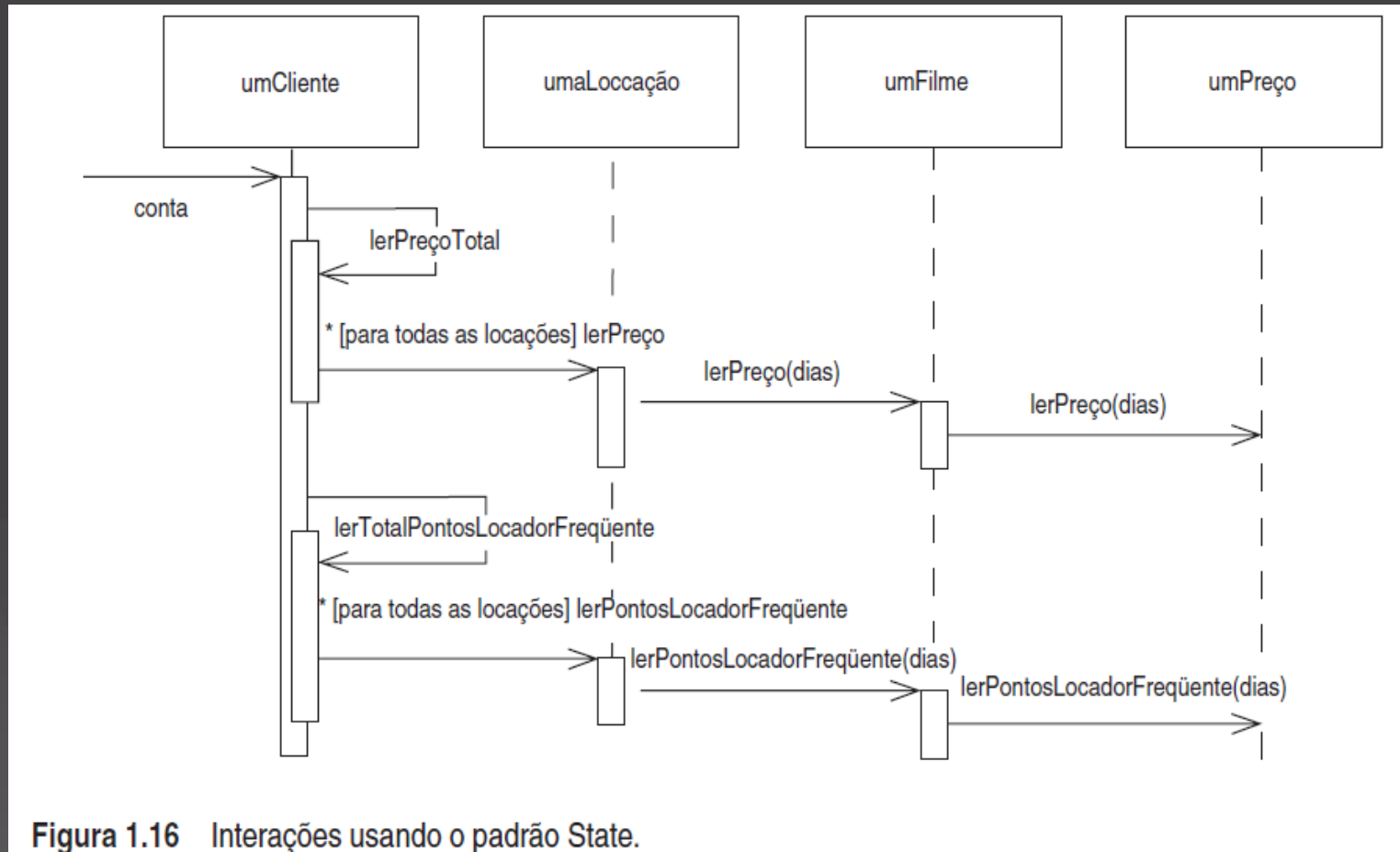


Figura 1.16 Interações usando o padrão State.

Exercício extra #1 – Mudança de requisitos

- Decidimos incorporar mais um tipo de filme: Musical
 - R\$ 2,00 a locação com direito a ficar com o filme por 2 dias
- Multa calculada da seguinte forma:
 - R\$ 1,50 de multa, por dia de atraso caso seja entregue **após 2 dias**.
- Pontos do locador freqüente.
 - Cada filme alugado conta **1 ponto**

Exercício extra #1 – Mudança de requisitos (solução)

```
public abstract class Filme { ..  
    public static final int INFANTIL = 2; // R$ 1,50  
    public static final int MUSICAL = 3; // R$ 2,00  
  
    public void gravarCodigoPreco(int codigoPreco) {  
        switch(codigoPreco){  
            ...  
            case LANCAMENTO_NOVO:  
                _preco = new PrecoLancamento();  
                break;  
            case MUSICAL:  
                _preco = new PrecoMusical();  
                break;  
            default:  
                throw new IllegalArgumentException("Código de preço incorreto");  
        }  
    }  
}
```

Exercício extra #1 – Mudança de requisitos (solução)

```
public class PrecoMusical extends Preco {  
    @Override  
    int lerCodigoPreco() {  
        return Filme.MUSICAL;  
    }  
}
```

```
    @Override  
    public double lerPreco(int diasAlugados) {  
        double resultado = 2.0; // R$ 2,00 por 2 dias por filme musical alugado  
        if (diasAlugados > 2) {  
            resultado += (diasAlugados - 2) * 1.5; // Multa de 1,50 por dia de atraso apos 2 dias.  
        }  
        return resultado;  
    }  
}
```

Exercício extra #1 – Mudança de requisitos (solução)

```
public class TestesCap1 {  
    ...  
    @Test  
    public void locacaoFilmes() {  
        String saidaEsperadaTemp = "Registro de Locacao para Rodrigo Martins Pagliares\n"  
            + "\tEncontros e Desencontros 6.5\n"  
            + "\tMeia-Noite em Paris          9.0\n"  
            + "\tParis, Texas          3.5\n"  
            + "\tBravura Indômita        3.0\n"  
            + "\tShrek                    3.0\n"  
            + "\tHair                      2.0\n"  
            + "Total devido, R$ 27.0\n"  
            + "Pontos adquiridos na locacao : 7\n";  
        ...  
        ...  
        rodrigo.adicionarLocacao(loc5);  
        rodrigo.adicionarLocacao(loc6);  
        assertEquals(saidaEsperadaTemp.trim(), rodrigo.conta().trim());  
    }  
}
```

Código similar dever ser
Implementado para o teste
testeContaHTML ()

Eclipse File Edit Source Refactor Navigate Search Project Run Window Help

workspaceUNIFAL - Refactoring/src/test/java/extra/exercicio1/TestesCap1.java - Eclipse IDE

Package Explorer

- Cliente.java
- Filme.java
- Locacao.java
- Preco.java
- PrecoInfantil.java
- PrecoLancamento.java
- PrecoMusical.java
- PrecoNormal.java
- TesteCap1HTML.java
- extra.exercicio2
- extra.exercicio3
- src/test/java
 - cap1.versao1
 - cap1.versao2.extract_method_rename_variable
 - cap1.versao3.move_method.parte1.manter_metodo
 - cap1.versao3.move_method.parte2.remover_metodo
 - cap1.versao4.replace_temp_with_query
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - cap1.versao6.replace_temp_with_query_calculo_total_conta.parte1
 - cap1.versao6.replace_temp_with_query_calculo_total_conta.parte2
 - cap1.versao7.conta.html
 - cap1.versao8.parte1.lerpreco
 - cap1.versao8.parte2.lerpontos.locador.frequente
 - cap1.versao9.parte1.substituir_enum_state_strategy
 - cap1.versao9.parte2.move_method
 - cap1.versao9.parte3.substituir_condicional_por_polimorfismo
 - cap1.versao9.parte4.move_method_locador_frequente
 - extra.exercicio1

TestesCap1.java

```
106 Filme indomita = new Filme("Bravura Indômita", 1);
107 Locacao loc4 = new Locacao(indomita, 1);
108
109 // Quinto filme sendo locado(infantil) (Resultado = R$ 1,50 locacao + R$ 1,50 de multa - Total parci
110 Filme shrek = new Filme("Shrek", 2);
111 Locacao loc5 = new Locacao(shrek, 4);
112
113 // Sexto filme sendo locado(infantil) (Resultado = R$ 2,00 locacao + R$ 0,00 de multa. Total parcial
114 Filme hair = new Filme("Hair", 3);
115 Locacao loc6 = new Locacao(hair, 2);
116
117 rodrigo.adicionarLocacao(loc1);
118 rodrigo.adicionarLocacao(loc2);
119 rodrigo.adicionarLocacao(loc3);
120 rodrigo.adicionarLocacao(loc4);
121 rodrigo.adicionarLocacao(loc5);
122 rodrigo.adicionarLocacao(loc6);
123
124
125 assertEquals(saidaEsperadaTemp.trim(), rodrigo.contaHtml().trim());
126 }
127 }
128
```

Outline

- extra.exercicio1
- TestesCap1
 - TestesCap1()
 - setUpClass() : void
 - tearDownClass() : void
 - locacaoFilmes() : void
 - testeContaHTML() : void

Problems Console Progress Git Repositories Rebase Interactive JUnit Git Reflog Debug Tasks

Finished after 0.054 seconds

Runs: 2/2 Errors: 0 Failures: 0

extra.exercicio1.TestesCap1 [Runner: JUnit 4] (0.001 s)

- testeContaHTML (0.000 s)
- locacaoFilmes (0.001 s)

Exercício Extra #2 - Herança

- Você tem **20min** para alterar a **versão 8** de nosso código para aderir ao **design** da figura abaixo.
 - Os testes deverão ser modificados?
 - O que aconteceu com o método **lerPreço** e com a classe **Filme** após a mudança?
 - Quais foram as mudanças no método **lerPontosLocadorFrequente** para aderir ao **design** do slide anterior?

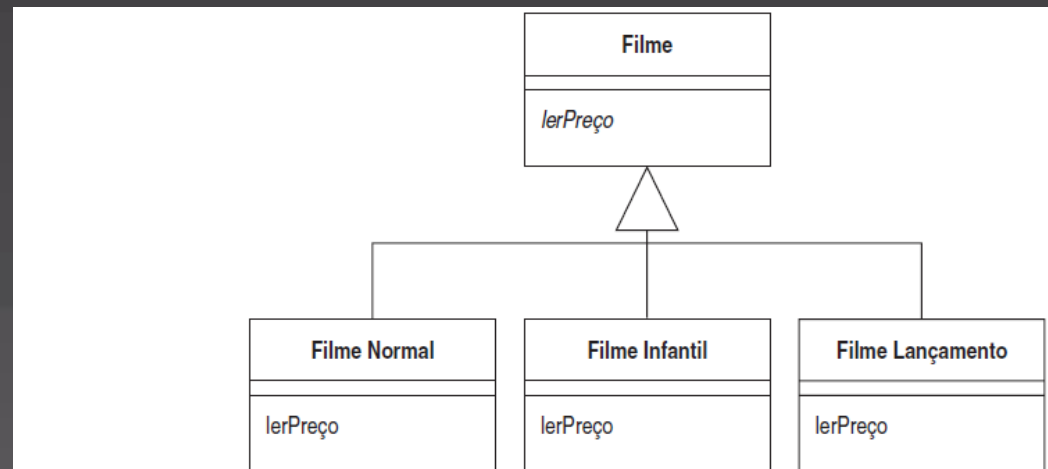


Figura 1.14 Usando herança na classe Filme.

Exercício Extra #2 - Herança

```
public abstract class Filme { ..  
    public static final int NORMAL = 0; // R$ 2,00  
    public static final int LANÇAMENTO_NOVO = 1; // R$ 3,00  
    public static final int INFANTIL = 2; // R$ 1,50  
  
    private String _titulo;  
    private int _codigoPreco;  
  
    public abstract double lerPreco(int diasAlugados);  
  
    public int lerPontosLocadorFrequente(int numDiasAlugados) {  
        return 1;  
    }  
}
```

Nesta direção, poderíamos eliminar as constantes e atualizar os códigos clientes. Todavia, vamos optar por uma outra direção, portanto manteremos as constantes por enquanto.

Exercício Extra #2 - Herança

```
public class FilmeNormal extends Filme {  
  
    public FilmeNormal(String titulo, int codigoPreco) {  
        super(titulo, codigoPreco);  
    }  
  
    @Override  
    public double lerPreco(int diasAlugados) {  
        double resultado = 0.0;  
        resultado += 2; // R$ 2,00 por filme normal alugado  
        if (diasAlugados > 2) { // Multa de 1.50 por dia de atraso caso seja entregue apos 2 dias  
            resultado += (diasAlugados - 2) * 1.5;  
        }  
        return resultado;  
    }  
}
```

Exercício Extra #2 - Herança

```
public class FilmeLancamento extends Filme { ...
```

```
    @Override
```

```
    public double lerPreco(int diasAlugados) {
```

```
        double resultado = 0.0;
```

```
        resultado += diasAlugados * 3;
```

```
        return resultado;
```

```
    }
```

```
    @Override
```

```
    public int lerPontosLocadorFrequente(int numDiasAlugados) {
```

```
        if (numDiasAlugados > 1) {
```

```
            return 2;
```

```
        } else {
```

```
            return 1;
```

```
        }
```

```
    }
```

```
}
```

Exercício Extra #2 - Herança

```
public class FilmeInfantil extends Filme {  
  
    public FilmeInfantil(String titulo, int codigoPreco) {  
        super(titulo, codigoPreco);  
    }  
  
    public double lerPreco(int diasAlugados) {  
        double resultado = 0;  
        resultado += 1.5; // R$ 1,50 por 3 dias por filme infantil alugado  
        if (diasAlugados > 3) {  
            resultado += (diasAlugados - 3) * 1.5; // Multa de 1,50 por dia de atraso caso seja  
            entregue apos 3 dias.  
        }  
        return resultado;  
    }  
}
```

Compile e Teste !!

Eclipse File Edit Source Refactor Navigate Search Project Run Window Help

workspaceUNIFAL - Refactoring/src/main/java/extra/exercicio2/Filme.java - Eclipse IDE

Package Explorer

- Locacao.java
- TesteCap1HTML.java
- extra.exercicio3
- src/test/java
 - cap1.versao1
 - cap1.versao2.extract_method_rename_variable
 - cap1.versao3.move_method.parte1.mantem_metodo
 - cap1.versao3.move_method.parte2.remover_metodo
 - cap1.versao4.replace_temp_with_query
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - cap1.versao6.replace_temp_with_query_calculo_total_conta.parte1
 - cap1.versao6.replace_temp_with_query_calculo_total_pontos.parte2
 - cap1.versao7.conta.html
 - cap1.versao8.parte1.lerpreco
 - cap1.versao8.parte2.lerpontos.locador.frequente
 - cap1.versao9.parte1.substituir_enum_state_strategy
 - cap1.versao9.parte2.move_method
 - cap1.versao9.parte3.substituir_condicional_por_polimorfismo
 - cap1.versao9.parte4.move_method_locador_frequente
- extra.exercicio1
- extra.exercicio2
 - TestesCap1.java
- extra.exercicio3
- JRE System Library [JavaSE-1.8]
- Maven Dependencies
- src
- target

TestesCap1.java

```
2 * To change this template, choose Tools | Templates
5
6 package extra.exercicio2;
7
8 public abstract class Filme {
9
10     public static final int NORMAL = 0; // R$ 2,00
11     public static final int LANCAMENTO_NOVO = 1; // R$ 3,00
12     public static final int INFANTIL = 2; // R$ 1,50
13
14     private String _titulo;
15     private int _codigoPreco; // codigoPreco contem uma das constantes acima
16
17     public Filme(String titulo, int codigoPreco) {
18         this._titulo = titulo;
19         this._codigoPreco = codigoPreco;
20     }
21
22     public int lerCodigoPreco() {
23         return _codigoPreco;
24     }
25
26     public void gravarCodigoPreco(int codigoPreco) {
```

Outline

- extra.exercicio2
 - Filme
 - NORMAL : int
 - LANCAMENTO_NOVO : int
 - INFANTIL : int
 - _titulo : String
 - _codigoPreco : int
 - Filme(String, int)
 - lerCodigoPreco() : int
 - gravarCodigoPreco(int) : void
 - lerTitulo() : String
 - lerPreco(int) : double
 - lerPontosLocadorFrequente(int) : int

Problems Console Progress Git Repositories Rebase Interactive JUnit Git Reflog Debug Tasks

Finished after 0.041 seconds

Runs: 2/2 Errors: 0 Failures: 0

extra.exercicio2.TestesCap1 [Runner: JUnit 4] (0.003 s)

- testeContaHTML (0.002 s)
- locacaoFilmes (0.001 s)

Failure Trace

Writable Smart Insert 8 : 16

Eclipse File Edit Source Refactor Navigate Search Project Run Window Help

workspaceUNIFAL - Refactoring/src/main/java/extra/exercicio2/Filme.java - Eclipse IDE

Package Explorer

- Locacao.java
- TesteCap1HTML.java
- extra.exercicio3
- src/test/java
 - cap1.versao1
 - cap1.versao2.extract_method_rename_variable
 - cap1.versao3.move_method.parte1.manter_metodo
 - cap1.versao3.move_method.parte2.remover_metodo
 - cap1.versao4.replace_temp_with_query
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - cap1.versao6.replace_temp_with_query_calculo_total_conta.parte1
 - cap1.versao6.replace_temp_with_query_calculo_total_pontos.parte2
 - cap1.versao7.conta.html
 - cap1.versao8.parte1.lerpreco
 - cap1.versao8.parte2.lerpontos.locador.frequente
 - cap1.versao9.parte1.substituir_enum_state_strategy
 - cap1.versao9.parte2.move_method
 - cap1.versao9.parte3.substituir_condicional_por_polimorfismo
 - cap1.versao9.parte4.move_method_locador_frequente
- extra.exercicio1
- extra.exercicio2
 - TestesCap1.java
- extra.exercicio3
- JRE System Library [JavaSE-1.8]
- Maven Dependencies
- src
- target

Outline

- extra.exercicio2
 - Filme
 - NORMAL : int
 - LANCAMENTO_NOVO : int
 - INFANTIL : int
 - _titulo : String
 - _codigoPreco : int
 - Filme(String, int)
 - lerCodigoPreco() : int
 - gravarCodigoPreco(int) : void
 - lerTitulo() : String
 - lerPreco(int) : double
 - lerPontosLocadorFrequente(int) : int

TestesCap1.java

```
19 this._codigoPreco = codigoPreco;
20 }
21
22 public int lerCodigoPreco() {
23     return _codigoPreco;
24 }
25
26 public void gravarCodigoPreco(int codigoPreco) {
27     this._codigoPreco = codigoPreco;
28 }
29
30 public String lerTitulo (){
31     return _titulo;
32 }
33
34 public abstract double lerPreco(int diasAlugados);
35
36 public int lerPontosLocadorFrequente(int numDiasAlugados) {
37     return 1;
38 }
39 }
40
41
```

Problems Console Progress Git Repositories Rebase Interactive JUnit Git Reflog Debug Tasks

Finished after 0.041 seconds

Runs: 2/2 Errors: 0 Failures: 0

extra.exercicio2.TestesCap1 [Runner: JUnit 4] (0.003 s)

- testeContaHTML (0.002 s)
- locacaoFilmes (0.001 s)

Writable Smart Insert 34 : 20

Eclipse File Edit Source Refactor Navigate Search Project Run Window Help

workspaceUNIFAL - Refactoring/src/main/java/extra/exercicio2/FilmeLancamento.java - Eclipse IDE

Package Explorer

- Locacao.java
- TesteCap1HTML.java
- extra.exercicio3
- src/test/java
 - cap1.versao1
 - cap1.versao2.extract_method_rename_variable
 - cap1.versao3.move_method.parte1.manter_metodo
 - cap1.versao3.move_method.parte2.remover_metodo
 - cap1.versao4.replace_temp_with_query
 - cap1.versao5.extract_method_locatario_frequente.parte1
 - cap1.versao5.move_method_locatario_frequente.parte2
 - cap1.versao6.replace_temp_with_query_calculo_total_conta.parte1
 - cap1.versao6.replace_temp_with_query_calculo_total_pontos.parte2
 - cap1.versao7.conta.html
 - cap1.versao8.parte1.lerpreco
 - cap1.versao8.parte2.lerpontos.locador.frequente
 - cap1.versao9.parte1.substituir_enum_state_strategy
 - cap1.versao9.parte2.move_method
 - cap1.versao9.parte3.substituir_condicional_por_polimorfismo
 - cap1.versao9.parte4.move_method_locador_frequente
 - extra.exercicio1
 - extra.exercicio2
 - TestesCap1.java
 - extra.exercicio3
- JRE System Library [JavaSE-1.8]
- Maven Dependencies
- src
- target

Outline

- extra.exercicio2
 - FilmeLancamento
 - FilmeLancamento(String, int)
 - lerPreco(int) : double
 - lerPontosLocadorFrequente(int) : int

TestesCap1.java

```
1 package extra.exercicio2;
2
3 public class FilmeLancamento extends Filme {
4
5     public FilmeLancamento(String titulo, int codigoPreco) {
6         super(titulo, codigoPreco);
7     }
8
9     public double lerPreco(int diasAlugados) {
10         double resultado = 0;
11         resultado += diasAlugados * 3; // R$ 3,00 a diaria para lancamentos por filme alugado
12         return resultado;
13     }
14
15     public int lerPontosLocadorFrequente(int numDiasAlugados) {
16         // Ponto adicional para o locatario para cada filme lancamento
17         if ((lerCodigoPreco() == Filme.LANCAMENTO_NOVO)
18             && numDiasAlugados > 1) {
19             return 2;
20         }
21         return 1;
22     }
23 }
```

Problems Console Progress Git Repositories Rebase Interactive JUnit Git Reflog Debug Tasks

Finished after 0.041 seconds

Runs: 2/2 Errors: 0 Failures: 0

extra.exercicio2.TestesCap1 [Runner: JUnit 4] (0.003 s)

- testeContaHTML (0.002 s)
- locacaoFilmes (0.001 s)

Writable Smart Insert 22 : 6

Exercício Extra #3

- Embora o design anterior não inviabilize a solução, existe uma forma, ao utilizá-lo, de mudar a classificação de um **Filme** em tempo de execução?
 - Por exemplo, um objeto **FilmeNormal** se Tornar **FilmeInfantil**
 - Reflita e tente implementar um pequeno programa que tente fazer esta alteração (20 min)

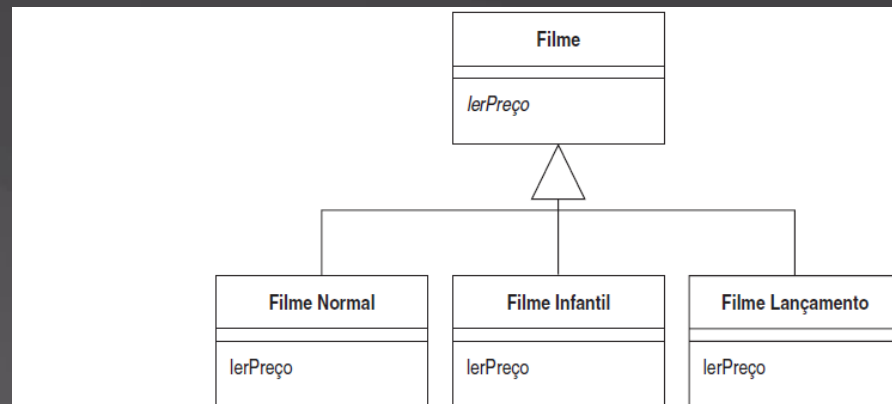


Figura 1.14 Usando herança na classe Filme.

Segundo Exercício Extra

```
public class TesteMudancaClasse {  
    public static void main(String[] args) {  
        // criando um Filme normal  
        Filme filme = new FilmeNormal("127 horas", 0);  
        System.out.println(filme); // toString sobreescrito para exibir Filme Normal  
  
        // como mudar o filme para outra classificação??  
    }  
}
```

Neste Capítulo

- Vimos um programa mal projetado sendo aperfeiçoado.
 - Mais fácil adicionar novos serviços para os clientes.
 - Mais fácil adicionar novos tipos de filmes.
- Nenhuma depuração durante a refatoração.
 - Passos apropriados reduzem as chances de bug.
 - Pequenos passos tornam os bugs fáceis de serem encontrados.

Neste Capítulo

- Ilustramos várias refatorações:
 - ✓ Extrair Método.
 - ✓ Mover Método.
 - ✓ Substituir Variável Temporária com Consulta.
 - ✓ Substituir Enumeração pelo Padrão State/Strategy.
 - ✓ Substituir Código Condicional por Polimorfismo.

Princípios da Refatoração

Mini-Agenda

- Definição de Refatoração.
- Por que devemos refatorar?
- Quando devemos refatorar?
- O que eu digo ao meu gerente?
- Problemas com refatoração.
- Quando não refatorar?
- Refatoração e a disciplina de Design.
- Refatoração e Desempenho.
- De onde veio a refatoração?

Refatoração (substantivo)

“Uma alteração feita na estrutura interna do Software para torná-lo fácil de entender e menos custoso para mudança sem mudar o seu comportamento observável.”

Refatorar (verbo)

“Reestruturar software aplicando uma série de refatorações sem alterar seu comportamento observável.”

Definição de Refatoração

- Refactoring não é uma bala de prata.
- Refactoring não é uma nova tecnologia.
- Refactoring não é uma nova metodologia.
- Refactoring não é uma nova tendência na Internet.

É mais como um novo hábito que adquirimos !!

Definição de Refatoração

Duas tarefas principais no desenvolvimento de software:

- Necessidade de se criar novas funcionalidades
 - Ponto de evolução no software.
- Necessidade de modificar o software.
 - Ponto de variação no software.

A refatoração afeta a maneira que você aborda estas duas tarefas.

Por que devemos refatorar?

- Para:
 - Melhorar o Design do Software.
 - Tornar o software mais fácil de entender.
 - Ajudar a encontrar falhas.
 - Ajudar a programar mais rapidamente.

Quando devemos refatorar?

A regra dos três:

3 vezes e você refatora

- 1 - Necessitou de uma nova funcionalidade?
 - Apenas faça !
- 2 - Segunda vez que faz algo parecido?
 - Faça novamente, mas comece a refletir.
- 3 - Necessitou de uma funcionalidade similar pela terceira vez?
 - **REFATORE!**

Quando devemos refatorar?

Refatore quando:

- Acrescentar funcionalidade.
- Precisar consertar uma falha.
- Enquanto revisa o código.

Quando devemos refatorar?

- Lembre-se do conceito de **Propriedade Coletiva do Código** bastante discutida no método ágil **Extreme Programming** - XP
 - Ninguém é dono de um código específico.
 - Um código pode ter um criador, mas o criador não o possui.
- Grupo, Time e pensamento coletivo é importante !

Quando devemos refatorar?

- Tenha em mente o que programas podem fazer por você hoje e o que eles podem fazer por você amanhã.
- Todo software evolui.
 - A técnica de refatoração permite tornar a manutenção menos trabalhosa.

O que eu digo ao meu gerente?

- Como explicar e justificar o ato de refatorar?
 - Nenhum código novo será criado.
 - Não há nenhum requisito novo.

O que eu digo ao meu gerente?

Você pode:

- **Mencionar por alto** o uso da refatoração.
- **Mostrar os benefícios** para a qualidade do produto como um todo.
 - Existem diversos artigos publicados mostrando que inspeções aumentam a produtividade da equipe.
- **Não diga !**

O que eu digo ao meu gerente?

- Alguns argumentos que podem te auxiliar:
 - Programas **difíceis de serem lidos** são difíceis de serem modificados.
 - Programas com **lógica duplicada** são difíceis de serem modificados.
 - Apesar de parecer contra-intuitivo, estudos mostram que a **refatoração aumenta a produtividade**.

O que eu digo ao meu chefe?

Refatoração é um investimento com retorno em um futuro próximo.

Problemas com a refatoração

- Quebra de código.
 - Podemos quebrar o que está funcionando bem.
- Bancos de Dados
 - Difíceis de modificar e migrar
 - Uma camada de integração pode amenizar o problema.
- Alteração de Interfaces.
 - Interfaces publicadas e não-publicadas.

NÃO PUBLIQUE INTERFACES PREMATURAMENTE !

Refatoração e a disciplina de Design

- A refatoração aperfeiçoa o **design** de código após ele ter sido escrito.
- A refatoração é o oposto da prática tradicional de criar o design e depois implementar.
 - Apesar de um desenho mínimo poder ser feito antes do código:
 - Análise CRC, Princípios GRASP, princípios SOLID, etc.

Refatoração e a Disciplina de Design

- Com a refatoração podemos a partir de um design ruim, transformá-lo em um código bem projetado.
- Com a refatoração, o design em vez de ser construído à priori, ocorre continuamente durante o desenvolvimento.

Refatoração e desempenho

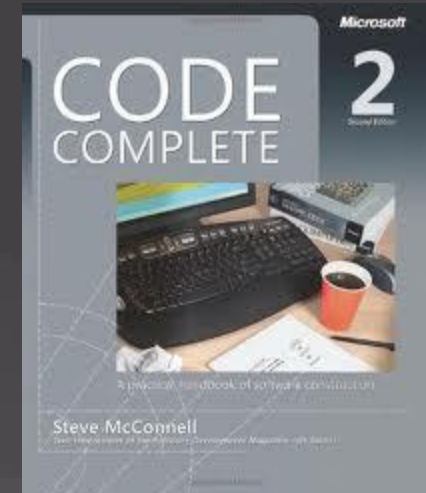
A melhor maneira de otimizar o desempenho é primeiro escrever um programa bem fatorado e, então, otimizá-lo.

Refatoração e desempenho

- A maioria do tempo de execução de um programa está localizada em uma parte pequena do código
- Use um “Profiler” para encontrar estes gargalos
 - Muito difíceis de serem encontradas pelo olho humano
- Otimize os gargalos e meça a melhoria.

Refatoração e desempenho

McConnell, S. Code Complete: A Practical Handbook of Software Construction. Microsoft Press, 1993



De onde veio a refatoração?

- Tese de doutorado de Bill Opdyke.
 - Autor do capítulo 13 do livro texto.
 - Principal referência sobre o assunto.
- Ward Cunningham, Kent Beck e Ralph Johnson.
 - Ralph foi o orientador de Bill Opdyke na Universidade de Illinois em Urbana-Champaign.

Os 22 Maus Cheiros no Código

Mini-Agenda

1. Código Duplicado.
2. Método Longo.
3. Classes Grandes.
4. Lista de Parâmetros Longa.
5. Alteração Divergente.
6. Cirurgia com Rifle.
7. Inveja dos Dados.
8. Grupo de Dados.
9. Obsessão Primitiva.

Mini-Agenda (Cont.)

10. Comandos Switch.
11. Hierarquias Paralelas de Herança.
12. Classe Ociosa.
13. Generalidade Especulativa.
14. Campo Temporário.
15. Cadeias de Mensagens.
16. Intermediário.
17. Intimidade Inadequada.
18. Classes Alternativas com Interfaces Diferentes.

Mini-Agenda (Cont.)

- 19. Biblioteca de Classes Incompleta.
- 20. Classes de Dados.
- 21. Herança Recusada.
- 22. Comentários.
- 23. Tabela com Maus Cheiros e Possíveis Refatorações.

Quando Refatorar?

Os 22 Maus Cheiros no Código

- Decidir **quando iniciar e quando parar** a refatoração é tão importante quanto à sua mecânica.
- Porém, nenhum conjunto de métricas pode se comparar à **intuição** humana.

Os 22 Maus Cheiros no Código

- **Maus Cheiros:**
 - Indícios de problemas que podem ser resolvidos com refatoração.
- Relacionados à problemas de **entendimento** do código e não de lógica, erro de compilação, etc.

Indícios não são regras !

Os 22 Maus Cheiros no Código

Sugestão

Use os **22 maus cheiros** descritos e a tabela apresentada no final dos slides deste capítulo quando não estiver certo sobre a **refatoração** a fazer.

1 - Código Duplicado

- **Número 1** no ranking dos maus cheiros
- Problema
 - Mesmo código em mais de um lugar.
- Sugestão:
 - Unifique-os.

1 - Código Duplicado (Cont.)

- O código duplicado se manifesta de diversas formas.
- Exemplos:
 - Mesma expressão em dois métodos da mesma classe.
 - Mesma expressão em duas subclasses.

2 – Método Longo

- Os programas orientados a objetos que vivem melhor e por mais tempo são aqueles com **métodos curtos**.
- Programas orientados a objetos parecem sequencias intermináveis de **delegações**.
- O segredo para o entendimento de métodos pequenos é a **boa nomeação**.
 - Se você tiver um bom nome para um método, não precisa olhar o seu corpo.

2 – Método Longo (Cont.)

- **Conclusão:**
 - Devemos ser mais agressivos na **decomposição de métodos**.
- Como identificamos trechos de métodos a serem extraídos?
 - Comentários, **condições**, laços
- Vale a pena extrair até uma única linha se ela necessitar de explicação.

3 - Classes Grandes

- **Problema:**
 - Uma classe está tentando fazer muita coisa.
- **Sugestão:**
 - Elimine redundância na própria classe.
 - criando novas classes,
 - mudando alguns métodos para outras classes
 - etc.

4 - Lista de Parâmetros Longa

- **Problema:**
 - Passar como parâmetro tudo que é necessário ao método.
- **Sugestão:**
 - Tente substituir alguns parâmetros por objetos em vez de tipos primitivos de dados.

5 – Alteração Divergente

- **Problema:**
 - Uma classe é frequentemente alterada de diversas maneiras por diferentes motivos.
- **Solução:**
 - identifique tudo que muda por uma causa particular e os coloque juntos em um método, por exemplo.

6 - Cirurgia com Rifle

- **Problema:**
 - Você tem que fazer uma porção de pequenas mudanças em um conjunto enorme de classes.
 - Oposto de Alteração Divergente.
- **Sugestão:**
 - Insira todas as mudanças em uma única classe (talvez uma nova classe)

7 - Inveja dos Dados

- **Problema:**
 - Um método usa muitos dados de outra classe.
- **Sugestão:**
 - Tente avaliar onde realmente o método deveria estar.

8 - Grupos de Dados

- **Problema:**

- Mesma quantidade de itens de dados juntos em vários lugares.
- Atributos em um conjunto de classes, parâmetros em muitas assinaturas de métodos.

- **Sugestão:**

- Tente criar uma nova classe com o conjunto de dados que é utilizado em conjunto.

9 - Obsessão Primitiva

- **Problema:**
 - Uso excessivo de tipos de dados primitivos.
- **Sugestão:**
 - Talvez o tipo primitivo possa ser substituído por um novo **objeto** ou um **struct**.

“Iniciantes em objetos normalmente ficam relutantes em usar pequenos objetos para pequenas tarefas, tais como **classes dinheiro**, intervalos e **strings especiais** como telefone”.

10 - Comandos Switch

- **Problema:**

- Duplicação da mesma **instrução switch** em mais de um local.

- **Sugestão:**

- Considere o uso do **polimorfismo** para centralizar as instruções switch.

“Um dos fenômenos mais óbvios de código orientado a objetos é a quase completa ausência de comandos switch.”

11 - Hierarquias Paralelas de Herança

- **Problema:**
 - Toda vez que você cria uma subclasse, você também precisar criar uma subclasse na outra hierarquia.
- **Sugestão:**
 - Certifique-se que instâncias de uma hierarquia se referem a instância da outra

12 - Classe Ociosa

- **Problema:**
 - Uma classe não está fazendo muita coisa.
- **Sugestão:**
 - Elimine-a.

13 - Generalidade Especulativa

- **Problema:**

- “Eu acho que precisaremos disto algum dia”.
- Cria-se todos os tipos de ganchos e casos especiais para lidar com coisas que não são exigidas.

- **Sugestão:**

- Se toda esta parafernália estivesse sendo usada, valeria a pena, mas se não estiver, não vale.

14 - Campo Temporário

- **Problema:**
 - Um objeto em que uma variável de instância é atribuída somente em certas circunstâncias.
- **Sugestão:**
 - Tente colocar todas as variáveis em outra classe em vez de usar atributos opcionais.

15 - Cadeias de Mensagens

- **Problema:**
 - Um cliente solicita a um outro objeto por outro objeto e por mais outro objeto, etc.
 - Difícil de entender além de fornecer alto acoplamento.
- **Exemplo:**
 - objeto.**getX()**.metodo1().**getY()**.fazAlgo().
- **Sugestão:**
 - Tente adicionar métodos e variáveis para remover este tipo de corrente, alterando o *design* do seu código.

16 - Intermediário

- **Problema:**
 - Excesso de **delegação**.
 - Você olha a interface de uma classe e descobre que metade dos métodos delegam para outra classe.
 - **Sugestão:**
 - Eliminar o intermediário e fornecer mais responsabilidade para a classe.
- “O encapsulamento, uma das características básicas de orientação a objetos, normalmente vem junto com delegação”.

17- Intimidade Inadequada

- **Problema:**
 - Classes com muito conhecimento sobre outras classes.
 - Acontece também entre classes e subclasses. Cria-se alto acoplamento.
- **Sugestão:**
 - Separar todo o acesso direto em mais classes ou colocar os interesses em comum em outro lugar.

18 - Classes Alternativas com Interfaces Diferentes

- **Problema:**
 - Métodos que fazem a mesma coisa mas possuem diferentes assinaturas.
- **Sugestão:**
 - Avalie o uso da sobrecarga e tente renomear os métodos ou classes.
 - Tente também refatorar o código duplicado.

19 - Biblioteca de Classes Incompleta

- **Problema**
 - Como introduzir novas funcionalidades em um biblioteca existente a altamente utilizada?
- **Solução**
 - Existem refatorações para alterar uma biblioteca existente ou componente para introduzir novas funcionalidades sem ter que redesenhar toda a biblioteca.
 - **Introduzir Método Externo e Introduzir Extensão Local**

20- Classes de Dados

- **Problema**

- Classes que possuem atributos, métodos get/set e nada a mais.
- Também conhecidas como **POJO** (Plain Old Java Object).
- Apenas repositório de dados.

- **Solução**

- Tente identificar novas responsabilidades para a classe.

21- Herança Recusada

- **Problema (1)**

- Uma subclasse que não quer ou precisa tudo que origina na superclasse.
- Algumas pessoas dizem que você deveria criar uma **classe irmã** em vez de uma **filha**

- **Sugestão**

- Tente avaliar se os métodos realmente precisam estar na superclasse ou se eles deveriam estar na subclasse.
- Normalmente não é um cheiro muito forte quanto ao sentido no problema 2 (próximo slide)

21- Herança Recusada (Cont.)

- **Problema (2)**
 - Uma subclasse está reutilizando comportamento mas não quer dar suporte à interface da superclasse.
- **Sugestão**
 - Avalie o uso de **Substituir Herança por Delegação**.

22 - Comentários

- **Problema**
 - Em alguns casos, comentários são escritos devido a código ruim.
- **Solução**
 - Você pode usar comentários para localizar problemas tais como **código duplicado**, classe longa, **lista de parâmetros longa**, etc.
 - Os **comentários** podem indicar onde iniciar a procura por locais a serem refatorados.

Maus cheiros e possíveis refatorações

Cheiro	Refatorações Comuns
Alteração Divergente	Extraír Classe
Biblioteca de Classes Incompleta	Introduzir Método Externo, Introduzir Extensão Local
Cadeias de Mensagens	Ocultar Delegação
Campo Temporário	Extraír Classe, Introduzir Objeto Nulo
Cirurgia com Rifle	Mover Método, Mover Campo, Internalizar Classe
Classe Ociosa	Internalizar Classe, Condensar Hierarquia
Classes Alternativas com Interfaces Diferentes	Renomear Método, Mover Método
Classes de Dados	Mover Método, Encapsular Campo, Encapsular Coleção

Maus cheiros e possíveis refatorações

Cheiro	Refatorações Comuns
Classes Grandes	Extrair Classe, Extrair Subclasse, Extrair Interface, Substituir Atributo por Objeto
Código Duplicado	Extrair Método, Extrair Classe, Substituir Método na Hierarquia, Criar um Método Padrão
Comandos Switch	Substituir Comando Condicional por Polimorfismo, Substituir Enumeração por Subclasses, Substituir Enumeração pelo Padrão State/Strategy, Substituir Parâmetros por Métodos Explícitos, Introduzir Objeto Nulo.
Comentários	Extrair Método, Introduzir Asserção.
Generalidade Especulativa	Condensar Hierarquia, Internalizar Classe, Remover Parâmetro, Renomear Método

Maus cheiros e possíveis refatorações

Cheiro	Refatorações Comuns
Grupos de Dados	Extraír Classe, Introduzir Objeto Parâmetro, Preservar o Objeto Inteiro
Herança Recusada	Substituir Herança por Delegação
Hierarquias Paralelas de Herança	Mover Método, Mover Campo
Intermediário	Remover Intermediário, Internalizar Método, Substituir Delegação por Herança
Intimidade Inadequada	Mover Método, Mover Campo, Transformar Associação Bidirecional em Unidirecional, Substituir Herança por Delegação, Ocultar Delegação
Inveja dos Dados	Mover Método, Mover Campo, Extraír Método

Maus cheiros e possíveis refatorações

Cheiro	Refatorações Comuns
Lista de Parâmetros Longa	Substituir Parâmetro por Método, Introduzir Objeto Parâmetro, Preservar Objeto Inteiro
Método Longo	Extrair Método, Substituir Variável Temporária por Consulta, Substituir Método por Objeto Método, Decompor Condicional
Obsessão Primitiva	Substituir Atributo por Objeto, Extrair Classe, Introduzir Objeto Parâmetro, Substituir Vetor por Objeto, Substituir Enumeração por Subclasses, Substituir Enumeração pelo Padrão State/Strategy

Construindo Testes

Mini-Agenda

- O valor do código auto-testável.
- Testes e o método ágil XP.
- Testes e produtividade.
- Um exemplo com o framework de testes JUnit
- Testes com erros.
- Os métodos `assertTrue` e `assertEquals`.
- JUnit e exceções.
- Teste funcionais e de unidade.
- Boas práticas.
- Refatoração e Testes.

O Valor do Código Auto-Testável

- Ter testes sólidos é pré-condição para refatorar.
- Assegure-se de que todos os testes sejam totalmente automatizados e verifiquem seus próprios resultados.
- Um conjunto de testes é um detector poderoso de falhas que diminui o tempo que se leva para encontrá-las.

O Valor do Código Auto-Testável

- É muito comum na prática termos um código que não seja auto-testável que gostaríamos de refatorar
 - Primeiro tornamos o código auto-testável para depois refatorarmos.

Testes e o Método Ágil XP

- De acordo com o método ágil “*Extreme Programming – XP*”, o momento mais útil para se escrever testes é antes de começar a programar.
 - Programadores extremos são testadores muito dedicados.
- Ao escrever o teste você está questionando sobre o que precisa ser feito
 - Concentra sua atenção na interface e não na implementação

Testes e Produtividade

- A maior parte do tempo de um programador é gasta depurando código
- Quando criamos testes automatizados se acrescentarmos uma falha no código ela é pega tão logo que rodamos os testes novamente.
 - Diminui-se bastante o tempo de depuração
- Resultado:
 - Escrever bons testes aumenta muito a velocidade de programação.

Testes e Produtividade

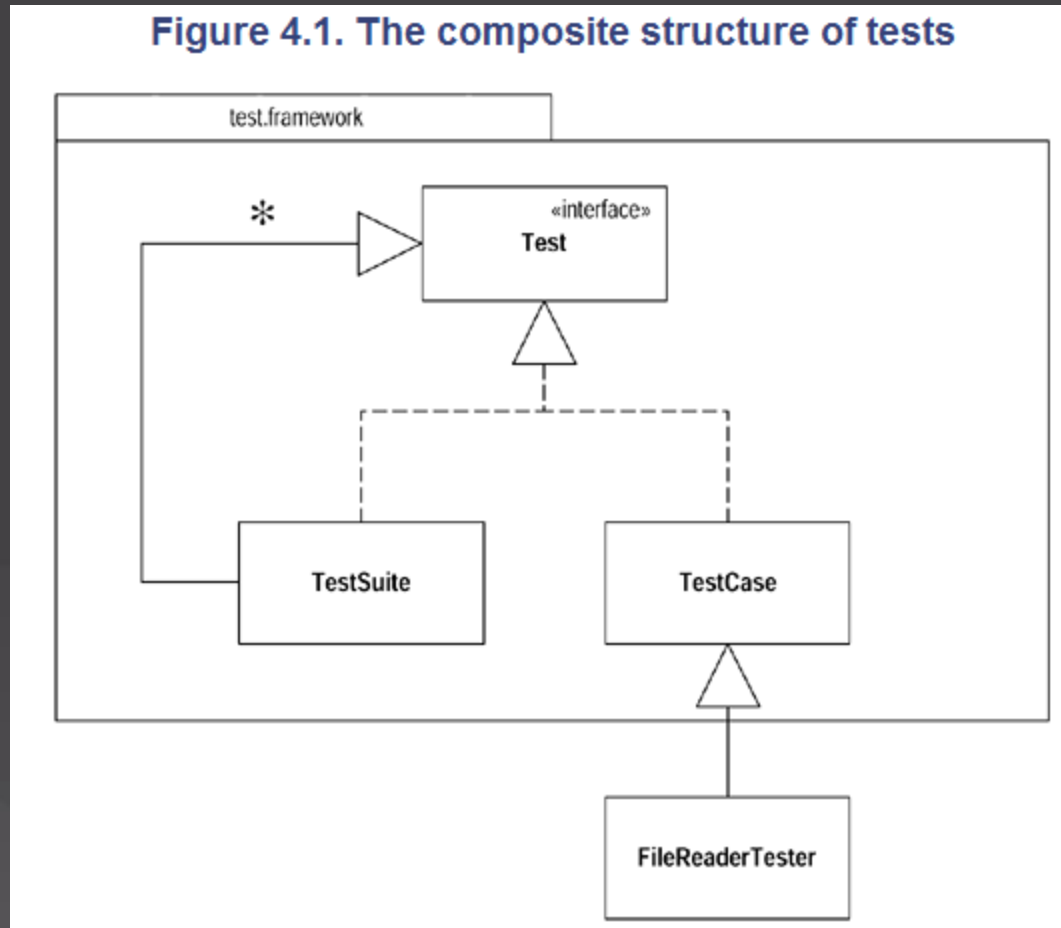
- Escrever testes significa escrever muito mais código
 - Difícil persuadir os outros a seguirem este caminho
- Lembre-se que o fato de escrever mais código não quer dizer que o programador será menos produtivo.
 - Com testes, o contrário é verdade

Um Exemplo com o Framework de Testes JUnit

- JUnit é um framework de testes de código aberto
 - Desenvolvido por Erich Gamma e Kent Beck
- O framework usa o **Design Pattern Composite** [GOF] que permite agrupar testes em suítes

Um Exemplo com o Framework de Testes JUnit

Figure 4.1. The composite structure of tests



Obs: No JUnit 4.x não precisamos herdar de **TestCase**, mas a estrutura de Composite é similar

Exemplo: Leitura de um Arquivo

- Inicialmente, criamos a infra-estrutura de testes (Fixture)
 - Uma fixture é essencialmente os objetos que atuam como exemplos para os testes
- Para nosso exemplo, um arquivo de texto.
 - Arquivo data.txt com 177 caracteres

```
Bradman 99.94 52 80 10 6996 334 29
Pollock 60.97 23 41 4 2256 274 7
Headley 60.83 22 40 4 2256 270 10
Sutcliffe 60.73 54 84 9 4555 194 16
```

Exemplo: O Método setup

```
public class TesteDeLeituraDeArquivo {
```

```
    private BufferedReader _entrada;
```

```
    private BufferedReader _vazio;
```

```
    private static final int NUMERO_CARACTERES = 177;
```

Arquivos no pacote
default

@Before

```
public void setUp() {
```

```
    _entrada = new BufferedReader(new
```

```
        InputStreamReader(getClass().getResourceAsStream("/data.txt")));
```

```
    _vazio = new BufferedReader(new
```

```
        InputStreamReader(getClass().getResourceAsStream("/empty.txt")));
```

```
}
```



Exemplo: O Método `tearDown()`

- Na maioria das vezes, você não precisará do método `tearDown`
 - Deixando a responsabilidade para o `Garbage Collector`
- No nosso caso, trata-mos isso explicitamente, fechando os arquivos
 - Próximo slide

Exemplo: O Método tearDown()

@After

```
public void tearDown() {  
    try {  
        if (_entrada != null) {  
            _entrada.close();  
        }  
  
        if (_vazio != null) {  
            _vazio.close();  
        }  
    } catch (IOException i) {  
        throw new RuntimeException("Erro ao fechar algum dos arquivos de  
teste");  
    }  
}
```

Exemplo: Teste para Leitura

...

...

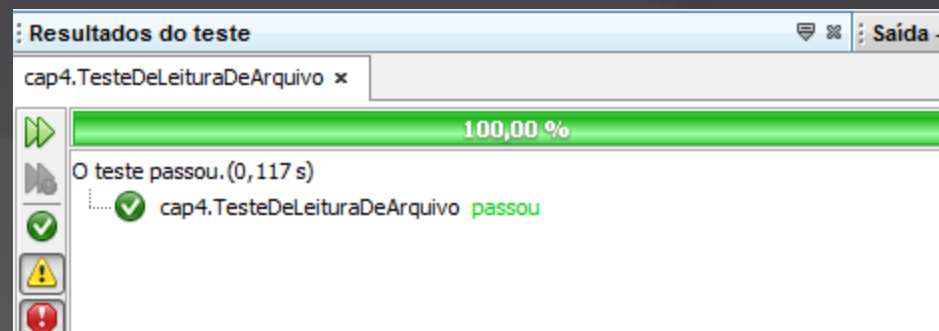
@Test

```
public void testeDeLeitura() throws IOException {  
    char ch = '&';  
    for (int i = 0; i < 4; i++)  
        ch = (char) _entrada.read();  
    assertTrue('d' == ch);  
}
```

....

....

}



Execute os seus testes frequentemente.

Testes com Erros

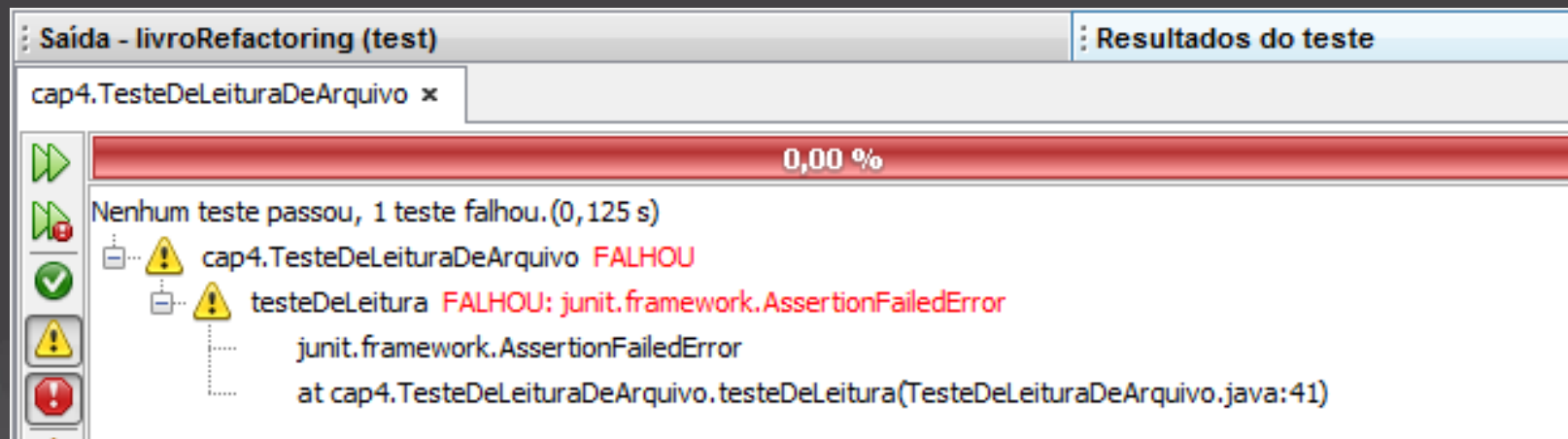
- O que acontece quando algo dá errado ao executarmos um teste automatizado?

- Exemplo

@Test

```
public void testeDeLeitura() throws IOException {  
    char ch = '&';  
    for (int i = 0; i < 4; i++)  
        ch = (char) _entrada.read();  
    assertTrue('2' == ch); // ERRO PROPOSITAL  
}
```

Testes com Erros



Os Métodos assertTrue e assertEquals

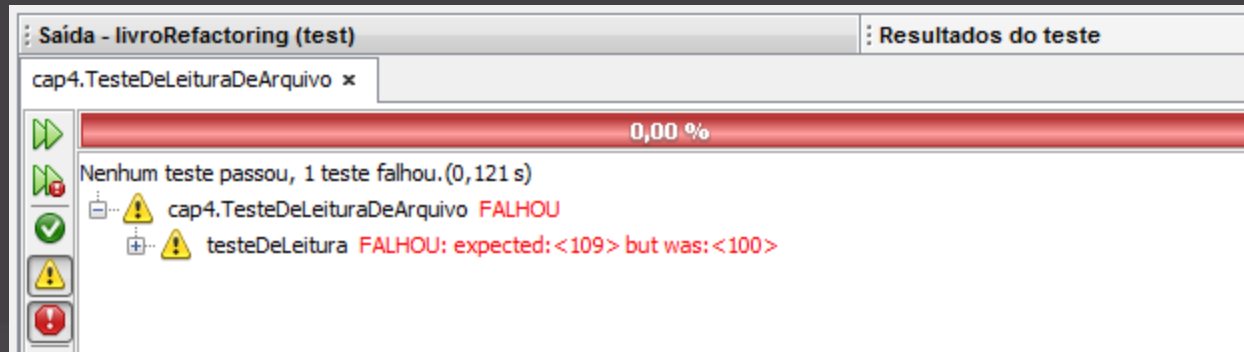
- A mensagem de erro anterior não ajuda muito
- Isso pode ser melhorado usando uma outra forma do método assert

@Test

```
public void testeDeLeitura() throws IOException {  
    char ch = '&';  
    for (int i = 0; i < 4; i++)  
        ch = (char) _entrada.read();  
    assertEquals('m', ch);  
}
```

Os Métodos assertTrue e assertEquals

Exercício: mudar para imprimir o char



- assertEquals usa automaticamente equals() em objeto e == em valores primitivos

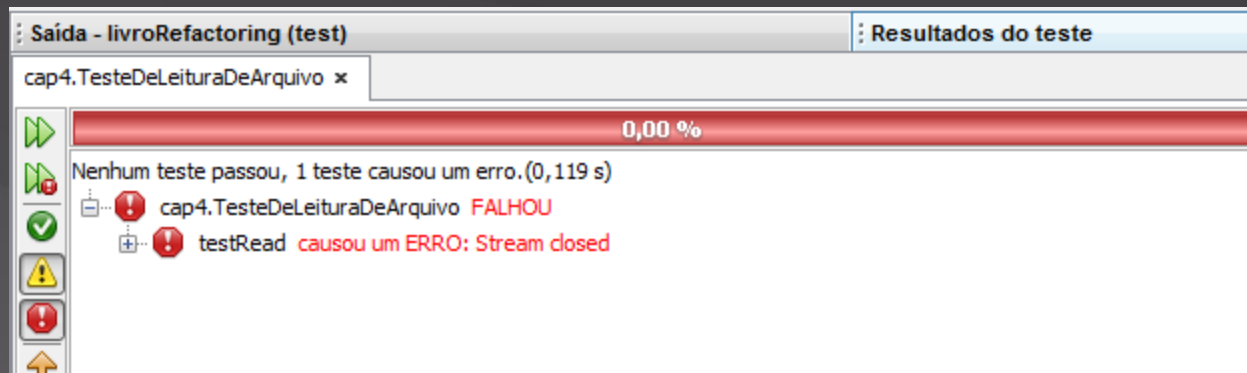
JUnit e Exceções

- Falhas
 - asserções com resultado falso
- Erros
 - exceções não esperadas
- O framework captura ambos.
 - Veja o próximo slide

JUnit e Exceções

@Test

```
public void testeDeLeitura() throws IOException {  
    char ch = '&';  
    _entrada.close();  
    for (int i = 0; i < 4; i++) {  
        ch = (char) _entrada.read();    // Lançará uma exceção  
    }  
    assertEquals('m', ch);  
}
```



JUnit e Exceções

@Test

```
public void testeLeituraArquivoFechado() {  
    try {  
        _entrada.close();  
        _entrada.read();  
        fail("o método read deve gerar uma exceção");  
    } catch (IOException i) {  
        assertTrue(true);  
    }  
}
```

- Observe que neste caso, o código lançar uma exceção é algo bom
 - O comportamento esperado

Testes Unitários e Testes Funcionais

- JUnit é usado para testes unitários
 - Escritos para melhorar a produtividade do programador.
- Testes unitários são altamente localizados
 - Cada classe de teste trabalha dentro de um único pacote
 - Ela testa a interface com outros pacotes, mas além disso, supõe-se que o resto funciona.

Testes Unitários e Testes Funcionais

- Testes Funcionais são diferentes de Testes Unitários
 - São escritos para assegurar que o software como um todo funciona.
 - Fornecem garantia de qualidade para o usuário sem se importar com a produtividade do programador.

Testes Unitários e Testes Funcionais

- Testes Funcionais devem ser desenvolvidos por um time diferente dos programadores.
 - Pessoas que gostam de procurar e encontrar falhas.
 - Estas pessoas usam técnicas e ferramentas específicas para realização de suas tarefas.
- Testes funcionais tipicamente tratam o sistema como um todo como sendo uma caixa preta

Testes Unitários e Testes Funcionais

- Quando testadores funcionais ou usuários encontram uma falha no software, duas coisas são necessárias
 - Alterar o código de produção
 - Criar um teste unitário que expõe a falha.

Testes Unitários e Testes Funcionais

- Podemos executar Testes Funcionais com JUnit
 - Porém, normalmente esta não é a forma mais eficiente.
- No contexto da refatoração, devemos contar com Testes Unitários já que os mesmos são escritos pelos programadores.

Adicionando novos testes

- Cuidado com a sugestão:

“Teste todo método público”

- Testes devem ser dirigidos a risco.
 - Normalmente não testamos métodos get/set

Adicionando novos testes

- Tentar escrever muitos testes normalmente nos leva a não escrever o bastante .
- O segredo é testar áreas que você acredita que possa dar errado.

É melhor escrever e executar testes incompletos do que não executar testes completos.

Adicionando novos testes

- Um truque chave com testes é procurar por condições de fronteira.
- Para leitura, as fronteiras devem ser o primeiro caracter, o último e o caractere após o último

Exemplo: Adicionando novos testes

@Test

```
public void testeDeLeituraAoFinal() throws IOException {  
    int ch = -1234;  
    for (int i = 0; i <= NUMERO_CARACTERES; i++) {  
        ch = _entrada.read();  
    }  
    assertEquals(-1, ch);  
}
```

Retorna -1 caso seja o final do arquivo

Exemplo: Adicionando novos testes

@Test

```
public void testeLeituraFronteiras() throws IOException {  
    assertEquals("Leitura do primeiro caracter", 'B', _entrada.read());  
    int ch;  
    for (int i = 1; i < NUMERO_CARACTERES - 1; i++) {  
        ch = _entrada.read();  
    }  
    assertEquals("Leitura do último caracter", '6', _entrada.read());  
    assertEquals("Leitura no final do arquivo", -1, _entrada.read());  
    assertEquals("Leitura após o final do arquivo", -1, _entrada.read());  
}
```

- Observe que podemos adicionar uma mensagem na instrução `assert` que é impressa caso o teste falhe.

“Pense sobre as condições de fronteira sob as quais as coisas podem dar errado para concentrar seus testes”.

Exemplo: Adicionando novos testes

- Outra questão sobre procurar por condições de fronteira é procurar por condições especiais que podem causar o teste falhar.
- Arquivos vazios são sempre uma boa escolha:

`@Test`

```
public void testeLeituraArquivoVazio() throws IOException {  
    assertEquals(-1, _vazio.read());  
}
```

Boas Práticas

- Observe que ao escrever testes, atuamos como inimigo do código
 - Pensando sobre como podemos quebrá-lo
- Não se esqueça de testar se as exceções são geradas quando é esperado que algo dê errado.
 - Se você tentar ler um stream após fechá-lo, você deve obter uma `IOException`

Boas Práticas

- Não deixe que o medo de que os testes não consigam capturar todos os erros impeça você de escrever os testes que irão capturar a maior parte dos erros.
- Escreva os testes antes de programar
- Tente integrar a suite de testes com o processo de build
 - Exemplos: Ant e Maven

Boas Práticas

- Diagramas de Sequencia UML também auxiliam na construção de testes unitários
- Para a refatoração, é uma boa idéia escrever pelo menos um teste para cada classe.
- Não se esqueça de ter um backup
 - Pode ser feito automaticamente com softwares de controle de versão como o Subversion (SVN) ou GIT

Refatoração e Testes

- Refatorar requer testes.
 - Se você quiser refatorar, terá de escrever testes.
- A refatoração muda o programa em pequenos passos.
 - Se você cometer um engano, é fácil encontrar a falha.

Exercício

- Usamos nestes slides as anotações `@After` e `@Before` nos métodos `tearDown()` e `setup()`
 - O que aconteceria se usássemos, em vez, as anotações `@AfterClass` e `@BeforeClass`?

Exercício

- Tutorial do JUnit no Netbeans.

http://netbeans.org/kb/docs/java/junit-intro_pt_BR.html